
AI4OS Documentation Documentation

AI4OS consortium

Mar 13, 2026

START HERE

1	Quickstart Guide	3
2	Access the platform	5
3	Platform overview	9
4	New features	13
5	Try a model	15
6	Develop a model	21
7	Train a model	53
8	Deploy a model in production	71
9	Create an AI Inference pipeline	109
10	Dashboard	129
11	Storage	143
12	AI Modules	147
13	AI4EOSC LLM	149
14	Environmental footprint of AI	163
15	Access Policies	167
16	Support	169
17	Frequently Asked Questions (FAQ)	171
18	Glossary	177
19	External links and resources	181
20	Useful Machine Learning resources	183
21	Video demos	189
22	DEEPaaS API	191

23	How to use Nextcloud with rclone	195
24	Rclone code examples	199
25	Support a new project	203
26	How to organize a tutorial	205
27	Storage providers	207

The AI4EOSC platform and the AI4OS stack empowers scientist by lowering the barrier to adopt the latest AI models and tools. The platform provides a user-friendly workbench to easily train, deploy, share and monitor AI models. The stack provides a set of tools and services to easily create and deploy AI models.

It covers the full ML cycle: from model creation, to training, deployment and monitoring in production. Following the FAIR principles for science, both our software (platform and models) are fully open-source and easily portable to any other platform (no vendor lock-in!).

The platform is currently used by the following projects:

- [AI4EOSC](#): AI for the European Open Science Cloud
- [iMagine](#): Imaging data and services for aquatic science
- [AI4Life](#): AI models and methods for the life sciences
- [KMD4EOSC](#): Universal infrastructure for efficient data processing in HPC

Getting started Start here to get an overview of the platform and how to use it.

[/getting-started/index](#) How-Tos How to accomplish common tasks, such as trying a model, developing a new one, training, deploying, etc.

[/howtos/index](#) Reference Detailed information about the platform, dashboard, storage, modules, etc. and how to use them.

[/reference/index](#) Help and support Get help and support, including a FAQ section.

[Support](#) Other resources Useful Machine Learning resources, video demos, and other links.

[/others/index](#)

QUICKSTART GUIDE

First of all, welcome onboard!

Now, here are some guiding steps for your first contact with the AI4OS/AI4EOSC platform:

1. If you do not have an account, please *register* first!
2. Get familiar with the basic *platform components and user workflows*.
3. Head into our *Dashboard*.
4. Try one of the modules, in order to *perform a simple inference*.
5. Once you are familiar, you can start *developing*!

Enjoy!

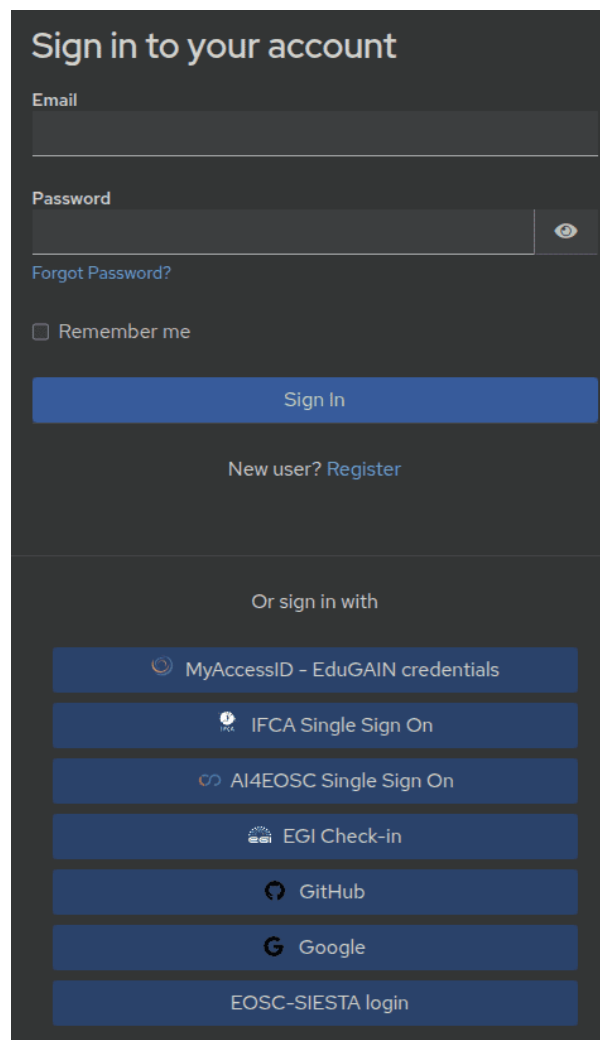
 Tip

If you run into problems you can always *contact support*!

ACCESS THE PLATFORM

2.1 Register an account

The [Login](#) uses Keycloak to federate different Identity providers, allowing to easily onboard new external communities with their own authentication systems.



The image shows a dark-themed login form titled "Sign in to your account". It includes input fields for "Email" and "Password", a "Forgot Password?" link, a "Remember me" checkbox, a "Sign In" button, and a "New user? Register" link. Below these, there is a section titled "Or sign in with" containing several buttons for different identity providers: MyAccessID - EduGAIN credentials, IFCA Single Sign On, AI4EOSC Single Sign On, EGI Check-in, GitHub, Google, and EOSC-SIESTA login.

There are two options to create accounts.

2.1.1 Option 1. Use a federated identity

This is the **recommended option** for:

- users coming from  **EGI-CheckIn** authentication (like AI4EOSC, iMagine and AI4life users). This is because Keycloak will automatically detect your EGI VO memberships (eg. `vo.ai4eosc.eu`, `vo.imagine-ai.eu`) and will grant you the proper platform full rights by default.

Open an EGI Check-In account

This mini tutorial is dedicated to users of the following projects:

- **AI4EOSC**: AI for the European Open Science Cloud
- **iMagine**: Imaging data and services for aquatic science
- **AI4Life**: AI models and methods for the life sciences

To access this authentication level, go to the **EGI Check-In (prod)** and login preferably with your **institutional account** (university, research center). If this is not possible, please, use one of **Github** or **ORCID** but with the email domain which identifies your organisation. This will automatically create your new account at EGI Check-In.

Then you have to enroll in one of the Virtual Organizations (VO) supported by the project. Please [ask support](#) for the request link of your Virtual Organization.

When enrolling, you have to provide the following information in **Comments**:

- Reason to use the platform (e.g. use case)
- Your institution or organization
- How did you learn about the platform

You will need to wait until you are **approved** before being a full member of the VO.

- researchers with  **EduGain** credentials, as you will be granted increased privileges due to your trusted researcher status

Additionally, we offer several federated authentication options including: [Github](#), [Google](#),  **IFCA SSO** and **ORCID**.

Tip

Accounts are mapped by email. So you can authenticate the first time with EGI Check-In, then can subsequently authenticate with any other option (eg. Github), as long as they share the same email.

Each authentication system will grant different *user access levels*. In short:

- users coming from the (**AI4EOSC**, **iMagine**, **AI4Life**) projects will have **full access** level.
- user coming from EduGain will have **intermediate access** level.
- remaining users (eg. Github, Google) will only have **basic access** level (observers).

Find how to [upgrade your access level](#).

2.1.2 Option 2. Register a new account

It is also possible to manually registering new account, although *this is not recommended* as the process is slower, since requests need to be manually approved by an administrator.

To register a new account manually:

- Click on the **Register** link on the login page,
- Please provide all the details that are listed in the registration form and preferably use your **institutional account** (university, research center). This will speed up the approval process.
- After you have filled in all the details, click on the **Register** button. You will receive an email with a link to confirm your registration. Click on the link to confirm your registration and then you will be approved by an administrator.

Accounts created this way will have **basic access** level by default (see [user access levels](#)).

Once created, you can input your credentials using the **Sign In** option, at the top of the page.

2.2 Login to the platform

Once registered, you can directly head to the [Dashboard](#) page and login to the platform.

PLATFORM OVERVIEW

AI4EOSC provides a comprehensive platform for artificial intelligence and machine learning applications for scientific usecases. The project offers a federated computing infrastructure and shared services that enable researchers, developers, and organizations to collaborate on AI model development, training, and deployment at scale.

A note on terminology

AI4OS is the name of the software stack described in this documentation.

AI4EOSC is the project that initially developed that stack and is currently maintaining it. AI4EOSC also host a particular deployment of the AI4OS stack components (under *cloud.ai4eosc.eu*). For example:

- The *Dashboard* deployed as the AI4EOSC Dashboard,
- The *Storage* deployed as the AI4EOSC Storage,
- etc.

In this regard, it is similar to other projects who have adopted the AI4OS Stack, like iImagine who deployed it's own version of the AI4OS Dashboard as the iImagine Dashboard.

To reduce duplicities and lower the entry barrier for external projects, many AI4OS components deployed by AI4EOSC (e.g. the *CI/CD pipeline* or the *Login*) also serve others projects, like iImagine.

3.1 Components

There are several different components in the AI4OS/AI4EOSC stack that are relevant for the users. Later on you will see how each different type of user can take advantage of the different components.

3.1.1 Dashboard

The *Dashboard* allows users to access computing resources to deploy, perform inference and train AI modules. The Dashboard simplifies the deployment and hides some of the technical parts that most users do not need to worry about.

3.1.2 AI modules

The *AI modules* are developed both by the platform and by users. For creating modules, we provide the AI Modules Template as a starting point. Every AI module of the platform exposes it's functionality under a *common API*, so that models can be accessed in a consistent way.

In addition to AI modules, the Dashboard also allows to deploy tools (eg. a Federated Server).

3.1.3 Training infrastructure

The Dashboard allows to deploy AI models in a **federated** computing infrastructure, based on **Nomad**. Each supported project can bring their own computing resources that can either be used exclusively by project members or shared with other projects.

Those are the datacenters that are currently part of the federation:

3.1.4 Inference infrastructure

The inference infrastructure, based on **OSCAR**, allows users to deploy trained AI modules in *serverless mode*. It supports horizontal scalability, quickly adapting to peaks in demand. Users can also compose those modules in *complex AI workflows*.

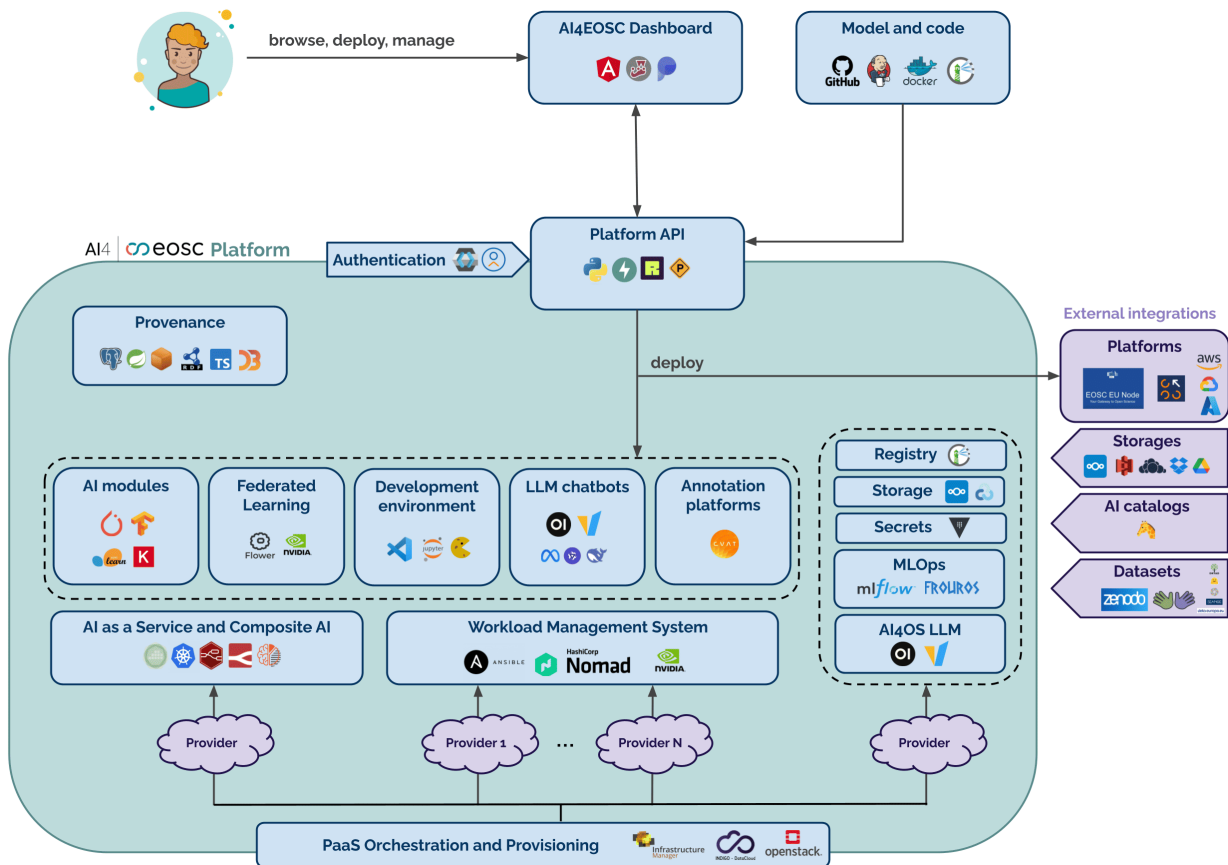
Other non-serverless deployment options *are available*, including deploying in external clouds.

3.1.5 The Storage

Storage is connected transparently to deployments, so that users can train AI modules on their custom data.

3.1.6 Architecture overview

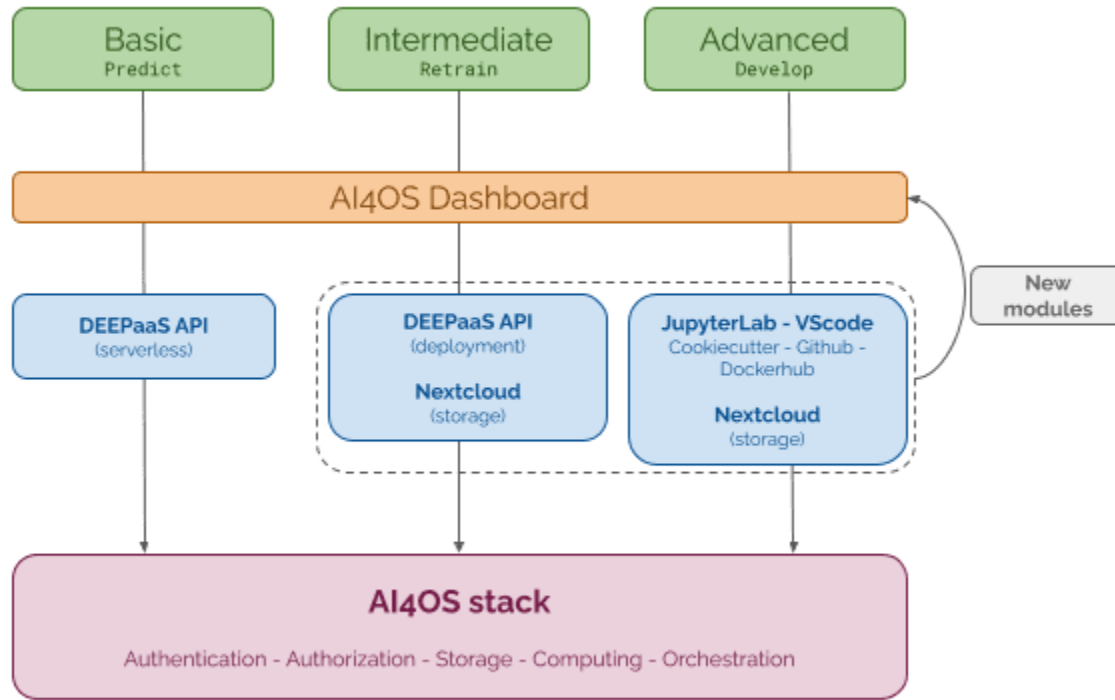
If you are curious, this is a very high level architecture overview of the platform:



And if you are feeling super-nerdy , these are the low-level C4 architecture diagrams of the platform.

3.2 Our different user roles

The platform is focused on three different types of users. Depending on what you want to achieve you should belong into one or more of the following categories:



3.2.1 The basic user

This user wants to use modules that are already pre-trained and test them with their data. Therefore, they don't need to have any particular machine learning knowledge. For example, they can take an already trained module for [plant classification](#) that has been containerized, and use it to classify their own plant images.

What the platform can offer to you:

- a *Dashboard* full of ready-to-use modules to perform inference with your data,
- a *GUI* to easily interact with the services,
- an *API* to integrate the AI modules with your own services,
- solutions to run the inference in *the Cloud* or in your *local resources*,
- the ability to *create pipelines* by composing different modules.

i Related HowTo's

- *How to try a model*
- *How to deploy a model in production*
- *How to create AI model pipelines*

3.2.2 The intermediate user

The intermediate user wants to retrain an available module to perform the same task but fine-tuning it to their own data. They still might not need high level knowledge on modelling of machine learning problems, but typically do need basic programming skills to prepare their own data into the appropriate format. Nevertheless, they can re-use the knowledge being captured in a trained network and adjust the network to their problem at hand by re-training the network on their own dataset. An example could be a user who takes the generic [image classifier](#) model and retrains it to perform [plant classification](#).

What the platform can offer to you:

- the ability to train out-of-the-box a module of the [Dashboard](#),
- the ability to easily connect your training to your dataset hosted on our [data storage resources](#),
- a private instance of [Computer Vision Annotation Tool \(CVAT\)](#) to annotate your dataset,
- a private server to create Federated Learning trainings with Flower,
- the ability to [use GPUs](#) to accelerate your training,
- an [API](#) to easily interact with the model,
- solutions to deploy your developed model in [the Cloud](#) or in your [local resources](#),
- the ability to share your module with other users in the [Dashboard Marketplace](#).

Related HowTo's

- [How to train a model](#)

3.2.3 The advanced user

The advanced users are the ones that will develop their own machine learning models and therefore need to be competent in machine learning. This would be the case for example if we provided an image classification model but the users wanted to perform object localization, which is a fundamentally different task. Therefore they will design their own neural network architecture, potentially re-using parts of the code from other models.

What the platform can offer to you:

- a [ready-to-use IDE](#) (VScode, Jupyterlab) with the main DL frameworks (Pytorch, Tensorflow) running on different types of hardware (CPUs, GPUs),
- the ability to easily connect your environment to your dataset hosted on our [data storage resources](#),
- the ability to integrate [experiment tracking with MLflow](#) in your trainings,
- tutorials on performing different types of trainings ([incremental learning](#), [distributed learning](#))
- the ability to [use GPUs](#) to accelerate your development,
- the possibility to [integrate your module with the API](#) to enable easier user interaction,
- solutions to deploy your developed model in [the Cloud](#) or in your [local resources](#),
- the ability to share your module with other users in the [Dashboard Marketplace](#).

Related HowTo's

- [How to develop a model](#)

NEW FEATURES

Here is a quick summary of the major updates in the platform that are relevant for users. Minor updates to the documentation will still be performed continuously but not notified here.

docs, dashboard, others

- we started supporting *smart scheduling* to greener datacenter (2026-03-12)
- we started supporting the *KMD4EOSC project* (2026-02-09)
- we release a new *Helpdesk and Community to improve interaction with users* (2025-08-01)
- we released our provenance tracking workflow (2025-07-10)
- we migrated to our *new authentication system* (2025-07-01)
- we started supporting *deploying models to the EOSC node* (2025-06-20)
- we started supporting *drift monitoring detection* (2025-06-10)
- we started supporting *batch mode trainings* (2025-05-25)
- we added support for developers to provide *minimum inference resources* (2025-05-06)
- we started supporting Federated Learning trainings with NVFLARE (2025-04-07)
- we changed the Dashboard layout and added an LLM catalog section (2025-04-04)
- we added support for *Vision-enabled LLMs* (2025-03-19)
- we started support deploying *your own LLM instance* (2025-02-28)
- Added documentation on how to integrate *the platform LLM as coding assistant in VScode* (2025-02-13)
- we started support deploying *external models from AI4Life* (2025-01-30)
- deployed in beta the *LLM service* (2025-01-15)
- added support for async calls to *OSCAR services* (2024-12-12)
- support the ability of *making snapshots of deployments* (2024-11-26)
- enable Gradio UI for inference on the *platform dedicated resources* (2024-11-26)
- added support to *label images with CVAT* (2024-11-14)
- add support for *Seanoë and Data Europa datasets downloading* (2024-10-18)
- redesign of the Dashboard Marketplace, with *new filtering features* (2024-10-18)
- redesign of the Dashboard module pages, following the new metadata schema (2024-10-18)
- we send notifications to users when deployments take long to deploy (2024-10-11)
- we started supporting the *AI4Life project* (2024-10-07)

- new ability *deploy OSCAR services* from the Dashboard (2024-09-25)
- redesign the Dashboard+Docs around the main user workflows: Try, Develop, Deploy (2024-09-20)
- new *Profile section* where you can link your storage providers (2024-09-04)
- new *try-me deployments* (2024-09-03)
- new federated cluster is available in production (2024-08-05)
- new *external datasets download* feature (2024-06-30)
- new guides to perform *Incremental Learning* (2024-06-20)
- new *stats section* in the Dashboard (2024-05-14)
- support for token authentication in Federated Learning trainings with Flower (2024-04-16)
- new *New features* page (meta!) (2024-04-03)
- new tutorials for creating AI Inference pipelines (*Elyra*, *Flowfuse*) (2024-03-18)
- new *MLflow* tutorials (2024-02-09)
- new *Frequently Asked Questions (FAQ)* page (2024-02-05)
- *MLflow* available under new domain (2024-01-29)
- we enforce a quota of 2 GPUs per user for better resource distribution (2024-01-25)
- updated *Quickstart* page (2024-01-24)
- users are able to select specific GPU models (2024-01-18)
- new *Nextcloud* storage available (2024-01-15)

4.1 Upcoming features

In this section, we will offer some insight on to where the AI4OS stack is heading. Some of these features are experimental and might not end up being integrated in the stack:

- integration of carbon monitoring

Do you find something missing? Please check *how to suggest a new feature*.

TRY A MODEL

You can try the model in two ways. The easiest way is to use the Dashboard, which provides a short-lived web UI for interacting with the model. Alternatively, you can run the model locally on your own resources.

Try via the Dashboard (Gradio) Use our short-lived web UI to interact with the model. Batteries included, no installation needed.

Try a model from the Dashboard (Gradio) Try locally Run the model on your own resources. Requires Docker to handle the model container, and manual execution of the model.

Try a service locally

5.1 Try a model from the Dashboard (Gradio)

Try a model from the Dashboard

Please, be aware that video demos can become quickly outdated. In case of doubt, always refer to the written documentation.

Requirements

You need a *platform account* with *basic access level*.

To create a deployment, *select a module in the Marketplace* and click on Try `Nomad` (Gradio).

Train an image classifier

Try ▾

Deploy ▾

Codespaces ▾

Batch

Train your own image classifier with your custom dataset. It comes also pretrained on the 1K ImageNet classes.

The deep learning revolution has brought significant advances in a number of fields [1], primarily linked to image and speech recognition. The standardization of image classification tasks like the [ImageNet Large Scale Visual Recognition Challenge](#) [2] has resulted in a reliable way to compare top performing architectures.

This Docker container contains the tools to train an image classifier on your personal dataset. It is a highly customizable tool that let's you choose between tens of different [top performing architectures](#) and training parameters.

The container also comes with a pretrained general-purpose image classifier trained on ImageNet. The PREDICT method expects an RGB image as input (or the url of an RGB image) and will return a JSON with the top 5 predictions.



References

1. Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. [Deep learning](#). Nature, 521(7553):436-444, May 2015.
2. Olga Russakovsky et al. [ImageNet Large Scale Visual Recognition Challenge](#). International Journal of Computer Vision (IJCV), 115(3):211-252, 2015.

Apache-2.0 build passing

Tasks

Computer Vision

Classification

Libraries

TensorFlow

Keras

Data Types

Image

Tags

deep learning

image classification

vo.imagine-ai.eu

general purpose

Platform Categories

AI4 trainable

AI4 pre trained

AI4 inference

Provenance

View

Download RDF

Additional Resources

GitHub

Report issue

Dataset

Docker

Weights

99 Citation

Dates

2024-04-25

2025-05-22

This will open a new tab where the new try-me environment will be deployed (this can take a few seconds).

Once this is deployed, you will see a [Gradio](#) interface where you can make your prediction. In the left hand-side, you will be shown the prediction inputs and in the right hand side you will be shown the prediction outputs.

imgclas

i This is a temporary deployment that will automatically delete itself after 10 minutes. To access more permanent inference options, [become a member of one of the supported projects](#).

Take into mind that this deployment runs on *limited resources*, therefore some resource-hungry functionalities (like processing videos or very big images) might not work as expected.



data

Select the image you want to classify.

urls

Select an URL of the image you want to classify.

timestamp

Model timestamp to use for prediction.

"default_imagenet"

ckpt_name

Checkpoint inside the timestamp to use for prediction.

"final_model.h5"

Clear

Submit

message

status

OK

classification scores

notebook

notebook	59%
laptop	16%
computer_keyboard	4%
tape_player	3%
mouse	2%

Flag

AI4

eosc

i Advice for model developers

If you are a model developer looking to improve the looks of your module's UI, take a look to our [developer guidelines](#).

Take into account that this try-me endpoints are offered to the general public, thus run on limited resources. This means that we limit the maximum duration of a try-me environment to 10 minutes. In addition, resource-hungry operations (eg. video processing) might not work as expected.

In line with this, we also limit the number of try-me endpoints that the user can deploy at the same time. If you reach the limit, you can wait until one of your previous deployments is automatically deleted or you can delete it manually in the Try me section of the Dashboard.

Dashboard
Marketplace
Try me
Deployments
Inference
Useful links

Try me deployments

Title	Status	Image	Creation time (UTC)	Actions
turquoise_coyote	running	ai4oshub/birds-audio-classification:latest	2024-10-29 11:51:53	⏏ ⓧ ⓧ
amaranth_cheetah	running	ai4oshub/ai4os-demo-app:latest	2024-10-29 11:51:45	⏏ ⓧ ⓧ

5.2 Try a service locally

i Requirements

This section requires having [Docker](#) installed:

- Starting from version 19.03 docker supports NVIDIA GPUs. For this, you also have to install the [nvidia-container-toolkit](#) to make them visible from inside the container. If you happen to be using an older version you can give a try to [nvidia-docker](#)
- If you need to use docker in an environment without root privileges (eg. an HPC cluster), check [udocker](#) instead of docker.

5.2.1 1. Choose your module

The first step is to choose a module from the [Dashboard](#). For educational purposes we are going to use a [general model to identify images](#). This will allow us to see the general workflow.

Once we have chosen the Module page we will find that it has an associated docker container in [DockerHub](#). For example, in the example we are running here, the container would be `ai4oshub/ai4os-image-classification-tf`. This means that to pull the docker image and run it you should:

```
$ docker pull ai4oshub/ai4os-image-classification-tf
```

Docker images have usually tags depending on whether they are using master or test and whether they use cpu or gpu. Tags are usually:

- latest or cpu: master + cpu
- gpu: master + gpu
- cpu-test: test + cpu

- `gpu-test`: test + gpu

So if you wanted to use gpu and the test branch you could run:

```
$ docker pull ai4oshub/ai4os-image-classification-tf:gpu-test
```

Instead of pulling from Dockerhub you can choose to build the image yourself:

```
$ git clone https://github.com/ai4os-hub/ai4os-image-classification-tf
$ cd ai4os-image-classification-tf
$ docker build -t ai4oshub/ai4os-image-classification-tf .
```

5.2.2 2. Launch the API and predict

Run the container with

```
$ docker run -ti -p 5000:5000 -p 6006:6006 -p 8888:8888 ai4oshub/ai4os-image-
↪classification-tf
```

If you wanted to run using a GPU, make sure to include the `--nvidia` flag in the command:

```
$ docker run -ti -p 5000:5000 -p 6006:6006 -p 8888:8888 --runtime=nvidia ai4oshub/ai4os-
↪image-classification-tf:gpu
```

Once running, point your browser to <http://127.0.0.1:5000/ui> and you will see the API documentation, where you can test the module's functionality, as well as perform other actions.

models		
GET	/v2/models/	Return loaded models and its information
GET	/v2/models/imgclas/	Return model's metadata
POST	/v2/models/imgclas/train/	Retrain model with available data
GET	/v2/models/imgclas/train/	Get a list of trainings (running or completed)
GET	/v2/models/imgclas/train/{uuid}	Get status of a training
DELETE	/v2/models/imgclas/train/{uuid}	Cancel a running training
POST	/v2/models/imgclas/predict/	Make a prediction given the input data

Go to the `predict()` function and upload the file/data you want to predict (in the case of the image classifier this should be an image file). The appropriate data formats of the files you have to upload are often discussed in the module's Marketplace page or in their Github README files.

The response from the `predict()` function will vary from module to module but usually consists on a JSON dict with the predictions. For example the image classifier return a list of predicted classes along with predicted accuracy. Other modules might return files (eg. images, zips, ...) instead of a JSON response.

DEVELOP A MODEL

You can develop a model in the platform using the different deployment options available. This guide will help you understand how to develop a model in the platform via our development environments.

Develop a model Learn how to develop a model with the different deployment options available in the platform.

Develop a model from scratch

6.1 Advanced guides

We also provide guides for developers to implement and support advanced training techniques in their modules:

Use Incremental learning Learn how to use incremental learning in your modules.

Incremental (online) Learning

Use Distributed learning Learn how to use distributed learning in your modules.

Distributed Learning

Support MLFlow for tracking your trainings Learn how to use MLFlow for tracking your trainings.

Experiment Tracking and Model versioning in MLflow

Support Drift detection at inference Learn how to use DriftWatch to monitor data drift during production.

Data drift detection with Frouros

6.1.1 Develop a model from scratch

Requirements

- For **Step 1**, to use the Module's template webpage, you need at least *basic authentication*.
- For **Step 2**, if you plan to use the Development Environment, you need *full authentication* to be able to access the Dashboard. Otherwise you can develop locally.
- For **Step 4** we recommend having *docker* installed (though it's not strictly mandatory).

This tutorial explains how to develop an AI module from scratch.

If you are new to Machine Learning, you might want to check some *useful Machine Learning resources* we compiled to help you getting started.

1. Setting the framework

This first step relies on the the AI Modules Template for creating a template for your new module:

- Access and authenticate in the [Template creation webpage](#).
- Then select the minimal branch of the template and answer the questions.
- Click on Generate and you will be able to download a .zip file with with your project directory.

2. Prepare your development environment

Although it is possible to develop your code locally, we also offer the possibility to develop from our [Development Environment](#).

This offers the benefits of:

- developing on dedicated resources (including GPUs),
- have direct access to your data hosted in the [Storage](#),
- develop on a Docker image that is already packaged with your favorite Deep Learning framework (eg. Pytorch, Tensorflow),
- develop on your favorite IDE (Jupyterlab or VScode),

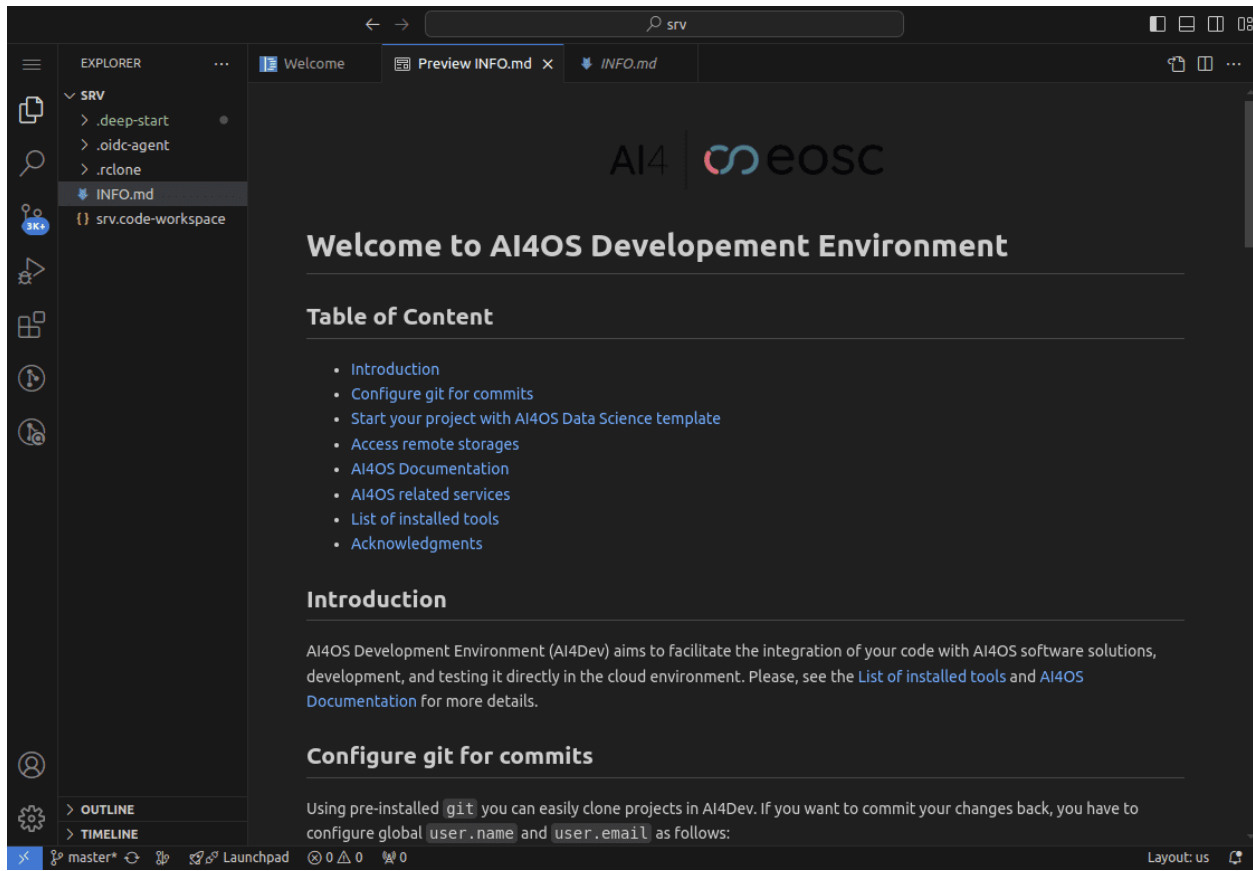
Check [how to create and configure](#) the Development Environment.

Use storage-synced folder to develop

We *strongly* recommend to create a *deployment attached to storage*.

By doing so, you can develop your code inside the `/storage` folder and any changes you make will *instantly* be synced with the [Storage](#). This will prevent any work loss in case of an unexpected deployment crash (which do happen from time to time).

This is what an Development Environment with VScode would look out-of-the-box:



Launching a development environment

Please, be aware that video demos can become quickly outdated. In case of doubt, always refer to the written documentation.

3. Editing the module's code

Drag and drop in the VSCode editor the zip file created in Step 1. Then unpack it.

Optimal VScode setup: using LLMs and more

Tip n°1: VScode by default is initialized in `/srv`. For a better coding experience we suggest opening VSCode with your folder project *only*. This will allow you to ignore other non-related folders under `/srv` when doing global searches or tracking changes, for example. For this, go to `File > Open Folder > /srv/<project-name>`.

As explained earlier, having your project under `/storage` will allow it to be automatically synced with the *Storage*.

Tip n°2: *Use the platform LLM as coding assistant* to help you develop faster. It is integrated directly in VScode through the use of the *Continue.dev* extension.

We recommend implementing first tip n°1, in order to avoid the Continue assistant from freezing when trying to index the whole workspace contents.

Install your project as a Python module in **editable** mode, so that the changes you make to the codebase are picked by Python.

```
cd <project-name>
pip install -e .
```

Troubleshooting pip installation

Some users have reported issues in some systems when installing `deepaas` (which is always present in the `requirements.txt` of your project). Those issues have been resolved as following:

- In [Pytorch Docker images](#), making sure `gcc` is installed (`apt install gcc`)
- In other systems, sometimes `python3-dev` is needed (`apt install python3-dev`).

Now you can start adding your AI model code inside `<project-name>/<project-name>`.

Integrating with the DEEPaaS API

Once your code is included, you need your module to be able to interface with the [DEEPaaS API](#), which allows any module in the Marketplace to be accessed using the same API. For this, *you have to define* in `api.py` the functions you want to make accessible to the user. For this tutorial we are going to head to our [official demo module](#) and copy-paste its `api.py` file.

Once this is done, check that DEEPaaS is interfacing correctly by running:

```
deepaas-run --listen-ip 0.0.0.0
```

Your module should be visible in <http://0.0.0.0:5000/ui>. If you don't see your module, you probably messed the `api.py` file. Try running it with `python` so you get a more detailed debug message.

```
python api.py
```

Remember to leave untouched the `get_metadata()` function that comes predefined with your module, as all modules should have proper metadata.

In general, you will have the following ports available when making a deployment in the platform:

- **5000**: default port used for exposing the API (eg. DEEPaaS)
- **6006**: default port used for exposing a monitoring endpoint (eg. Tensorboard)
- **8888**: default port used for exposing the IDE (eg. JupyterLab, VScode)
- **80**: port available to let developers expose their custom endpoint

Model Quality Assurance

In order to improve the readability of the code and the overall maintainability of your module, we enforce some quality standards in `tox` (including style, security, etc). Modules that fail to pass style tests won't be able to build docker images. You can check locally if your module passes the tests:

```
tox -e .
```

There you should see a detailed report of the offending lines (if any). You can always [turn off flake8 testing](#) in some parts of the code if long lines are really needed.

Using code formatters

If your project has many offending lines, it's recommended using a code formatter tool like [Black](#). It also helps for having a consistent code style and minimizing git diffs. Black formatted code will always be compliant with flake8.

Once [installed](#), you can check how Black would have reformatted your code:

```
black <code-folder> --diff
```

You can always [turn off Black formatting](#) if you want to keep some sections of your code untouched.

If you are happy with the changes, you can make them permanent using:

```
black <code-folder>
```

Remember to have a backup before reformatting, just in case!

4. Editing the module's Dockerfile

Your `./Dockerfile` is in charge of creating a docker image that integrates your application, along with deepaas and any other dependency. You can modify that file according to your needs.

If you need to add instructions based on the runtime (eg. perform certain actions depending on whether you detected a GPU), please use the `ENTRYPOINT` statement, as `CMD` will be overwritten by the platform when you deploying a given service (eg. JupyterLab).

We recommend checking the installation steps are fine. If your module needs additional Linux packages add them to the Dockerfile. Check your Dockerfile works correctly by building it **locally** (outside the Development Environment) and running it:

```
docker build --no-cache -t your_project .
docker run -ti -p 5000:5000 -p 6006:6006 -p 8888:8888 your_project
```

Your module should be visible in <http://0.0.0.0:5000/ui> . You can make a POST request to the `predict` method to check everything is working as intended.

5. Update your project's metadata

The module's metadata is located in the `ai4-metadata.yml` file ([example](#)). This is the information that will be displayed in the Marketplace. The fields you need to edit to comply with our [schemata](#) are:

- `title` (*mandatory*): short title,
- `summary` (*mandatory*): one liner summary of your module,
- `description` (*optional*): extended description of your module, like a README,
- `links` (*mostly optional*): links to related info (training dataset, module citation. etc),
- `tags` (*mandatory*): relevant user-defined keywords (can be empty),
- `categories, tasks, libraries, data-type` (*mandatory*): one or several keywords, to be chosen from a closed list (can be empty).

Supported values

Libraries	Tasks	Categories	Data Type
TensorFlow	Computer Vision	AI4 pre trained	Image
PyTorch	Natural Language Processing	AI4 trainable	Text
Keras	Time Series	AI4 inference	Time Series
Scikit-learn	Recommender Systems	AI4 tools	Tabular
XGBoost	Anomaly Detection		Graph
LightGBM	Regression		Audio
CatBoost	Classification		Video
Other	Clustering		Other
	Dimensionality Reduction		
	Generative Models		
	Graph Neural Networks		
	Optimization		
	Reinforcement Learning		
	Transfer Learning		
	Uncertainty Estimation		
	Other		

- **inference** (*optional*): this is the minimum resources your module needs to run an inference correctly (eg. CPU cores, RAM, GPUs, etc). If not specified, the Dashboard will prefill with some defaults, that can later be adapted by the user during the [configuration step](#).
- **provenance** (*optional*): this will allow your model to have a more rich provenance information, as your model provenance graph will show the resources and the hyper-parameters you used to train. There are two subfields you can specify:
 - `nomad_job`: the Dashboard deployment UUID you used to train the final model,
 - `mlflow_run`: the MLflow run UUID you used to train the final model,

Some fields are pre-filled via the AI Modules Template and usually do not need to be modified. Check you didn't mess up the YAML definition by running our [metadata validator](#):

```
pip install ai4-metadata
ai4-metadata validate ai4-metadata.yml
```

6. Integrating the module in the Marketplace

Once your repo is set, it's time to integrate it in the [Marketplace](#):

1. [Open an issue](#) in the [AI ModuleCatalog repo](#).
2. A platform admin will create the Github repo for your module inside the [ai4os-hub organization](#). You will be granted `write` permissions in that repo.

Naming conventions

Modules repos follow the following convention:

- `ai4os-hub/ai4-<project-name>`: module officially developed by the project
 - `ai4os-hub/<project-name>`: modules developed by external users
3. Upload your code to that repo.

4. An admin will review your code and add it to the AI Module Catalog. Once a module is approved it will take roughly 6 hours to appear in the Dashboard's Marketplace.

i Next steps

If to go further, check our tutorials on how to:

- *develop a module that support incremental learning*
- *develop a module that supports distributed learning*
- *develop a module that support MLflow for tracking experiments*

6.1.2 Experiment Tracking and Model versioning in MLflow

i Requirements

You need a *platform account* with *full access level*.

We currently have two instances of MLflow running:

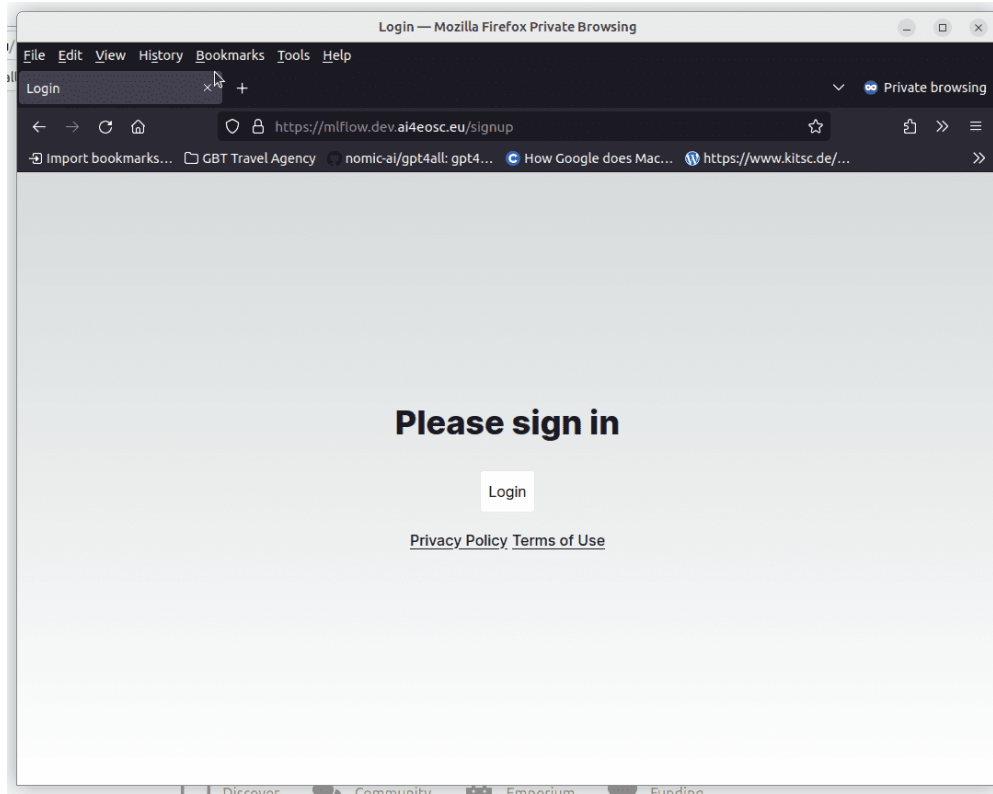
- [MLflow \(AI4EOSC\)](#)
- [MLflow \(iImagine\)](#)

When following this tutorial, adapt the MLflow links depending on which *Project or Virtual Organization* you belong to.

In case you already have a MLflow account, you can proceed to step 2.

1. Register for an account in MLflow

Go to the [Sign Up](#) page for self registration.



Then, in the next window:

- insert your password.
- read and accept the Privacy Policy and Terms of Use.

Finally you will be shown your user settings:

- In case you want to change the password you can enter a new password in the text box Password and then click the Update button.
- You can check your user info (your username is your email!)
- You have also the option of deleting your account as well as logging out.

Once you are ready, proceed to the new step by clicking in Go to mlflow.

2. Login the MLflow UI

In the [MLflow login page](#) you will be asked to input your credentials:

- Username: the email associated with your authentication account
- Password: the password you choose in step 1.

Once you login, you will see the default MLflow UI as follows:

Experiments 2.7.1 Experiments Models GitHub Docs

Search Experiments

- ☐ wind_power_forecast_1
- ☐ wind_power_forecast
- ☐ green-taxi-duration
- ☐ green-taxi-duration-1
- ☐ fasterrcnn
- ☒ wind_power_forecast_W
- ☒ green-taxi-duration-x
- ☐ sklearn_el_wine
- ☐ yolov8
- ☐ TUFSeg_training
- ☐ yolov8_L

Displaying Runs from 2 Experiments Share

Q metrics.rmse < 1 and params.model = "tree" Time created State: Active Sort: Created Columns

Expand rows

Table Chart Evaluation **Experimental**

Run Name	Created	Dataset	Duration	Source	Models
charming-whale-210	1 month ago	dataset (4af87e85) Training	1.1min	mlflow...	wind-forec.../5
bustling-mole-325	1 month ago	dataset (4af87e85) Training	4.2min	mlflow...	wind-forec.../4
crawling-shrimp-763	1 month ago	dataset (4af87e85) Training	4.2min	mlflow...	wind-forec.../3
exultant-grub-452	1 month ago	dataset (4af87e85) Training	4.3min	mlflow...	wind-forec.../2
likeable-squid-119	1 month ago	dataset (4af87e85) Training	4.3min	mlflow...	wind-forec.../1
sklearn_HPO_2023-11-29 11:...	2 months ago	-	1.3min	mlflow...	-
trial_4	2 months ago	-	14.4s	mlflow...	sklearn
trial_3	2 months ago	-	13.9s	mlflow...	sklearn
trial_2	2 months ago	-	14.3s	mlflow...	sklearn
trial_1	2 months ago	-	14.8s	mlflow...	sklearn
trial_0	2 months ago	-	15.1s	mlflow...	sklearn
whimsical-whale-943	2 months ago	-	3.2s	mlflow...	-
sklearn_HPO_2023-11-28 21:...	2 months ago	-	1.3min	mlflow...	-
trial_4	2 months ago	-	13.4s	mlflow...	sklearn

51 matching runs

3. Log your Experiments

Now you are ready to start logging in your experiments and saving the (best) trained model with a version in Model Registry.

For this you have to do the following steps in your deployment.

Install MLflow

First install mlflow client from the IDE that you are using to build your AI model, by executing:

```
pip install mlflow[extras]
```

You can also use `mlflow-skinny` for a minimal setup, therefore reducing potential dependency conflicts with your other module packages.

Define MLflow credentials

MLflow reads the credentials as envvars. When creating a deployment in the platform, those credentials are automatically injected in the environment by the platform (no need to create them!).

You can check these vars from your command line in your deployment:

```
echo $MLFLOW_TRACKING_USERNAME
echo $MLFLOW_TRACKING_PASSWORD
echo $MLFLOW_TRACKING_URI
```

If you find that your credentials are not correctly injected by the platform, try *updating your password*.

Modify your Python code

Here is an example on how to start tracking in your training loop:

```
import mlflow

mlflow.set_experiment(experiment_name='your_experiment_name')

# or Name of the experiment (e.g. name of the code repository)
MLFLOW_EXPERIMENT_NAME="your_experiment_name"

# Name of the model to train. HAS TO BE UNIQUE, Please, DEFINE ONE!
MLFLOW_MODEL_NAME="your_model_name"

#MLflow specific statements to log your experiment
#Insert the following statements in your code where you are training your model,e.g.
def train_model():
    # your existing code here

    history = model.fit(X_train, y_train, epochs=100, batch_size=64,
                        validation_data=(X_val, y_val), callbacks=[early_stopping])

    with mlflow.start_run(run_name="run-demo") as run: # mlflow starting command

        # Log metrics to MLflow for each epoch
        batch_size = 10 # Log metrics every 10 epochs (adjust as needed)
        for epoch, (loss, val_loss) in enumerate(zip(history.history["loss"],
                                                    history.history["val_loss"])):

            if epoch % batch_size == 0:
                mlflow.log_metric("train_loss", loss, step=epoch)
                mlflow.log_metric("val_loss", val_loss, step=epoch)

        # Log params
        mlflow.log_params({
            "hidden_units": 100,
            "activation": "relu",
            "epochs": 100,
            "batch_size": 64,
            "validation_split": 0.2
        })

        # Log model using: mlflow.<flavor>.log_model()
        # Log the TensorFlow using mlflow.tensorflow.log_model
        mlflow.tensorflow.log_model(model, artifact_path='artifacts')

        # Log additional artifacts
        # Log the CSV file as an artifact in MLflow
        mlflow.log_artifact(data_csv, artifact_path='artifacts/dataset')
```

We provide some examples of mlflow implementations to serve as reference, as well a specific integration of mlflow once you have created your api in the Marketplace.

For more information, see the [Getting Started](#) guide in the official MLflow docs. Additionally, you can go to AI4EOSC YouTube channel and check these videos on MLFlow: 1) [How to create an account in MLFlow](#): 2) [How to Log an experiment in MLFlow](#):

Finally, to save the models in the registry, you have to add the following code in your deployment:

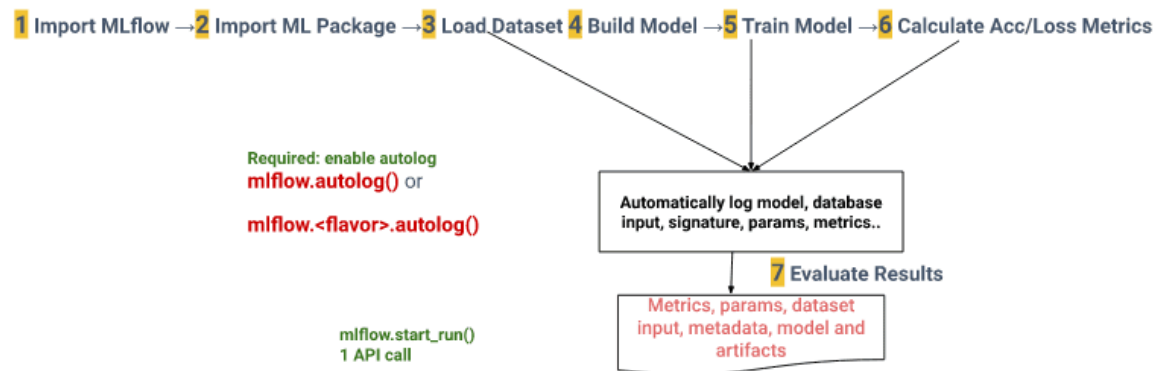
```
# REGISTER MODEL to MODEL REGISTRY #
result = mlflow.register_model(
    f"runs:{run_id}/artifacts/", MLFLOW_MODEL_NAME
)
```

4. MLflow AutoLogging and CustomLogging

There exists two Logging options as illustrated in the following Figures.

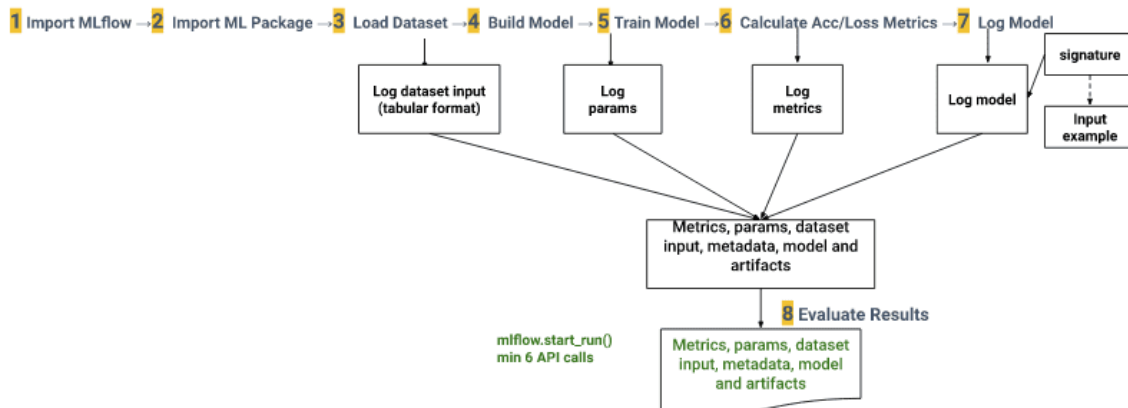
MLflow AutoLogging QuickView:

! with Autologging



MLflow Custom Logs QuickView:

! without Autologging



- Important commands to know

```
# Log Param (Log a parameter under the current run):
mlflow.log_param("batch_size", 64)
# Log Params (Log multiple parameter under the current run):
mlflow.log_params({"hidden_units": 100,
```

(continues on next page)

(continued from previous page)

```

        "activation": "relu",
        "batch_size": 64,
        "validation_split": 0.2})
# Log Metric (Log a metric under the current run):
mlflow.log_metric("mse", 90.00)
# Log Metric (Log multiple metrics under the current run):
mlflow.log_metrics({"mse": 90.00,
                    "rmse": 75.00})

```

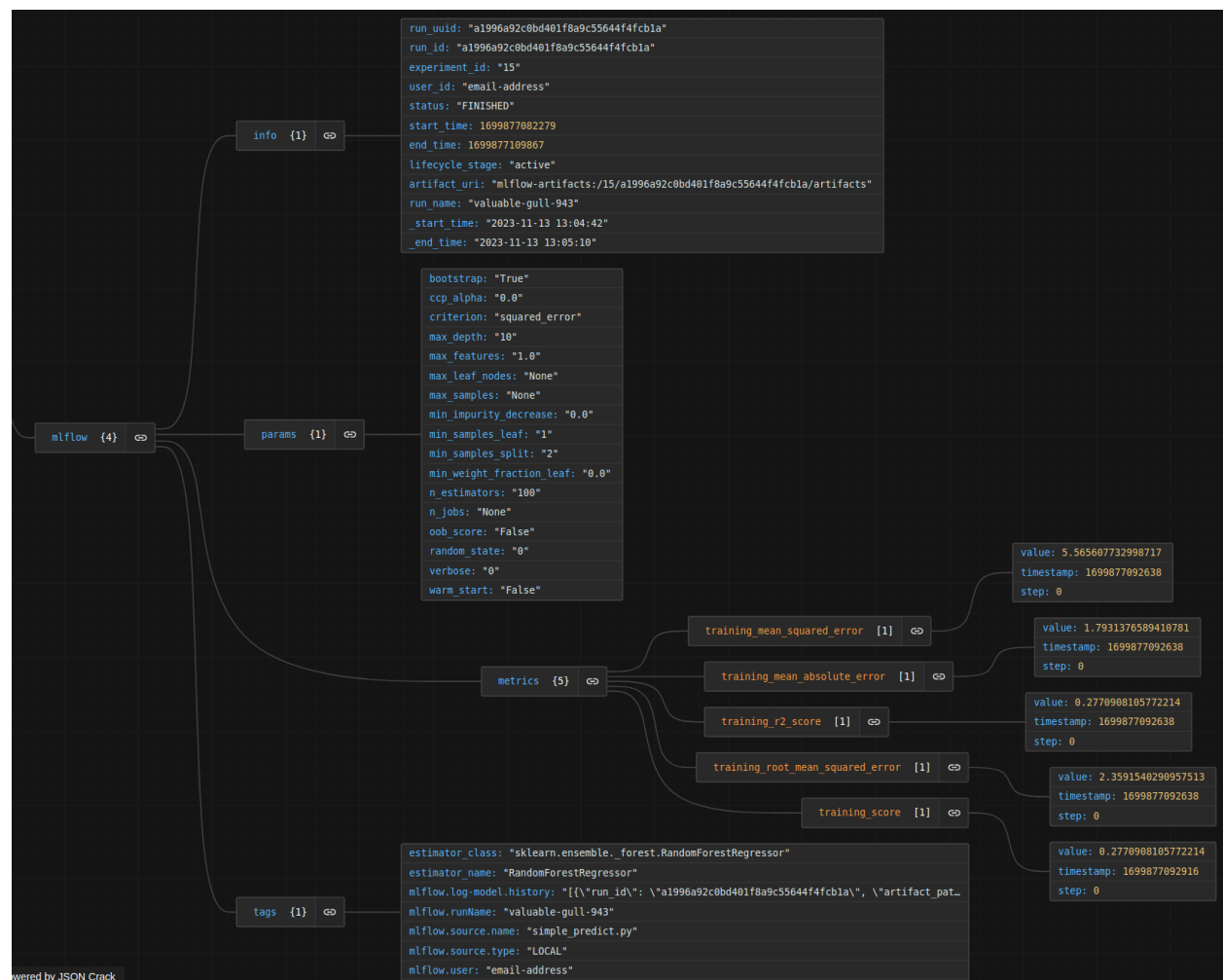
- Log Artifact(s)

```

# Log Figure (Log a figure as an artifact)
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.plot([1,2],[4,5])
mlflow.log_figure(fig, "fig_plot.png")

```

- Relevant information extracted from an Experiment-Run (see figure below) defined as a single execution of a machine learning code



5. MLflow Model Versioning and Production Deployment

- Adding Tags to Model Versions You can add tags to model versions to include additional metadata:

```
from mlflow import MlflowClient

# Get the latest model version
client = MlflowClient()
latest_versions = client.get_latest_versions(MLFLOW_MODEL_NAME)
latest_version = latest_versions[0].version

# Add tags to a specific model version
client.set_model_version_tag(
    name=MLFLOW_MODEL_NAME,
    version=latest_version,
    key="deployment_status",
    value="active"
)

# Add multiple tags
tags = {
    "data_version": "v2.0",
    "algorithm": "random_forest",
    "responsible_team": "data_science",
    "accuracy": "0.92"
}

for key, value in tags.items():
    client.set_model_version_tag(
        name=MLFLOW_MODEL_NAME,
        version=latest_version,
        key=key,
        value=value
    )
```

- Setting Model Version Aliases for Production MLflow now uses aliases instead of stages (which are deprecated). Aliases provide a more flexible way to manage model deployment status. Using Aliases (Champion/Challenger Model Pattern)

```
from mlflow import MlflowClient

# Set the 'champion' alias for your production model
client = MlflowClient()
client.set_registered_model_alias(
    name=MLFLOW_MODEL_NAME,
    alias="champion",
    version=latest_version
)
```

- Loading the Production (Champion) Model You can add tags to model versions to include additional metadata:

```
# Load the champion model for inference
champion_model = mlflow.pyfunc.load_model(
    model_uri=f"models://{MLFLOW_MODEL_NAME}@champion"
```

(continues on next page)

(continued from previous page)

```
)

# Make predictions
predictions = champion_model.predict(data)
```

- Search model versions Search for a specific model name and list its version details using `search_model_versions()` method and provide a filter string such as `name='sk-learn-random-forest-reg-model'`

```
from mlflow import MlflowClient

client = MlflowClient()
for mv in client.search_model_versions("name='sk-learn-random-forest-reg-model'"):
    pprint(dict(mv), indent=4)
```

6.1.3 Data drift detection with Frouros

Requirements

You need a *platform account* with *full access level*.

The platform allows to detect **data drift** in your data at inference time. This is a useful warning that the inference results might not be reliable anymore and that some action should be taken (eg. cleaning the sensor, retraining the model, etc).

In this tutorial, we are going to demonstrate how to implement drift detection in an image object detection pipeline. Specifically, we are going to use the [OBSEA Fish Detection module](#), where the drift detector will signal that the underwater camera is dirty and needs cleaning.

You can find the [full code](#) of this tutorial, as well as [reference notebooks](#).

In this tutorial, we use [Frouros](#) as the main drift detection library, but the tutorial still applies to other popular drift detection libraries like [Alibi-detect](#), [Evidently](#), [Eurybia](#), etc.

What is drift detection?

In drift detection, we monitor a model at inference time to detect when the input data starts to deviate from the training data distribution: we call this **data drift**. There could be many reasons causing data drift:

- the sensor taking the images is dirty, so we need to clean it,
- the distribution of data has really changed, so the model needs to be retrained,

In any case, the predictions are no longer reliable and an action has to be taken by the user.

To detect drift, we take the inference data vector and compare it with a reference training dataset. We compute a distance that summarizes what is the likelihood that the inference vector could come from the training data. If the distance is above a threshold, we can confidently assert that the data has indeed drifted.

In the case of images, the pure pixels values are not a good summarizer of the image statistics. So we typically train an **autoencoder model** that is able to summarize the pixel values into a smaller vector that more accurately describes the image. We then use this vector to compute the distance, as before.

Find more information on the [fundamentals of drift detection](#).

Create your drift detector

1. Define your reference data

The first step is to define a “normal” reference dataset (ie. clean camera). The clean images will be used as reference to train the detector and determine the statistical properties of the data under normal conditions. You can also define anomalous images (ie. dirty camera) for testing the detector.

The images can be defined in a configuration file:

Configuration file (TOML)

```
[transform]
resize = [216, 384]
mean = [0.00, 0.00, 0.00]
std = [1.00, 1.00, 1.00]

[camera_state]
clean = [
    "20230728-083036-IPC608_8B64_165.jpg",
    # ...
]
dirty = [
    "20230720-073036-IPC608_8B64_165.jpg",
    # ...
]
```

Use `torchvision` (or your preferred library) to load the images and convert them to tensors. It is recommended to resize the images to a smaller size (e.g., 216x384) to reduce computational cost and complexity. This can be done using the `torchvision.transforms` module.

Load your images (Python)

```
import tomllib

from PIL import Image
from torch.utils.data import Dataset
from torchvision import transforms

from obsea import config

class ImageDataset(Dataset):
    def __init__(self, image_paths, transform=None):
        self.image_paths = image_paths
        self.transform = transform

    def __len__(self):
        return len(self.image_paths)

    def __getitem__(self, idx):
        image = Image.open(self.image_paths[idx]).convert("RGB")
        if self.transform:
            image = self.transform(image)
        return image.to(config.device)
```

(continues on next page)

(continued from previous page)

```
transform = transforms.Compose([
    transforms.Resize(settings["resize"]),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=settings["mean"],
        std=settings["std"]
    ),
])

with open("config.toml", "rb") as f:
    settings = tomlib.load(f)

image_names = settings["camera_state"]["clean"]
image_paths = [images_parent / name for name in image_names]
dataset = ImageDataset(image_paths, transform=transform)
```

Once the pipeline to load the images and convert them to tensors is defined, we can proceed to the next step.

2. Choose the detection method

It's time to select the appropriate detection method for our usecase, based on the [Frouros table](#) of available methods:

- In our task, we want to analyze changes in data properties, not to evaluate a model's performance, so we need to select a **Data drift** detection method.
- Since our service processes one image per call (e.g., one image per day), we need a **Streaming** method.
- For image data with multiple features, a **Multivariate method** is required.
- As the input data is numerical, the method must support **numerical** data.

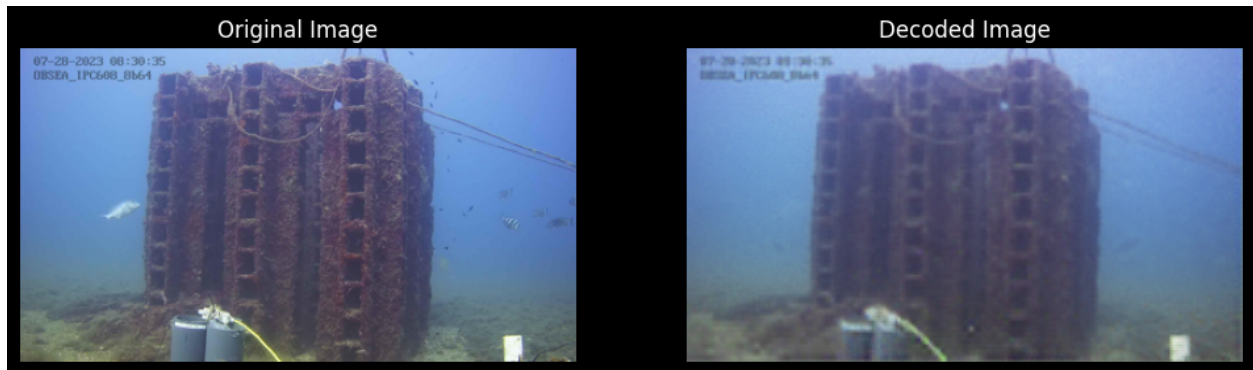
Based on this analysis, the best method is *Maximum Mean Discrepancy* (`MMDStreaming()`).

3. Train an autoencoder

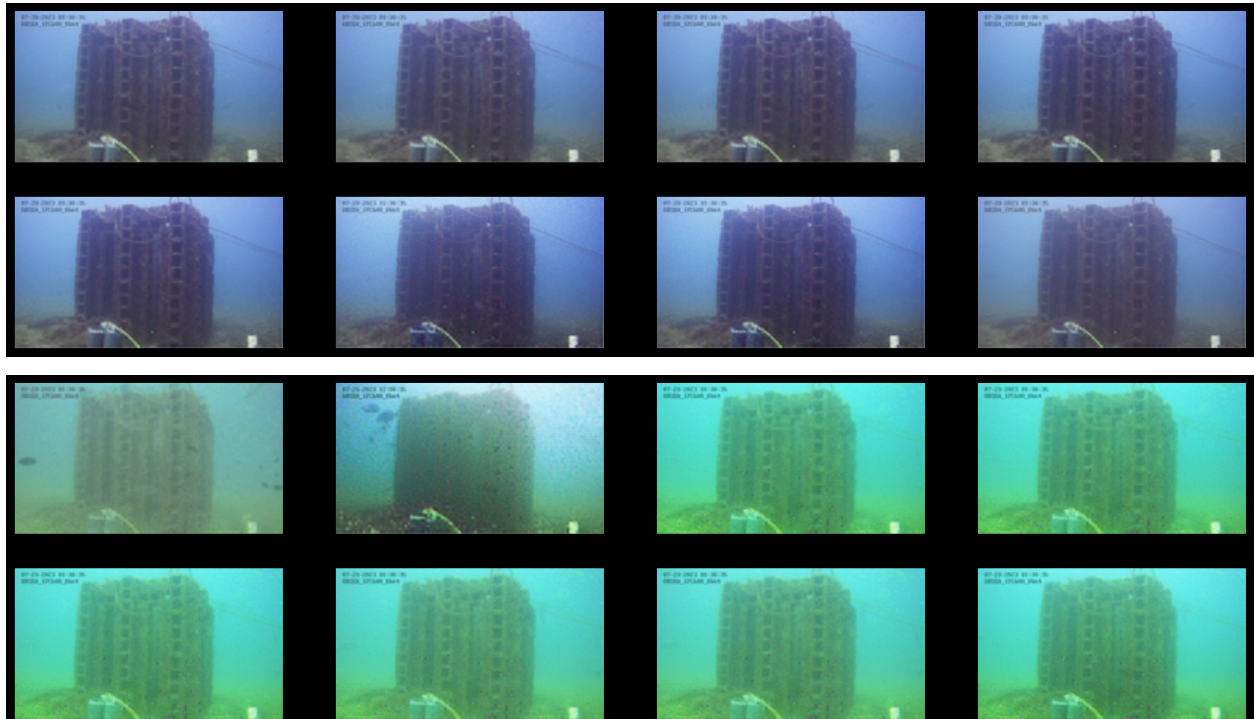
Tip

If your module does not involve image data, you can skip this step. You change the references in the text below from *clean embeddings* to *clean data*.

Drift detection struggles to understand images because of their high dimensionality (e.g. 224x224x3). To reduce computational cost and complexity, we can train an autoencoder to lower the dimensionality of the image data before feeding them to the drift detector.



This tutorial will not cover the details of training an autoencoder, but you can find many online tutorials on how to do it using [TensorFlow](#) or [PyTorch](#). What is important is to train the autoencoder with images, so that it learns to encode the clean (and ideally dirty) states of the camera.



At inference time, you will need to create the embeddings of the incoming images to pass them to the drift detector model. So you need to save the autoencoder weights in the *Storage* to be able to load them at inference time.

Additionally, you can also save the embeddings of clean camera images to warm the the drift detector at inference time, so it starts to detect drift from the first inference call.

Saving autoencoder and clean embeddings (Python)

```
# Load the autoencoder model
autoencoder = Autoencoder() # define your autoencoder architecture
train(autoencoder, dataset) # train the autoencoder on the dataset
autoencoder.eval()

# Generate embeddings for clean images
clean_embeddings = []
for image in dataset:
    with torch.no_grad():
        embedding = autoencoder.encoder(image.unsqueeze(0))
        clean_embeddings.append(embedding)

# Save the model weights and clean embeddings
torch.save(autoencoder.state_dict(), "/storage/autoencoder.pth")
torch.save(clean_embeddings, "/storage/clean_embeddings.pth")
```

4. Create and train the data drift detector

Using the [Frouros](#) library, we can create a drift detector that will monitor the incoming data and compare it with the reference data (clean embeddings). As defined in the previous step, we will use the `MMDStreaming()` method to detect drift in the data.

This method compares the distribution of incoming data with the reference data in real-time by using a sliding window approach. The first calls to `update()` will be used to fill the sliding window, and then the detector will start to compare the incoming data with the reference data. Due to this process, the first 12 calls to `update()` will not be used to detect drift and will return `None`. Optionally, we can warm up the detector by calling `update()` with the clean embeddings defined in the previous section.

Finally we define a threshold for the drift detection metric. If the metric exceeds the threshold, it indicates potential drift.

Implementing the detector (Python)

```
from functools import partial
from frouros.detectors.data_drift import MMDStreaming
from frouros.utils.kernels import rbf_kernel

detector = MMDStreaming(window_size=12, kernel=partial(rbf_kernel, sigma=0.3))
clean_embeddings = load_encodings(...)
detector.fit(clean_embeddings.cpu().numpy()) # Frouros expects numpy arrays

# Warm up the detector with clean embeddings
for embedding in clean_embeddings:
    detector.update(embedding.cpu().numpy())

# Now you can start monitoring incoming data
for image in incoming_images:
    with torch.no_grad():
        embedding = autoencoder.encoder(image.unsqueeze(0))
        drift_score, _ = detector.update(embedding.cpu().numpy())
        print(f"Drift score: {drift_score.distance}")
```

(continues on next page)

(continued from previous page)

```
# Define a threshold for drift detection
warning_threshold = 0.05 # Adjust this value based on your requirements
drift_threshold = 0.10 # Adjust this value based on your requirements

# Check for drift
if drift_score.distance > drift_threshold:
    print("Drift detected!")
elif drift_score.distance > warning_threshold:
    print("Warning: Drift score is approaching the threshold.")
```

We recommend simulating different scenarios (e.g., clean vs. dirty camera images) to set the appropriate threshold value. Ensure that it correctly identifies drift and triggers appropriate alerts.

Integrate the drift detector with the DEEPaaS API

Now that you have your detector ready you need to integrate it with the *DEEPaaS API* so that it will be used at inference time.

If you followed the steps in *Develop a model (tutorial)*, you should have a model the basic DEEPaaS functions, including: `warm()`, `get_predict_args()` and `predict()`.

Once this is done, you need to perform the following updates:

1. Update the warm function

In the `warm()` function, you need to initialize the drift detector with the clean embeddings, saved in the *Storage*. Note that the state of the detector is restarted every time the module is restarted.

```
def warm():
    # Load detector
    detector = MMDStreaming(window_size=12, kernel=partial(rbf_kernel, sigma=0.30))

    # Warm up the detector with clean data
    clean = load_encodings("/storage/clean_embeddings.pth")
    detector.fit(clean.cpu().numpy())
    for sample in clean[:utils.detector.window_size]:
        detector.update(sample.cpu().numpy())
```

2. Update the predict function

In the `predict()` function, you need to define the logic to monitor incoming data and check for drift. To do so, first, we need to define a schema that will be used to define and validate the incoming data.

Implementing predict schema (Python)

```
import marshmallow
from marshmallow import fields, validate

class PredArgsSchema(marshmallow.Schema):
    """Prediction arguments schema for api.predict function."""

    class Meta: # Keep order of the parameters as they are defined.
        ordered = True
```

(continues on next page)

(continued from previous page)

```

input_file = fields.Field(
    metadata={
        "description": "Image used to evaluate the data drift.",
        "type": "file",
        "location": "form",
    },
    required=True,
)
drift_distance = fields.Float(
    metadata={
        "description": "Minimum distance to consider data drift.",
    },
    load_default=0.125,
    validate=validate.Range(min=0.0),
)

def get_predict_args():
    return PredArgsSchema().fields()

```

As the arguments for inference are defined, we can proceed to implement the logic to monitor the incoming data.

The `predict()` function is called when the module is used to make predictions about the data drift status. The function will load the image, encode it using the autoencoder, and then use the drift detector to check if the image is clean or dirty. The function returns whether drift exists or not.

```

def predict(input_file, drift_distance):
    # Load the image and encode it
    image = load_image(input_file.filename)
    normalized = transform(image).to(config.device)
    encoded = autoencoder.encoder(normalized.unsqueeze(0))[0]

    # Check if the image is clean
    result, _ = detector.update(encoded.cpu().numpy())
    return {
        "drift": bool(result.distance > drift_distance),
    }

```

Monitor drift with Driftwatch

The previous section has showed how we could compute drift inside our predict function.

But for a better user experience, we have developed [DriftWatch](#) to visualize the drift over time in an interactive way. It allows to save the drift metrics for each inference call and plot them over time.

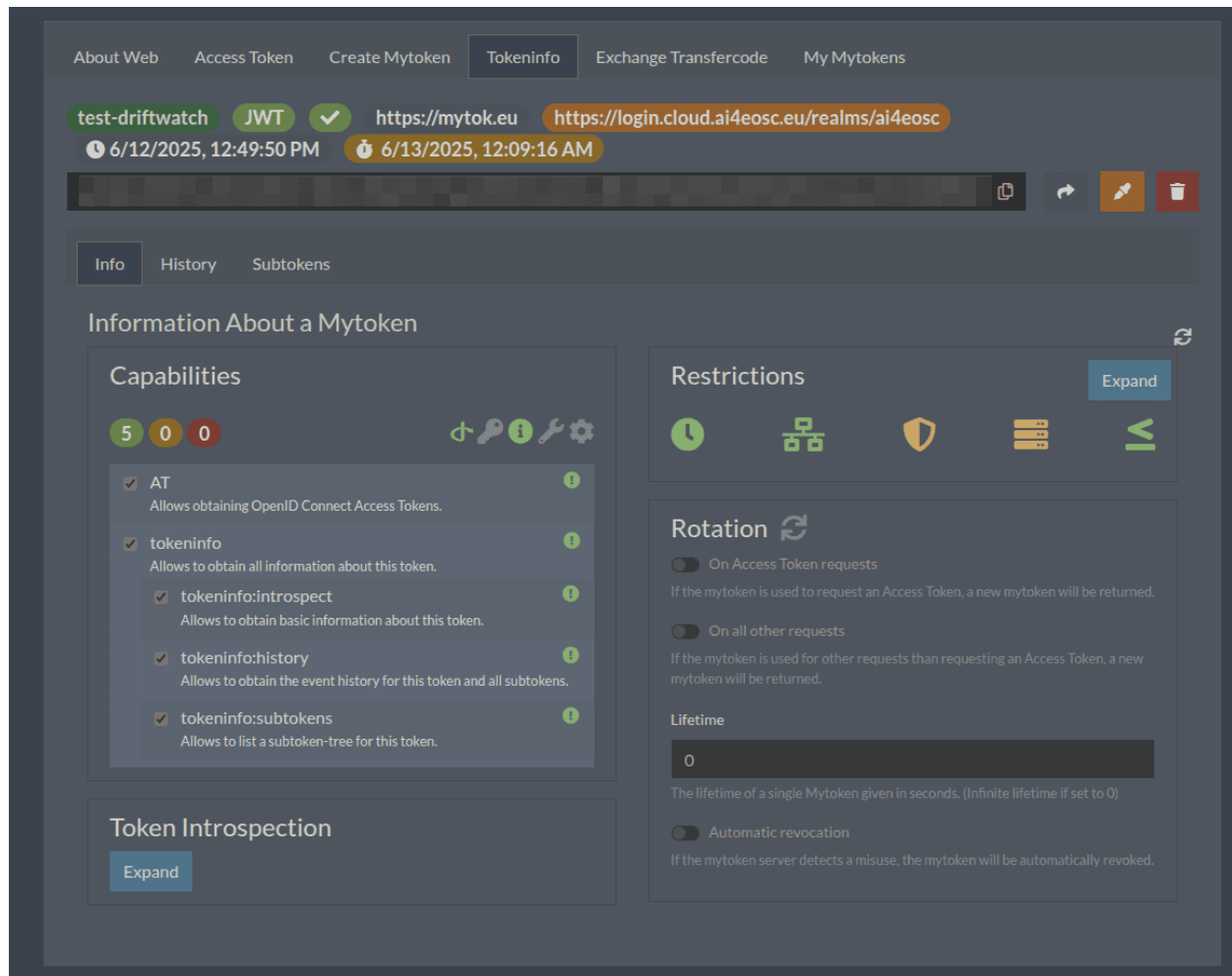
To connect your module with DriftWatch, follow these steps:

1. Obtain a MyToken to authenticate to the service

To store data into DriftWatch server, users need to authenticate. To do so, DriftWatch offers compatibility with federated authentication via [mytoken](#), a service which allows the use of OIDC based tokens with enhanced security and long life extensions.

To obtain your token:

1. Login into [mytoken](#) selecting the AI4EOSC provider
2. Go to Create MyToken:
 - Provide a Token name
 - Set Audiences to <https://drift-watch.cloud.ai4eosc.eu/>
 - Click on Create new Mytoken
3. This will open a new tab to approve the token. Once approved, switch back to the previous tab to see the token value.



2. Initialize DriftWatch in your module

Create an environment variable `DRIFT_MONITOR_MYTOKEN` and assign your mytoken to it.

To add the DriftWatch library to your module, you need to add the [drift-monitor](#) package to the requirements file. This package is used to connect your modules with DriftWatch and send the drift metrics to be monitored.

```
$ pip install -U drift-monitor
```

Once the package is installed, you need to accept the license agreement and register to be able to create experiments in the [DriftWatch](#) service. You do it at the start of the `api.py` file:

```
import drift-monitor as dw

dw.register(accept_terms=True)
description = "This is an experiment to track camera status on OBSEA project."
try:
    dw.new_experiment("obsea-camera", description, public=True)
except ValueError:
    print("Experiment already exists. Skipping creation.")
```

3. Update the predict function

Final step is to extend the `predict()` function with the functionality to upload your drift jobs to the [DriftWatch](#) server. To do so, you simply need to open a python context with `DriftMonitor()` defining a model id and the tags you want to use to identify your results on the experiment.

```
def predict(input_file, drift_distance):
    model_id, tags = config.data_version, config.tags
    parameters = {"some_parameter": "value"}
    ...
    # Check if the image using drift detection
    result, _ = detector.update(encoded.cpu().numpy())
    with dw.DriftMonitor("obsea-camera", model_id, tags) as monitor:
        result, _ = detector.update(encoded.cpu().numpy())
        parameters["distance"] = result.distance
        monitor(result.distance > drift_distance, parameters)
    ...
    return ... # format and return the results as before
```

Every time the inference calls the `predict()` function, a new job is opened at [DriftWatch](#). If an exception is raised during the execution of the code under the `DriftMonitor()` context, the job will be closed with **Failed** status. Otherwise, normal exit of the context will close the job as **Completed**.

4. Add additional context data to your drift

As you might have notice, the second parameter of the `monitor()` function is a dictionary with the parameters you want to add to your drift job. You can add any additional information you want to include in the job. For example, you can add the name of image that was used for the prediction, the drift distance, and any other information that you want to include in the job.

If your *deployment is mounted with storage*, you can save the images in the *storage* (accessible in the `/storage` folder). If you then include the image name in the drift parameters, you will be able to locate what was the image that caused the drift. The resulting `predict()` function would look as following:

```
# Init the image dir
image_dir = "/storage/test-driftwatch"
os.makedirs(image_dir, exist_ok=True)

def predict(input_file, drift_distance):
    ...
    # Save image to permanent storage
    timestamp = dt.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
    shutil.copy(input_file.filename, f"{image_dir}/{timestamp}.jpg")
    ...
```

(continues on next page)

(continued from previous page)

```
# Add image name to parameters
parameters["image_name"] = f"{timestamp}.jpg"
...
return ... # format and return the results as before
```

Deploy your module in production

In the module page, click on the option **Codespaces > Jupyter**. You will be shown a *configuration page* where the option **Jupyter** is selected. Make sure to *connect your storage* to be able to retrieve the weights of the drift detector. Then submit the deployment.

In the Deployments tab, go to the Modules table and find your created deployment. Click the **Quick** access to access the JupyterLab terminal.

Now we need to define the mytoken variable as envvar:

```
$ export DRIFT_MONITOR_MYTOKEN=<your_token>
```

Now we can *start the DEEPaaS API*:

```
$ deep-start --deepaas
```

Once the module is running, you can use the **POST .../predict** method to send an image to the module and check if it is clean or dirty.

Access to **DriftWatch** in order to visualize the uploaded drift in the dashboard.

Name	Created	Public	Actions
experiment_1	01/01/2020, 01:00:00	false	View
experiment_2	01/01/2021, 01:00:00	true	View
experiment_3	02/01/2021, 01:00:00	false	View
experiment_4	01/01/2022, 01:00:00	false	View
Coastal data drift detection	29/01/2025, 14:25:42	true	View
CYGNSS	29/01/2025, 20:02:14	false	View
test1	03/03/2025, 14:38:04	false	View
obsea-camera	06/05/2025, 06:30:31	true	View

Click on your experiment and you will be shown a list of the drift jobs that have been uploaded. You can select the desired jobs and configure the visualization options to see the drift distance over time.



Use the View button to see the saved parameters of a particular inference call.

☐	Created	Status	Tags	Drift	Schema	Actions
☐	13/05/2025, 14:58:12	Completed	data-drift, DDM, example_service, streaming	Yes	1.0.0	View
☐	13/05/2025, 15:18:52	Completed	data-drift, DDM, example_service, streaming	Yes	1.0.0	View

Parameters

```
{
  "distance": 0.2785177770742673,
  "drift_distance": 0.125,
  "link": "https://localhost:8000?path=2025-05-13_17-18-50",
  "warning": true,
  "warning_distance": 0.1
}
```

 Co-funded by the European Union
 

6.1.4 Incremental (online) Learning

Usually, when developing machine or deep learning models we can encounter problems in the assumption of training them with non-stationary stream data. In many research papers this type of learning is referred to as *continuous/incremental/online learning*.

Incremental learning arises in order to solve the problem of loss of predictive capability of the models in cases of non-stationary streams of data. This is important in cases where the trend of the data can change quickly, as well as their distributions.

The problem is related to data drift detection in machine or deep learning scenarios, where one aims to detect in the distribution of data during the inference step is varies from the one used for training. For more information on data drift, we recommend to checking [Frouros](#), a Python library for data drift and concept drift detection.

Specifically, in an incremental learning scenario, the model to be trained does not have all the data at the beginning, but they arrive sequentially or in different steps, which may cause the distributions to vary over time. The idea is that the model is able to adapt to new data without losing the knowledge previously acquired.

From a theoretical point of view, three types of incremental learning data are [proposed](#):

- task-incremental learning,
- domain-incremental learning,
- class-incremental learning.

Frameworks

Using River

In particular, if you are applying machine learning models such as those available in [scikit-learn](#), you can use [river](#) to migrate your model from batch learning (classic) to incremental (or online) learning.

We recommend reviewing the [AirlinePassengers example](#), where online learning is applied to the widely known AirlinePassengers dataset (that contains the number of air-passengers per month), reproduced below.

1. Install river and pytest (this dependency is needed):

```
$ pip install river
$ pip install pytest
```

2. Create the pipeline for processing the data, scaling and creating the model

```
from river import compose
from river import linear_model
from river import preprocessing
from river import metrics
from river import utils
import calendar

def get_ordinal_date(x):
    return {'ordinal_date': x['month'].toordinal()}

def get_month(x):
    return {
        calendar.month_name[month]: month == x['month'].month
        for month in range(1, 13)
    }

model = compose.Pipeline(
    # Features used as input for the model
    ('features', compose.TransformerUnion(
        ('ordinal_date', compose.FuncTransformer(get_ordinal_date)),
        ('month', compose.FuncTransformer(get_month)),
    )),
    # Scaling the data
    ('scale', preprocessing.StandardScaler()),
    # Applying the model
    ('lin_reg', linear_model.LinearRegression(intercept_lr=0))
)
```

3. Evaluate the model applied using online learning and return the real values and the predictions

```
def evaluate_model(model):
    metric = utils.Rolling(metrics.MAE(), 12)

    y_true = []
    y_pred = []
    for x, y in datasets.AirlinePassengers():
        # Obtain the prior prediction and update the model with the next data
        pred = model.predict_one(x)
        model.learn_one(x, y)

        # Update the error metric
        metric.update(y, pred)

        # Save the true value and the prediction
        y_true.append(y)
        y_pred.append(pred)

    return y_true, y_pred
```

Using Tensorflow

The biggest problem with batch learning is that you need to retrain your model frequently (periodically in the presence of new data). Online or incremental learning is especially useful in environments where data arrives continuously or storing and processing large data sets all at once is not feasible.

However, it is important to note that in most cases, for example when we are applying a deep learning model using TensorFlow, the batch learning will be sufficient and it is not necessary to apply online learning. Instead, we can retrain the model in mini-batches in case of new data or changes in the trend of the data. In this line, we will apply incremental training as a learning technique where a model is updated and improved over time by training it on a new data while retaining from previous learned patterns. In this case, the deep learning model will be updated regularly with new data.

For example, using tensorflow, to solve this problem of new data arriving, it is possible to explore the possibility of applying a specific retraining of the model. The following is an example for the MNIST dataset.

1. Read and process the data. Let's assume that initially we only have 70% of the data (x_train_ini, y_train_ini). We train the initial model and save it.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv2D, MaxPooling2D, Dense, Flatten
from tensorflow.keras.datasets import mnist
import time

# Read and process:
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

# First training data used:
x_train_ini = x_train[:int(0.7*len(x_train))]
y_train_ini = y_train[:int(0.7*len(x_train))]
```

2. Define and compile the model

```
model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(32, kernel_size=(3, 3), activation="relu"),
    MaxPooling2D(pool_size=(2, 2)),
    Conv2D(64, kernel_size=(3, 3), activation="relu"),
    MaxPooling2D(pool_size=(2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

3. Train the model with the first batch of data available. Save the initial model

```
model.fit(x_train, y_train, epochs=10, batch_size=16, validation_split=0.2)
model.save('initial_model.h5')
```

4. Create a logic to get new data (in this case we use `x_train_new` and `y_train_new` as new data), in a real case we should get that new data currently available.

```
def get_new_data(x_train, y_train):
    # In a real scenario here the new data that are available should be obtained.
    # In this case, we take the remaining 30% of the data from the train set
    x_train_new = x_train[int(0.7*len(x_train)):]
    y_train_new = y_train[int(0.7*len(x_train)):]
    return x_train_new, y_train_new
```

5. Re-train the model periodically with the new data that will be obtained (in this case we do not obtain new data but take the final 30% of the train set, but this should be changed in a real case where new data are periodically available):

```
while True:
    x_train_new, y_train_new = get_new_data(x_train, y_train)

    if x_train_new is not None and y_train_new is not None:
        model = tf.keras.models.load_model('updated_model.h5')
        model.fit(x_train_new, y_train_new, epochs=5, batch_size=16, validation_split=0.
→2)
        model.save('updated_model.h5')
        print(f'Accuracy test: {model.evaluate(x_test, y_test)[1]}')

    # Repeat the process of acquiring new data and re-training the model periodically (e.
→g. every one hour).
    # Note: new data must be obtained, in this example the final 30% of the train set
→would be retaken.
    time.sleep(3600)
```

Integrating online learning in your modules

Online learning can be integrated in two ways:

1. Create a cronjob in your train deployment that periodically parses your Nextcloud and retrains in the new data found
2. As it to the `deepaas.predict()` to retrain the model at inference time on the new data it receives.

6.1.5 Distributed Learning

Essentially, carrying out the training of AI/ML/DL models in a distributed way can be done due to two fundamental reasons: the first one is that there are privacy issues that prevent to centralize the data in a single site, the second one is that due to the need of computational resources the training must be distributed in different workers (e.g. the model or the data do not fit in memory). Thus, we can consider two main paradigms to carry out distributed machine learning (DML):

- **Data parallelism:** this is the most common case, and is used when the data available for training the model does not fit in memory in a single node or worker. Thus, the data is distributed among the different available workers or machines. Then, the model is trained on each data subset and the resulting model is aggregated using the parameters obtained in each case.
- **Model parallelism:** the model, usually a neural network, is split and distributed among different available computation nodes. An important aspect here is to ensure a correct synchronization and coordination between the different workers when performing gradient descent and backward propagation. This method is used in case of very large models that do not fit in memory.

From the point of view of the development and training of a model in a distributed way using the platform, the following solutions are proposed depending on the user's needs:

- **The data is distributed in different sites naturally or artificially outside the platform.** There are different computing nodes allocating the different datasets. For this approach, we propose to perform the distributed training using the federated server deployed on the platform, since the federated learning (FL) architecture can be seen as an specific case of DML where data parallelism (data distributed on different nodes) is performed. Thus, in each computing node or worker, a client of the federated architecture will be created, and a deployment will be created in the platform using the federated server tool to be in charge of orchestrating the distributed architecture.
- **The data is distributed naturally or artificially and can be uploaded to the platform.** In this case, as in the previous one, the user can distribute each dataset to a deployment created on the platform, each of these constituting a worker in a DML architecture. As in the previous case, to carry out the distributed training in this case, the federated server can be used, deploying this tool in another deployment and creating a client in each of the deployments provided with distributed data previously created.
- **The user wants to train a model on a large dataset. The model is complex enough to require the use of multiple GPUs.** You would create a multi-GPU deployment in the platform, upload the data to it and train the model in a distributed way using [horovod](#). Below is an example that can be followed, specifically applied to the [CIFAR100](#) dataset using [Tensorflow](#) (it would be enough to replace the configuration related to this use case to extrapolate it to the user case).

Note :class: info

Due to the restricted number of GPUs, for optimal usage, we currently do not support creating multi-GPU deployments.

```
import tensorflow as tf
import horovod.tensorflow as hvd
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D
from tensorflow.keras.layers import BatchNormalization
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

# Initialize Horovod
hvd.init()

# Pin GPU to be used to process local rank (one GPU per process)
gpus = tf.config.experimental.list_physical_devices('GPU')
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)
if gpus:
    tf.config.experimental.set_visible_devices(gpus[hvd.local_rank()], 'GPU')

# Read the process the data
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar100.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
y_train = to_categorical(y_train, 100)
y_test = to_categorical(y_test, 100)
```

```

dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))
dataset = dataset.repeat().shuffle(10000).batch(128)

# Create the model to be trained:
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(100, activation='softmax')
])

loss = tf.losses.CategoricalCrossentropy()
opt = tf.optimizers.Adam(0.001 * hvd.size())
metrics = ['accuracy']
model.compile(optimizer=opt, loss=loss, metrics=metrics)

checkpoint_dir = './checkpoints'
checkpoint = tf.train.Checkpoint(model=model, optimizer=opt)

@tf.function
def training_step(x_train, y_train, first_batch):
    with tf.GradientTape() as tape:
        probs = model(x_train, training=True)
        loss_value = loss(y_train, probs)

    # Add Horovod Distributed GradientTape.
    tape = hvd.DistributedGradientTape(tape)
    grads = tape.gradient(loss_value, model.trainable_variables)
    opt.apply_gradients(zip(grads, model.trainable_variables))

    # Broadcast initial variable states from rank 0 to all other processes.
    # This is necessary to ensure consistent initialization of all workers when
    # training is started with random weights or restored from a checkpoint.
    # Note: broadcast should be done after the first gradient step to ensure optimizer
    # initialization.
    if first_batch:
        hvd.broadcast_variables(model.variables, root_rank=0)
        hvd.broadcast_variables(opt.variables(), root_rank=0)

    return loss_value

# Horovod: adjust number of steps based on number of GPUs.
for batch, (x_train, y_train) in enumerate(dataset.take(10000 // hvd.size())):
    loss_value = training_step(x_train, y_train, batch == 0)

    if batch % 10 == 0 and hvd.local_rank() == 0:
        print('Step #%d\tLoss: %.6f' % (batch, loss_value))

```

```
# Save checkpoints only on one worker (e.g. worker 0):  
if hvd.rank() == 0:  
    checkpoint.save(checkpoint_dir)
```


TRAIN A MODEL

This section will guide you through the process of training a model with the Dashboard, and some other aspects that influence the training process, like labeling your images with CVAT, or using rclone to sync your dataset. Also, you will learn how to make a federated learning training.

Train a model in standard mode Learn how to train a model in standard mode.

Train in standard mode Train a model in batch mode Learn how to train a model in batch mode.

Train in batch mode Run a federated learning training Learn how to make a federated learning training, both with the Flower and NVFLARE frameworks

Train using Federated Learning Label your images with CVAT Learn how to label your images with CVAT.

Labeling images with CVAT

7.1 Training

This page serves a guide on the different options to train a model in the platform.

7.1.1 Training options

There are currently three main options to train a model in the platform:

- *standard mode*: you are given access to a persistent deployment that you can interact with via an IDE (ie. VS-code).
- *batch mode*: you deploy a temporary job that runs your training and then is killed when the training is completed
- *federated mode*: you deploy a federated learning server that orchestrates the training. Then you can have several clients joining forces to distribute the training load among all of them.

All these options have the respective pros and cons.

Table 1: Training options from the Dashboard

Option	Pros	Cons
<i>Standard mode</i> (persistent deployment)	It's very convenient to edit and run directly from an IDE,	The GPU is dedicated to you full time, even when you are not actually training. Thus resources are not used optimally.
<i>Batch mode</i> (temporary jobs)	You are only using resources for as long as your training needs to run. <i>To promote the usage of batch mode between users, we have dedicated Tesla V100 GPU nodes exclusively devoted to batch mode.</i>	Less convenient because you cannot debug from IDE.
<i>Federated mode</i>	- You can scale your training across many deployments, mixing both GPU and CPU deployments, both inside and outside the platform. - Your training data remain local, so it's a perfect match for privacy respecting usecases (eg. healthcare).	- Not all modules can be used for default in federated mode, you need to adapt their code first. - It might require a bit more work to setup.

Given the above specifications, we recommend the following typical workflows:

- *Use standard mode* for you preliminary trainings, when you still might need to have direct access to the code/data to debug things.
- *Use batch mode* when your training script is stable, and you are basically tweaking hyperparameters.
- *Use federated mode* if you have sensitive data and/or need to distribute you training across many machines.

Note

Bear in mind that all the modules of the platform are fully open-source, as they are packaged as Docker containers. So if you have access to your own HPC resources, you can always take the container and train it there using [udocker](#).

7.1.2 Additional training-related tasks

We have a tutorials on additional training-related tasks:

- *Label your sdata with CVAT*
- *Use MLflow to track your training runs*

If you are new to Machine Learning, you might want to check some *useful Machine Learning resources* we compiled to help you getting started.

7.2 Train in standard mode

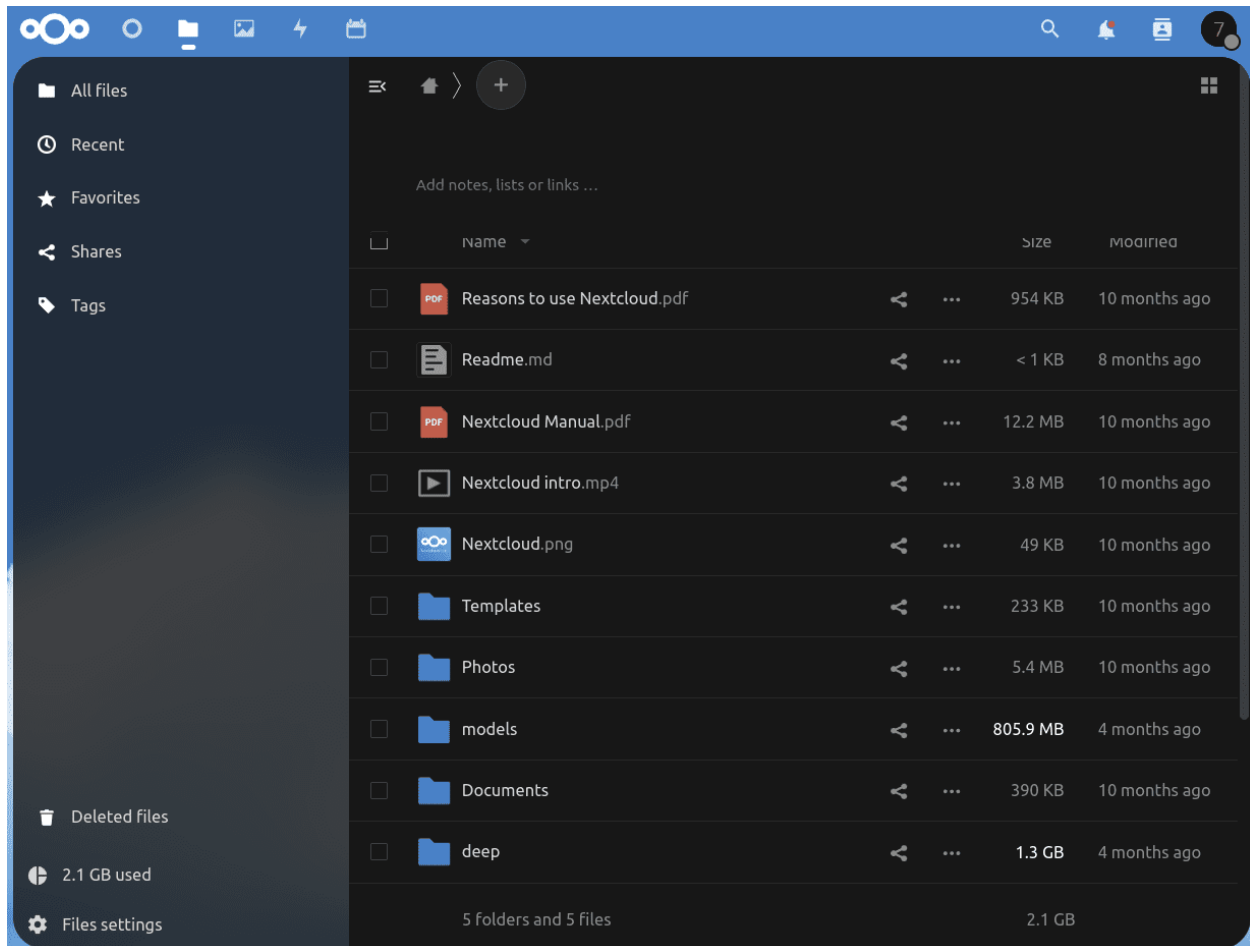
Requirements

- You need a *platform account* with *full access level* to be able to access both the Dashboard and Nextcloud storage.
- For **Step 8** we recommend having [docker](#) installed (though it's not strictly mandatory).

This is a step by step guide on how to train a model with your own dataset in standard mode. In standard mode, you will get a persistent deployment that you will be able to interact with via an IDE.

7.2.1 1. Upload your dataset to Nextcloud

For this example we are going to use the [AI4EOSC Nextcloud](#) for storing the dataset you want to retrain the model with. So login to Nextcloud with your credentials and you should access to an overview of your files.



Now it's time to upload your dataset. When training a model, the data has usually to be in a specific format and folder structure. It's usually helpful to read the README in the source code of the module (in this case [located here](#)) to learn the correct way to setting it up.

In the case of the **image classification module**, we will create the following folders:

- A folder called `models` where the new training weights will be stored after the training is completed
- A folder called `data` that contains two different folders:
 - The sub folder `images` containing the input images needed for the training
 - The sub folder `dataset_files` containing a couple of files:
 - * `train.txt` indicating the relative path to the training images
 - * `classes.txt` indicating which are the categories for the training

Again, the folder structure and their content will of course depend on the module to be used.

Once you have prepared your data locally, you can drag your folder to the Nextcloud Web UI to upload it.

i Uploading tips

- If you need to upload your dataset from a remote machine (ie. no GUI), you can *install rclone* on your remote machine, *configure it* and do an *rclone copy* to move your data to Nextcloud.
- Uploading to Nextcloud can be particularly slow if your dataset is composed of lots of small files. Considering zipping your folder before uploading.

```
$ zip -r <foldername>.zip <foldername>
$ unzip <foldername>.zip
```

7.2.2 2. Prepare your training environment

In this tutorial we will see how to retrain a [generic image classifier](#) on a custom dataset to create a [phytoplankton classifier](#). If you want to follow along, you can download the toy phytoplankton dataset [here](#).

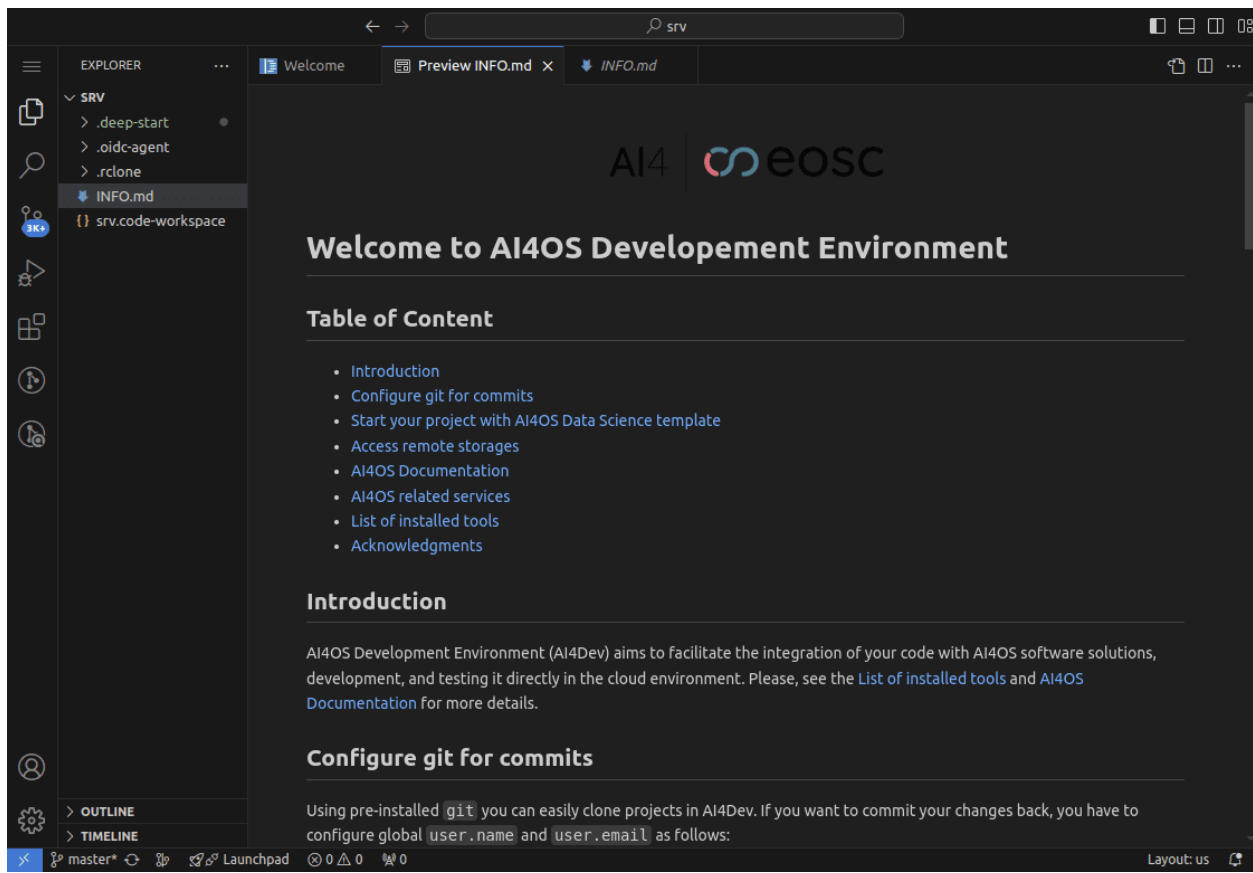
The first step is to choose a model from the [Dashboard](#). Make sure to select a module with the `AI4 trainable` tag. For educational purposes we are going to retrain a [generic image classifier](#). Some of the model dependent details can change if using another model, but this tutorial will provide a general overview of the workflow to follow when using any of the modules in the Dashboard.

Check [how to configure](#) the image classifier. During the configuration, you should make sure:

- to select either JupyterLab or VScode as the **service** to run, because we want the flexibility of being able to interact with the code and the terminal, not just the API.
- to select GPU as **hardware**, because training is a very resource consuming task. This will also imply that you might need to select a **Docker tag** that is compatible with GPUs.
- to connect with one of your synced **storage providers** (in our case, the project's Nextcloud instance)

7.2.3 3. Access your deployment

After submitting you will be redirected to the deployment's list. In your new deployment, select `Info` and click in the IDE endpoint, when it becomes active. After logging in you should be able to see your IDE:



Now, open a Terminal to perform some sanity checks:

- **Check the GPU is correctly mounted:**

```
$ nvidia-smi
```

This should output the GPU model along with some extra info.

- **Your storage is correctly mounted:**

```
$ ls /storage
```

This should output your Nextcloud folder structure.

Accessing storage

Your files under `/storage` are mounted via a virtual filesystem. This has *pros and cons*. We also offer the possibility to *copy the files to the local machine* as long as they fit the available disk.

7.2.4 4. Start training the model

We will use the DEEPaaS API to interactively run the training. In your Terminal type:

```
$ nohup deep-start --deepaas &
```

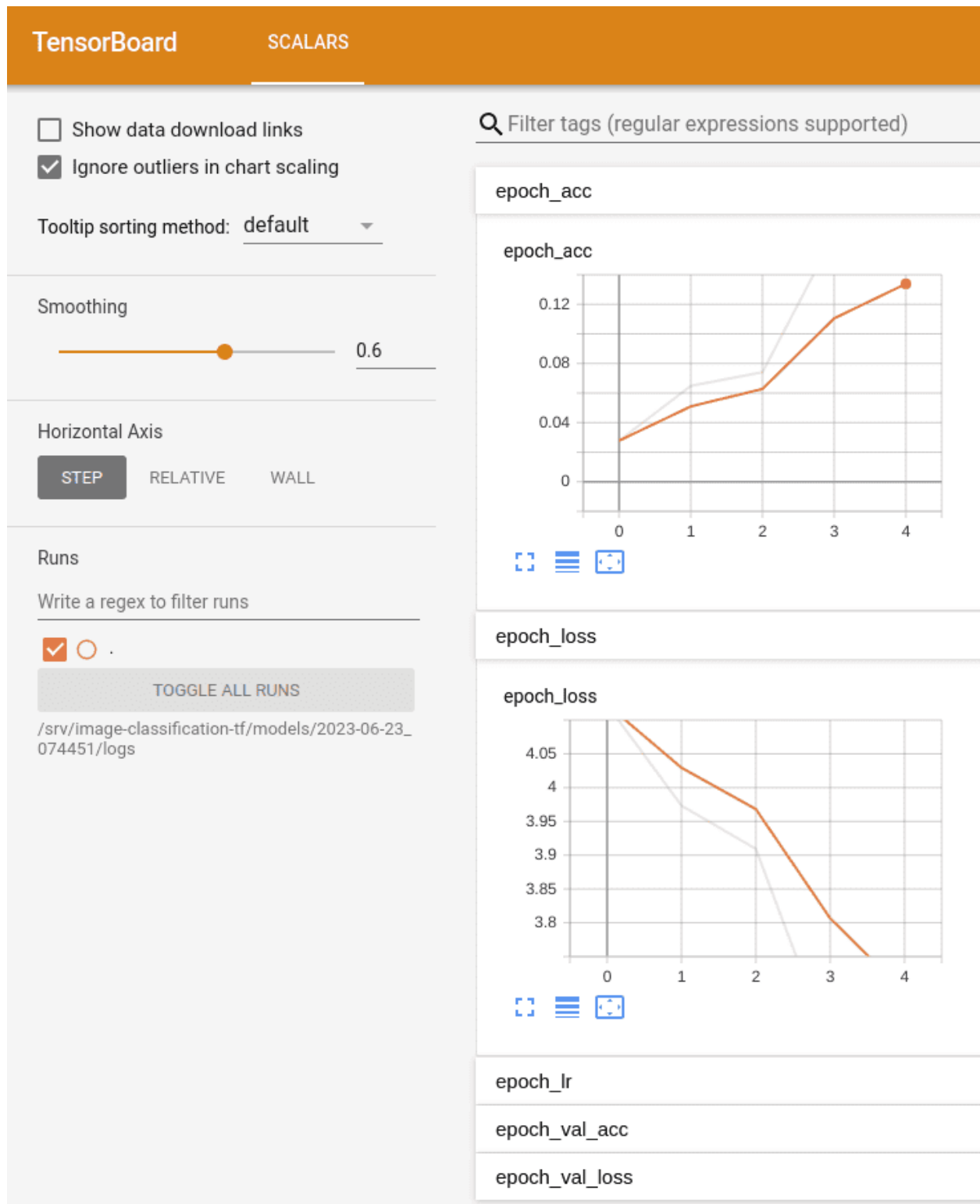
The `&` will keep your command running even if you close the terminal, and `nohup` will produce a log file `nohup.out` that you can always look at if you want to know what is going on under the hood.

Now go back to the Dashboard, in the Deployments list view. In your deployment go to **Info** and click on the API active endpoint.

models		
GET	/v2/models/	Return loaded models and its information
GET	/v2/models/imgclas/	Return model's metadata
POST	/v2/models/imgclas/train/	Retrain model with available data
GET	/v2/models/imgclas/train/	Get a list of trainings (running or completed)
GET	/v2/models/imgclas/train/{uuid}	Get status of a training
DELETE	/v2/models/imgclas/train/{uuid}	Cancel a running training
POST	/v2/models/imgclas/predict/	Make a prediction given the input data

Look for the **train** POST method. Modify the training parameters you wish to change and execute. In our case, you might need to correctly point to the training dataset location.

If some kind of monitorization tool is available for the module, you will be able to follow the training progress at **Monitor** active endpoint. In the case of the image classification module, you can monitor training progress with Tensorboard.



```
$ pid=$(pgrep -f deepaas-run) && child_pids=$(pgrep -P $pid) && kill -9 $pid $child_pids
```

7.2.5 5. Test and export the newly trained model

Once the training has finished, you can directly test it by clicking on the `predict` POST method. For this you have to kill the process running `deepaas`, and launch it again.

```
$ kill -9 $(ps aux | grep '[d]eepaas-run' | awk '{print $2}')
$ kill -9 $(ps aux | grep '[t]ensorboard' | awk '{print $2}') # optionally also kill
↳ monitoring process
$ nohup deep-start --deepaas & # relaunch
```

Note

We need to do this because the user inputs for `deepaas` are generated at the `deepaas` launching. Thus the original `deepaas` process is not aware of the newly trained model.

Once `deepaas` is restarted, head to the `predict` POST method, select your new model weights and upload the image you want to classify.

If you are satisfied with your model, then it's time to save it into your remote storage. Open a Terminal window and run:

```
$ cd /srv/ai4os-image-classification-tf/models
$ tar cfJ <modelname.tar.xz> <foldername> # create a tar file
$ cp <modelname.tar.xz> /storage/ # save to storage
```

Now you should be able to see your new models weights in Nextcloud. For **Step 8**, you will need to download the weights from the Dockerfile. To allow this, make the weights atr file [publicly available](#). For this, click on `Share Link` (Create a new share link)

Zenodo preservation

Optionally, in order to improve the reproducibility of your code, we encourage you to share your training dataset on [Zenodo](#). Once you upload the dataset, make sure to link it with the relevant Zenodo community ([AI4EOSC](#), [iMagine](#)).

If long-term preservation and versioning of model weights is important to you, you can also upload the model weights to [Zenodo](#) in addition to Nextcloud.

7.2.6 6. Create a repo for your new module

Now, let's say you want to share your new application with your colleagues. The process is much simpler that when *developing a new module from scratch*, as your code is the same as the original application, only your model weights are different.

To account for this simpler process, we have prepared a version of the the AI Modules Template specially tailored to this task:

- Go to the [Template creation webpage](#). You will need an [authentication](#) to access to this webpage.
- Then select the `child-module` branch of the template and answer the questions.
- Click on `Generate` and you will be able to download a `.zip` file with the project's directory. Extract it locally.

7.2.7 7. Update your project's metadata

The module's metadata is located in the `ai4-metadata.yml` file ([example](#)). This is the information that will be displayed in the Marketplace. The fields you need to edit to comply with our [schemas](#) are:

- **title** (*mandatory*): short title,
- **summary** (*mandatory*): one liner summary of your module,
- **description** (*optional*): extended description of your module, like a README,
- **links** (*mostly optional*): links to related info (training dataset, module citation. etc),
- **tags** (*mandatory*): relevant user-defined keywords (can be empty),
- **categories, tasks, libraries, data-type** (*mandatory*): one or several keywords, to be chosen from a closed list (can be empty).

Supported values

Libraries	Tasks	Categories	Data Type
TensorFlow	Computer Vision	AI4 pre trained	Image
PyTorch	Natural Language Processing	AI4 trainable	Text
Keras	Time Series	AI4 inference	Time Series
Scikit-learn	Recommender Systems	AI4 tools	Tabular
XGBoost	Anomaly Detection		Graph
LightGBM	Regression		Audio
CatBoost	Classification		Video
Other	Clustering		Other
	Dimensionality Reduction		
	Generative Models		
	Graph Neural Networks		
	Optimization		
	Reinforcement Learning		
	Transfer Learning		
	Uncertainty Estimation		
	Other		

- **inference** (*optional*): this is the minimum resources your module needs to run an inference correctly (eg. CPU cores, RAM, GPUs, etc). If not specified, the Dashboard will prefill with some defaults, that can later be adapted by the user during the [configuration step](#).
- **provenance** (*optional*): this will allow your model to have a more rich provenance information, as your model provenance graph will show the resources and the hyper-parameters you used to train. There are two subfields you can specify:
 - `nomad_job`: the Dashboard deployment UUID you used to train the final model,
 - `mlflow_run`: the MLflow run UUID you used to train the final model,

Some fields are pre-filled via the AI Modules Template and usually do not need to be modified. Check you didn't mess up the YAML definition by running our [metadata validator](#):

```
pip install ai4-metadata
ai4-metadata validate ai4-metadata.yml
```

7.2.8 8. Update your project's Dockerfile

Your `./Dockerfile` is in charge of creating a docker image that integrates your application, along with deepaas and any other dependency.

You will see that the base Docker image is the image of the original repo. Modify the appropriate lines to replace the original model weights with the new model weights. In our case, this could look something like this:

```
ENV SWIFT_CONTAINER https://share.cloud.ai4eosc.eu/index.php/s/r8y3WMK9jwEJ3Ei/download
ENV MODEL_TAR phytoplankton.tar.xz

RUN rm -rf ai4os-image-classification-tf/models/*
RUN curl --insecure -o ./image-classification-tf/models/${MODEL_TAR} \
    ${SWIFT_CONTAINER}/${MODEL_TAR}
RUN cd ai4os-image-classification-tf/models && \
    tar -xf ${MODEL_TAR} &&\
    rm ${MODEL_TAR}
```

Check your Dockerfile works correctly by building it locally and running it:

```
$ docker build --no-cache -t your_project .
$ docker run -ti -p 5000:5000 -p 6006:6006 -p 8888:8888 your_project
```

Your module should be visible in <http://0.0.0.0:5000/ui>

7.2.9 9. Integrating the module in the Marketplace

Once your repo is set, it's time to integrate it in the *Marketplace*:

1. Open an issue in the [AI ModuleCatalog](#) repo.
2. An platform admin will create the Github repo for your module inside the [ai4os-hub](#) organization. You will be granted write permissions in that repo.

Naming conventions

Modules repos follow the following convention:

- `ai4os-hub/ai4-<project-name>`: module officially developed by the project
 - `ai4os-hub/<project-name>`: modules developed by external users
3. Upload your code to that repo.
 4. An admin will review your code and add it to the AI Module Catalog. Once a module is approved it will take roughly 6 hours to appear in the Dashboard's Marketplace.

Next steps

If to go further, check our tutorials on how to:

- run a federated learning training

7.3 Train in batch mode

i Requirements

You need a *platform account* with *full access level*.

This is a step by step guide on how to train a model with your own dataset in batch mode. In batch mode, you will create a temporary job that is killed after the training is completed.

We are aware that batch mode is not as convenient for end users as *standard mode*, where you are given a persistent deployment that you can use with an IDE. Nevertheless, batch mode is preferable platform-wise because resources are optimally allocated.

To promote the usage of batch mode between users, we have **dedicated Tesla V100 GPU nodes** exclusively devoted to batch mode.

Batch mode can be used very similarly to *standard mode*, most of the tutorial still applies:

1. You start by preparing your data,
2. You create a batch job with the commands you want to run,
3. Your model weights are saved in the Storage when the job is completed,
4. You finally follow the same steps to add your model to the Marketplace,

7.3.1 What can I deploy in batch mode?

You can deploy any existing module in the marketplace. To create a batch job, click on the **Batch** button in the *module detail*.

Train an image classifier

Try ▾

Deploy ▾

Codespaces ▾

Batch

Train your own image classifier with your custom dataset. It comes also pretrained on the 1K ImageNet classes.

The deep learning revolution has brought significant advances in a number of fields [1], primarily linked to image and speech recognition. The standardization of image classification tasks like the [ImageNet Large Scale Visual Recognition Challenge](#) [2] has resulted in a reliable way to compare top performing architectures.

This Docker container contains the tools to train an image classifier on your personal dataset. It is a highly customizable tool that let's you choose between tens of different [top performing architectures](#) and training parameters.

The container also comes with a pretrained general-purpose image classifier trained on ImageNet. The PREDICT method expects an RGB image as input (or the url of an RGB image) and will return a JSON with the top 5 predictions.



References

1. Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. [Deep learning](#). Nature, 521(7553):436-444, May 2015.
2. Olga Russakovsky et al. [ImageNet Large Scale Visual Recognition Challenge](#). International Journal of Computer Vision (IJCV), 115(3):211-252, 2015.

Apache-2.0

build passing

Tasks

Computer Vision

Classification

Libraries

TensorFlow

Keras

Data Types

Image

Tags

deep learning

image classification

vo.imagine-ai.eu

general purpose

Platform Categories

AI4 trainable

AI4 pre trained

AI4 inference

Provenance

View

Download RDF

Additional Resources

GitHub

Report issue

Dataset

Docker

Weights

99 Citation

Dates

2024-04-25

2025-05-22

It's also possible to *redploy from a snapshot*. In this case, a typical workflow would be for example:

1. to develop a model in a CPU deployment using VScode,
2. when you feel the model is ready to be trained, create a snapshot of your deployment,

3. in the snapshot table, redeploy your snapshot in batch mode

7.3.2 Configuring a batch job

Batch mode is very similar to standard mode configuration. The main differences are the following.

Storage

In batch mode, it is mandatory to *connect a storage* to your job. This is done because:

- you need data to train, usually located in the storage (not in the Docker image itself)
- since the job is killed when it concludes, you cannot directly access the job to retrieve any outputs. So you need to **save your modelweights in your storage** to be able to retrieve them afterwards.

Batch commands

These are the commands your job will execute. In the configuration form, they can be provided either via a file upload or by adding the commands into an integrated editor.

Batch options

Upload file

Write bash commands

 Select file*

Supported formats: .sh, .bash, .command, .run

In the following dummy example, we can save the current date in our storage, mimicking the saving of the modelweights at the end of a real training.

```
echo "Test started"
date > /storage/test-batch.txt
sleep 20
```

7.3.3 Listing batch jobs

In the Batch tab on the left of the Dashboard, you will be able to see what are your current batch jobs, as well as the batch jobs that completed in the last 24 hours.

Dashboard

Statistics

Catalog

Modules

Tools

LLMs

Tasks

Try me

Deployments

Batch training

Inference

Batch training

Name	Status	Container name	Creation time (UTC)	Actions
	running	ai4oshub/ai4os-demo-app:latest	2025-05-19 12:43:58	🔍 🗑️
	complete	ai4oshub/ai4os-demo-app:latest	2025-05-19 09:50:49	🔍 🗑️
	queued	ai4oshub/ai4os-demo-app:latest	2025-04-30 14:40:17	🔍 🗑️

Clicking on the job details you will see the resources as well as the training commands that the job executed.

Batch deployment detail

status running

Resources

CPU frequency: 8000 MHz

Number of CPUs: 4

Disk memory: 10000 MB

Number of GPUs: 0

RAM memory: 8000 MB

Datacenter

ifca-ai4eosc

Endpoints

-

Batch details

Commands

```
1
2 echo "Test started"
3 date > /storage/test-batch.txt
4 sleep 2000
5
```

Close

Once you batch job is completed, you should be able to retrieve the model weights from Nextcloud if you had a saving step in your training script. Additionally, if your model is *integrated with MLflow* you should be able to view your

training stats in the MLflow UI.

7.4 Train using Federated Learning

The platform allows you to deploy servers to orchestrate Federated Learning trainings. We currently support the two main Federated Learning frameworks:

- Flower: due to its simpler configuration, this is the **recommended** option for beginners.
- NVFLARE: for people who want a more fine-grained control of the training using a Dashboard UI.

Clients connect to those server to perform training on their local data. Clients can be deployed either in the platform (as a standard [Development Environment](#)) or on external infrastructures. See the tutorials on each Federated Learning framework to see how to configure the clients:

- Flower tutorial
- NVFLARE tutorial

7.5 Labeling images with CVAT

Requirements

You need a *platform account* with *full access level*.

In this tutorial, we will guide you on how to use the [Computer Vision Annotation Tool \(CVAT\)](#) in the platform to annotate images.

7.5.1 Deploying CVAT

The CVAT tool is located at the top of the [Marketplace](#), in the **Tools** section.

The workflow for deploying CVAT is similar to the one for [deploying a module](#). In this particular case, you will need to pay attention to:

- **CVAT credentials:** When configuring the deployment of CVAT, you will need to enter your CVAT `username` and CVAT `password` to authenticate yourself in the CVAT instance.
- **Storage:** For using CVAT, it is *mandatory* to have a [storage provider linked](#). This is because, each time you delete a CVAT instance a snapshot will automatically be created in your storage. When you deploy a new CVAT instance, you can either start from an existing snapshot or from a blank state (no snapshot).

Storage configuration

Storage service
share.services.ai4os.eu

Advanced settings

Restore from snapshot

CVAT_test_2024-11-07-13-12-18-752450018

-

CVAT_test_2024-11-07-13-12-18-752450018

CVAT_test_2024-11-12-12-43-56-250536173

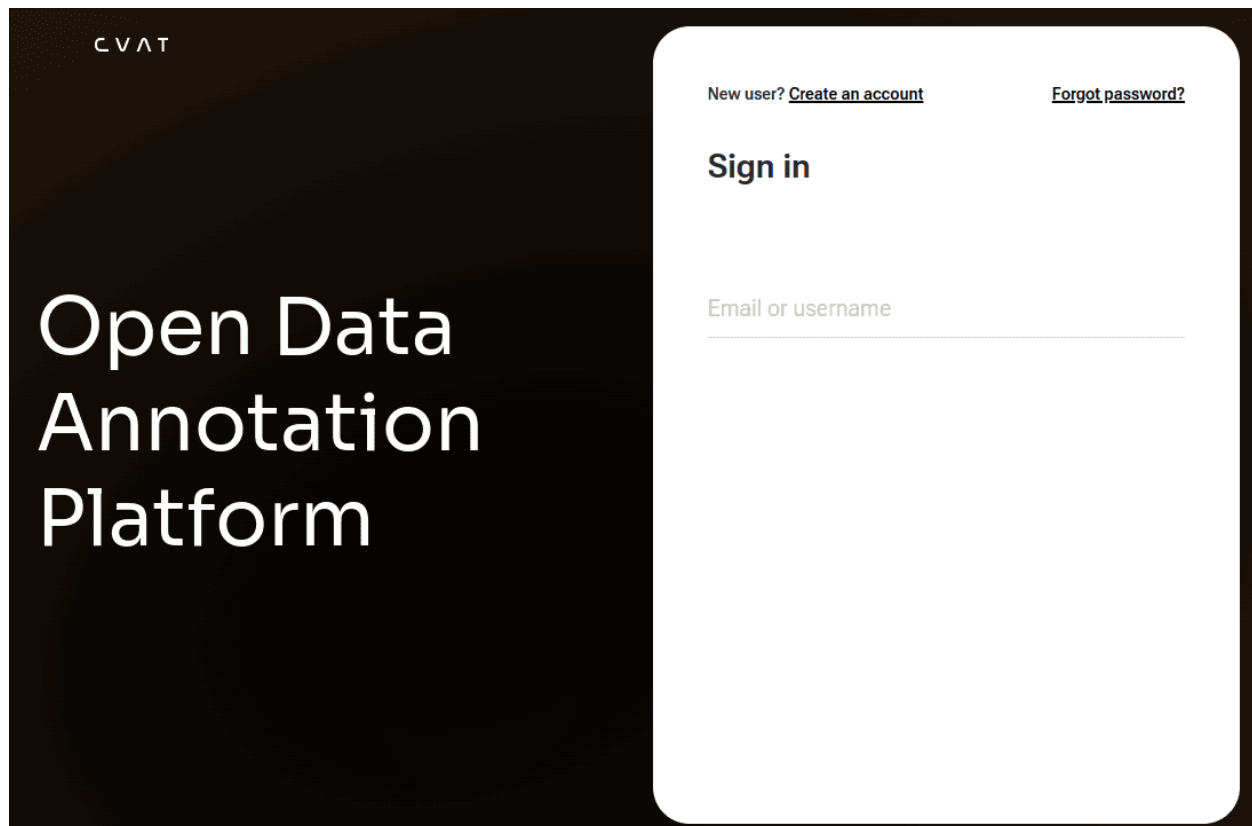
fuchsia_octopus_2024-11-12-12-47-16-316184003

Back

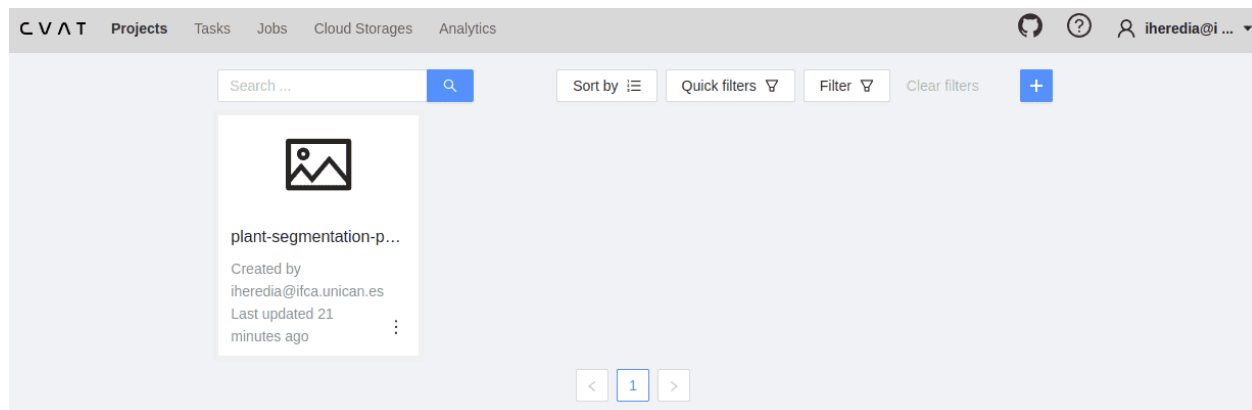
Submit

7.5.2 Using CVAT

In the *deployments list* you will be able to see your newly created CVAT instance. Clicking the Quick access button, you will directly enter the CVAT UI.



The enter you CVAT username and CVAT password and voilà, you're in!



CVAT uses a hierarchical structure to organize annotation work:

- **Projects** are used to organize multiple related tasks under a shared label schema and configuration. Projects are ideal for managing large annotation efforts that span multiple datasets or annotation tasks.
- **Tasks** are a specific annotation assignment within a project. Each task defines the label schema to use for annotation and contains the data to be annotated.
- **Jobs** are a subdivision of a task, containing a subset of the data. They allow for the distribution of annotation work among different annotators.

We recommend creating start by creating a project then create subsequent tasks. Creating a project is important because this will enable *periodic backups* to automatically be generated.

Connecting your dataset

When creating a new task you have to provide some data to annotate. You have the following options:

1. **My computer:** Upload data from your local computer
2. **Connected file share:** Use datasets you already have in the [Storage](#).
In this case, anything you save under `ai4os-storage/tools/cvat/share` will be available for annotation in CVAT.
3. **Remote sources:** Use publicly available images, by providing URLs.
For example, you can make public any folder in the [Storage](#) by clicking → **Open details** → **Share link**
4. **Cloud storage:** In case you have your dataset hosted on ASW S3, Azure or Google Cloud.

For more information on using CVAT, please follow the [official CVAT documentation](#).

7.5.3 CVAT automated backups

Annotating a dataset is a very time consuming task, so having automated backups is a must.

In the platform, we support multiple ways to backup your annotations. All those backup are saved in the [Storage](#) under `ai4os-storage/tools/cvat`.

To avoid collapsing your storage quota, we adapt the backup schedule to the backup size (ie. lighter backups are made more frequently). The performed backups are:

- **when a deployment is deleted by the user**, we save a full backup of the deployment. This not only includes project annotations, but also meta configurations (like user groups). Those are the snapshots that you will later be able to *select in the configuration form*.

Location: `ai4os-storage/tools/cvat/backups`

- **every day**, we save a full project backup with images and annotations. To restore from that backup, you will need to import it manually in the CVAT UI.

Location: `ai4os-storage/tools/cvat/backups-periodic`

- **every hour**, we save a project backup just with annotations. To restore from that backup, you will need to import it manually in the CVAT UI.

Location: `ai4os-storage/tools/cvat/backups-periodic`

DEPLOY A MODEL IN PRODUCTION

Once you have trained your model, you can deploy it in production. This section provides several guides in how to deploy your model in different environments.

Overview Start here to get an overview of the different deployment options.

Deployment options Deploy as serverless Deploy your model in the platform using the serverless option, using a shared serverless environment.

Deploy a model on the Inference platform using the Dashboard Deploy persistently Deploy your model in the platform using a dedicated deployment and a load balancer.

Deploy a model on dedicated resources using the Dashboard Deploy in your cloud Deploy your model in your cloud using the provided Docker image.

Deploy a model on your own cloud resources Deploy in the EOSC EU Node Deploy your model in the resources provided by the EOSC EU Node.

Deploy a model on the EOSC EU Node Deploy external models Deploy models from external marketplaces (BioImage Model Zoo).

Deploy an external model Deploy your own LLM Deploy your own LLM from a selection of open-source models (DeepSeek, Qwen, LLama, etc), using vLLM and Open-WebUI.

Deploy your own LLM chatbot

8.1 Deployment options

This page serves a guide, summarizing the pros and cons of each deployment option. With this information in mind, users can make the best decision on where to deploy their models.

Table 1: Deployment options from the Dashboard

Option	Pros	Cons
<i>Deploy as serverless</i> (model is loaded on demand)	- You are not consuming resources when you are not using the model, - Deployments can auto-scale to fit peaks in user queries, - Supports both sync and async calls - Supports predictions via default UI (Oscar) - Zero configuration needed as the model is deployed internally, - Can be manually customized for specific needs (like historical data batch prediction).	Predictions can have some latency, due to the AI model being loaded at each prediction.
<i>Deploy persistently</i> (model is always loaded)	- Low latency in predictions, - Supports sync calls - Supports predictions via dedicated UI (Gradio) - Zero configuration needed as the model is deployed internally.	You are consuming resources even when not actively making predictions.
<i>Deploy in your own cloud resources</i>	- You control where you deploy (no need to be a project member). - Supports sync calls	- More work to configure the deployment, - You are consuming resources even when not actively making predictions (if not deployed as serverless).
<i>Deploy in the EOSC node</i>	- You control where you deploy (no need to be a project member). Free limited resources for any European researcher. - Supports sync calls	- More work to configure the deployment, - You are consuming resources even when not actively making predictions (if not deployed as serverless).

Given the above specifications, we recommend the following typical workflows:

- *Deploy as serverless* if your service does not need low latency and expects to either (1) receive lots of concurrent user queries or (2) receive user queries that are spaced-out in time. This is the *recommended option* by default for all users.
- *Deploy persistently* if your service needs to handle, with low latency, few concurrent user queries that are close in time.
- *Deploy in your own cloud resources* if you do not belong to the project and have access to private cloud resources.
- *Deploy in the EOSC node* if you are a European researcher who does not belong to the project and don't have access to private cloud resources.

If you need to generate one-off predictions on a given dataset but not maintain a running service, you have two options:

- *Deploy persistently* with a GPU, make your predictions and delete the deployment.
- *Deploy as serverless* and upload your dataset files to a bucket to perform *async predictions*. If your dataset is really big, you can *contact support* to create a custom batch processing pipeline tailored to your usecase.

In addition to the above deployment options from the Dashboard, there are several additional deployment methods:

- Deploy a model manually in the Inference Platform (serverless)
- *Deploy a model locally*

8.2 Deploy a model on the Inference platform using the Dashboard

i Requirements

You need a *platform account* with *full access level*.

The Dashboard offers the possibility to deploy an AI module as serverless on the platform resources.

Pros	Cons
- You are not consuming resources when you are not using the model, - Deployments can auto-scale to fit peaks in user queries, - Zero configuration needed as the model is deployed internally.	Predictions can have some latency, due to the AI model being loaded at each prediction.

8.2.1 1. Choose your module and deploy

The first step is to choose a module from the *Dashboard*. Under the Platform Categories tag section, select any module that has the tag **AI4 Inference**.

In this tutorial, we will demonstrate a simple prediction using the popular **YOLO module**.

YoloV8 model

Try ▾

Deploy ▾

Codespaces ▾

Object detection using YoloV8 model

Ultralytics YOLOv8 represents the forefront of object detection (segmentation/classification) models incorporating advancements from prior YOLO iterations while introducing novel features to enhance performance and versatility.

YOLOv8 prioritizes speed, precision, and user-friendliness, positioning itself as an exceptional solution across diverse tasks such as object detection, oriented bounding boxes detection, tracking, instance segmentation, and image classification. Its refined architecture and innovations make it an ideal choice for cutting-edge applications in the field of computer vision.

NOTE: Among the training arguments, there are options related to augmentation, such as flipping, scaling, etc. The default values are set to automatically activate some of these options during training. If you want to disable augmentation entirely or partially, please review the default values and adjust them accordingly to deactivate the desired augmentations.

References

1. Jocher, G., Chaurasia, A., & Qiu, J. (2023). YOLO by Ultralytics (Version 8.0.0) [Computer software]. <https://github.com/ultralytics/ultralytics>
2. <https://docs.ultralytics.com>
3. Redmon, J., et al., You Only Look Once: Unified, Real-Time Object Detection, 2015, <https://arxiv.org/abs/1506.02640> [cs.CV]

build passing

Tasks

Computer Vision

Classification

Libraries

PyTorch

Data Types

Image

Video

Tags

deep learning

In the module page, click on the option **Deploy > Inference API (serverless)**. This will automatically deploy your OSCAR service with some predefined configuration.

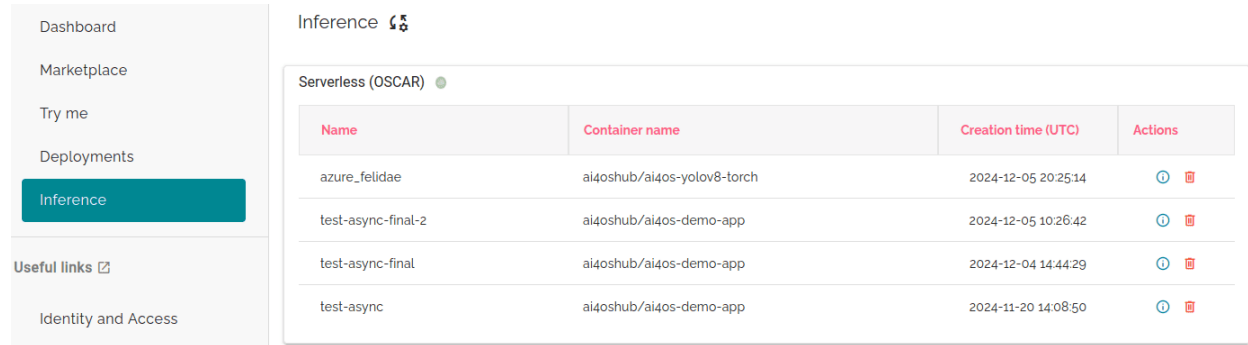
i Module compatibility

- Inference endpoints only work for models that return a JSON response (that is the case for most models).
- Modules require a DEEPaaS version higher than 2.5.0 to work with OSCAR inference endpoints (that is the case for most models). If the module does not have a supported DEEPaaS, it will return a clear error message when trying to make an inference.

8.2.2 2. Make a prediction

Retrieve the service information

In the Inference tab, go to the Serverless (OSCAR) table and find your newly created service.

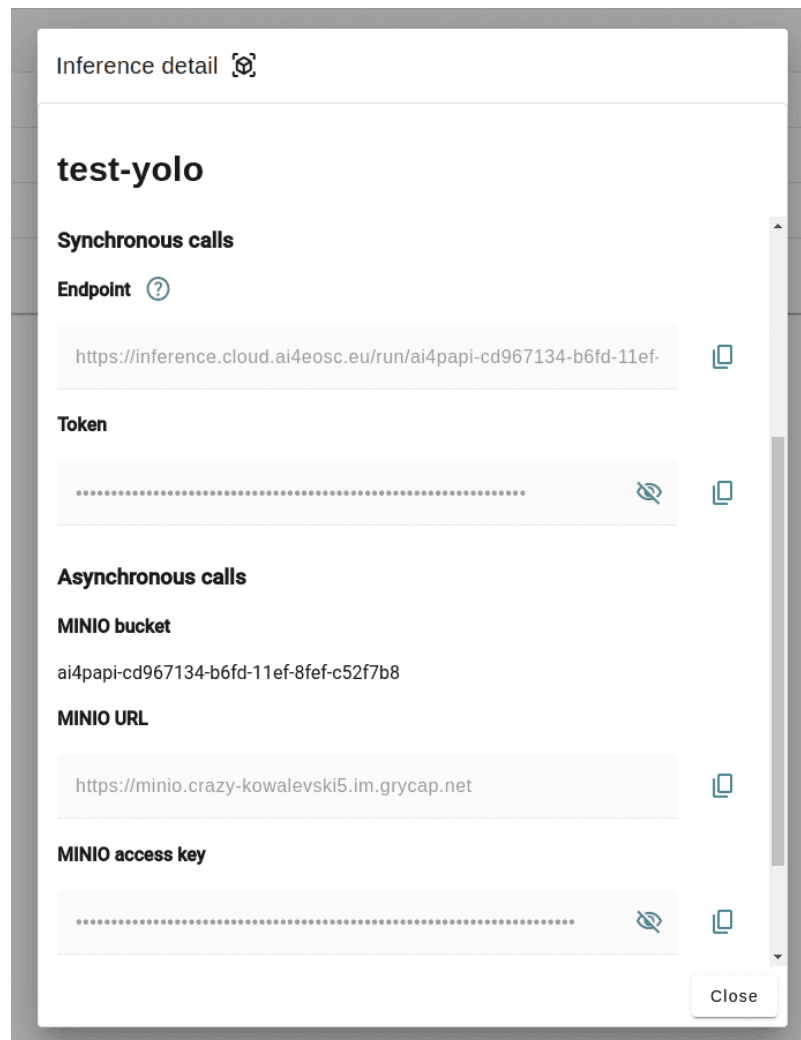


The screenshot shows the 'Inference' tab in the AI4OS interface. On the left is a sidebar with navigation links: Dashboard, Marketplace, Try me, Deployments, Inference (highlighted), Useful links, and Identity and Access. The main area displays the 'Inference' tab with a sub-header 'Serverless (OSCAR)' and a green status indicator. Below this is a table with the following data:

Name	Container name	Creation time (UTC)	Actions
azure_felidae	ai4oshub/ai4os-yolov8-torch	2024-12-05 20:25:14	 
test-async-final-2	ai4oshub/ai4os-demo-app	2024-12-05 10:26:42	 
test-async-final	ai4oshub/ai4os-demo-app	2024-12-04 14:44:29	 
test-async	ai4oshub/ai4os-demo-app	2024-11-20 14:08:50	 

Click on Info for your service and you will find all the variables you need for making a prediction:

- **for synchronous calls:** the endpoint you will query and the secret token you need for authentication.
- **for asynchronous calls:** all the variables related to the [MINIO storage](#) that handles your predictions.



The screenshot shows the 'Inference detail' modal for a service named 'test-yolo'. The modal is divided into two sections: 'Synchronous calls' and 'Asynchronous calls'.

Synchronous calls

- Endpoint** (with a help icon): `https://inference.cloud.ai4eosc.eu/run/ai4papi-cd967134-b6fd-11ef-` (with a copy icon)
- Token**: A masked secret token (with an eye icon to toggle visibility and a copy icon)

Asynchronous calls

- MINIO bucket**: `ai4papi-cd967134-b6fd-11ef-8fef-c52f7b8`
- MINIO URL**: `https://minio.crazy-kowalevski5.im.grycap.net` (with a copy icon)
- MINIO access key**: A masked secret token (with an eye icon to toggle visibility and a copy icon)

A 'Close' button is located at the bottom right of the modal.

Finding the model inputs

Before making a prediction with that service, you first need to know what are the required inputs the model needs.

The following Python script demonstrates how to make a *synchronous call* asking for help. To use the script, you have to replace the endpoint and token variables with the values *retrieved above*.

Help Python script

```
import base64

import requests

endpoint = "https://inference.cloud.ai4eosc.eu/run/ai4papi-*****"
token = "*****"

data = {"help": ""}
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {token}",
}
r = requests.post(url=endpoint, headers=headers, json=data)

if r.status_code == 401:
    raise Exception("Invalid token.")
if not r.ok:
    raise Exception(f"Some error has occurred: {r}")

print(r.text)
```

In this case, we receive what are the inputs needed by the YOLO model.

```
[...]
options:
-h, --help
    show this help message and exit
--files FILES
    Input an image or Video.
    accepted image formats: .bmo, .dng, .jpg, .jpeg, .mpo, .png, .tif, .tiff, .pfm,
→ and .webp.
    accepted video formats: .asf, .avi, .gif, .m4v, .mkv, .mov, .mp4, .mpeg, .mpg, .
→ ts, .wmv, .webm
    Type: str (filepath)
    *Required*
[...]
```

i Response status codes

If the call returns a 502 error, you should wait a bit more, your docker image is probably getting pulled!

If the error persists, please [contact support](#).

Synchronous predictions

Now that we know the inputs needed, it's time to make a synchronous prediction call on a [bear image](#) with the YOLO model service.

The following Python script demonstrates how to make a *synchronous call* with some model inputs. It is important to note that:

- All parameters for the model should be passed inside a JSON payload.
- If the model needs a file (as it is the case here), it should be passed encoded as base64 in the `oscar-files` field in the data.

As before, to use the script, you have to replace the `endpoint` and `token` variables with the values [retrieved above](#).

Synchronous prediction Python script

```
import base64

import requests

endpoint = "https://inference.cloud.ai4eosc.eu/run/ai4papi-*****"
token = "*****"

def get_base64(fpath):
    with open(fpath, "rb") as f:
        encoded_str = base64.b64encode(f.read()).decode("utf-8")
    return encoded_str

data = {
    "oscar-files": [
        {
            "key": "files",
            "file_format": "jpg",
            "data": get_base64("./bear.jpg"),
        },
    ],
}

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {token}",
}

r = requests.post(url=endpoint, headers=headers, json=data)

if r.status_code == 401:
    raise Exception("Invalid token.")
if not r.ok:
    raise Exception(f"Some error has occurred: {r}")
```

(continues on next page)

(continued from previous page)

```
print(r.text)
```

The script will print the logs, along with the JSON output of the model (in this case, the prediction made by YOLO).

```
[...]
2024-09-30 12:09:19.502 29 INFO deepaas.cmd.cli [-] return: '['\n {\n      "name": "bear",
↪ \n      "class": 21,\n      "confidence": 0.93346,\n      "box": {\n      "x1": 109.39322,\n↪
↪      "y1": 26.39718,\n      "x2": 627.42999,\n      "y2": 597.74689\n      }\n }\n']
[...]
```

Asynchronous predictions

It is also possible to make asynchronous calls to the same service. This kind of execution relies on the event-driven approach, where the OSCAR service reacts automatically to events that occur in the object-storage system so that the interaction is directly done uploading files to the buckets in the MinIO storage system.

This approach is especially beneficial when:

- You have lots of data. This is because you upload all the data to the Minio bucket, and we will process that data in parallel. When the predictions are completed you can retrieve them from the bucket.
- You need to automatize the processing of the predictions. This is because with this approach we save separately the logs (in a .log file) and the predictions (in a .json file). If the prediction was not successful, you will only retrieve the log file.

The following Python script demonstrates how to upload some inputs to the bucket, list the contents of the bucket and download the outputs when the predictions are ready.

To use the script, you have to replace the Minio-related variables with the values *retrieved above*.

Asynchronous prediction Python script

```
import base64
import json
import time

import boto3

# This information is retrieved from your deployment information window
MINIO_BUCKET = "ai4papi-*****"
MINIO_URL = "https://*****"
MINIO_ACCESS_KEY = "*****@egi.eu"
MINIO_SECRET_KEY = "*****"

# This is how you decide to name your new prediction
prediction_ID = "test-prediction"

# Local paths (in current folder)
pth_local_input = f"input-{prediction_ID}.json"
pth_local_output = f"output-{prediction_ID}.json"
pth_local_logs = f"output-{prediction_ID}.log"
```

(continues on next page)

(continued from previous page)

```

# Remote paths (in the bucket)
# Two files will be produced in the output folder of the bucket
# * <input_filename>.json: this file has the output of the prediction, in JSON format.
# --> this will only be created if the prediction is successful
# * <input_filename>.log: this file has the logs of the prediction.
# --> this will always be created
pth_remote_input = f"inputs/{prediction_ID}.json"
pth_remote_output = f"outputs/{prediction_ID}.json"
pth_remote_logs = f"outputs/{prediction_ID}.log"

# Prepare the data you want to predict
def get_base64(fpath):
    """
    Encode files as base64. We need to do this to pass files as prediction inputs in
    the JSON file.
    """
    with open(fpath, "rb") as f:
        encoded_str = base64.b64encode(f.read()).decode("utf-8")
    return encoded_str

data = {
    "oscar-files": [
        {
            "key": "files",
            "file_format": "jpg",
            "data": get_base64("./bear.jpg"),
        },
    ],
}

# Create the JSON file
with open(pth_local_input, "w") as f:
    json.dump(data, f)

# Init the Minio Object Store
client = boto3.client(
    "s3",
    endpoint_url=MINIO_URL,
    region_name="",
    verify=True,
    aws_access_key_id=MINIO_ACCESS_KEY,
    aws_secret_access_key=MINIO_SECRET_KEY,
)

# Check if input file already exists in bucket, if so delete it to make the prediction.
↪ again
try:
    client.head_object(Bucket=MINIO_BUCKET, Key=pth_remote_input)
    client.delete_object(Bucket=MINIO_BUCKET, Key=pth_remote_input)
    print(f"Deleted existing file {pth_remote_input} from bucket {MINIO_BUCKET}")
except client.exceptions.ClientError as e:
    if e.response["Error"]["Code"] == "404":

```

(continues on next page)

(continued from previous page)

```

        print(f"No existing file {pth_remote_input} found in bucket {MINIO_BUCKET}")
    else:
        raise

# Upload the inputs to the bucket
with open(pth_local_input, "rb") as data:
    client.upload_fileobj(data, MINIO_BUCKET, pth_remote_input)
print(f"Uploaded data to {pth_remote_input} in bucket {MINIO_BUCKET}")

# Now we wait until the prediction is made
while True:
    # List objects in the bucket
    r = client.list_objects_v2(Bucket=MINIO_BUCKET)
    contents = [i["Key"] for i in r["Contents"]]

    # If the output is available, download it
    if pth_remote_logs in contents:
        with open(pth_local_logs, "wb") as data:
            client.download_fileobj(MINIO_BUCKET, pth_remote_logs, data)
        print(f"Downloaded logs from {pth_remote_logs} in bucket {MINIO_BUCKET}")

        # Prediction JSON will only be available if the prediction was successful
        if pth_remote_output in contents:
            with open(pth_local_output, "wb") as data:
                client.download_fileobj(MINIO_BUCKET, pth_remote_output, data)
            print(f"Downloaded data from {pth_remote_output} in bucket {MINIO_BUCKET}")

            break

    else:
        print("Waiting for the prediction to complete ...")
        time.sleep(5)

```

This script will produce a `.log` file with the OSCAR logs and a `.json` file with the prediction of the YOLO model.

Using the OSCAR Web UI interface

Another option to interact with the OSCAR service is through the graphical web interface (UI):

- [AI4EOSC OSCAR UI](#)
- [iMagine OSCAR UI](#)

To access the OSCAR service through the UI, follow these steps:

1. In the Inference tab, go to the Serverless (OSCAR) table and find your newly created service.
2. Click on Info button on the right to show the service details and check the Deployment ID.
3. Go to the **OSCAR UI** in your browser, log in with your credentials and search for the service with the same name as the Deployment ID.
4. Click on the More Actions button of your service and select Invoke to open a new screen where you can provide the input to the service invocation. At this point, as you have seen in the asynchronous and synchronous calls, you have to take into account that most of the AI Module whose input is an image need to convert the input into a compatible JSON format. This implies converting the image to base64 and expressing the input in

a JSON file. To help with this input preparation, we provide you a short Python script to convert your file into a compatible JSON format:

Data preparation Python script

```
import base64
import json

def get_base64(fpath):
    """Encodes a file in Base64 format."""
    with open(fpath, "rb") as f:
        encoded_str = base64.b64encode(f.read()).decode("utf-8")
    return encoded_str

# Prepare the JSON payload
data = {
    "oscar-files": [
        {
            "key": "files",
            "file_format": "png",
            "data": get_base64("./inputs_Cat.png"),
        },
    ],
}

# Save the JSON data to a file
with open("input2.json", "w") as f:
    json.dump(data, f, indent=4)
```

5. Once your file is ready, you can use it to invoke the service using the Run button.

8.2.3 3. More info

Make a prediction using Bash

For completeness sake, we provide the equivalent commands to perform the above operations in Bash, instead of Python.

Synchronous call with YOLO (Bash)

Find the input parameters needed by the model:

```
$ curl --location '***endpoint***' \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer ***token***' \
--data '{"help": ""}'
```

Make a synchronous call with an image input:

```
# Create a JSON payload with the base64 data and save it to a temporary file
JSON_PAYLOAD=$(cat <<EOF
{
"oscar-files": [
    {
```

(continues on next page)

(continued from previous page)

```

    "key": "files",
    "file_format": "jpg",
    "data": "$(base64 /home/iheredia/bear.jpg | tr -d "\n")"
  }
]
}
EOF
)

# Save the JSON payload to a temporary file
TEMP_JSON_FILE=$(mktemp)
echo "$JSON_PAYLOAD" > "$TEMP_JSON_FILE"

# Step 3: Use curl to send the request with the JSON payload from the temporary file
curl --location ***endpoint*** \
--header 'Content-Type: application/json' \
--header 'Authorization: Bearer ***token***' \
--data @"$TEMP_JSON_FILE"

# Clean up the temporary file
rm "$TEMP_JSON_FILE"

```

Learn how to feed different input types

We are going to demonstrate how to send a more complete set of input parameters to OSCAR.

For educational purposes, we are going to use the [official demo module](#). While this model does not perform an AI task, it is very helpful as it shows the wide variety of inputs that can be sent to OSCAR inference endpoints.

So go back to the previous steps and deploy the [ai4os-demo-app](#). Once you have retrieved your endpoint and token, you can run the following Python script to make the prediction from your local computer:

Synchronous call with the demo app (Python)

```

import ast
import base64

import requests

token = '*****'
url = '*****'

headers = {
    'Content-Type': 'application/json',
    'Authorization': f'Bearer {token}',
}

def get_base64(fpath):
    with open(fpath, "rb") as f:
        encoded_str = base64.b64encode(f.read()).decode('utf-8')
    return encoded_str

```

(continues on next page)

(continued from previous page)

```

data = {
    'demo_str': 'hi there!!!!',
    'demo_str_choice': 'choice1',
    'demo_int': -3,
    'demo_int_range': 42,
    'demo_float': -0.9,
    'demo_bool': False,
    'demo_dict': '{"c": "d"}',
    'demo_list_of_floats': "[1.2, -1.8]",
    'oscar-files': [
        {
            'key': 'demo_image',
            'file_format': 'png',
            'data': get_base64('./sample-image.png'),
        },
        {
            'key': 'demo_audio',
            'file_format': 'wav',
            'data': get_base64('./sample-audio.wav'),
        },
        {
            'key': 'demo_video',
            'file_format': 'mp4',
            'data': get_base64('./sample-video.mp4'),
        },
    ],
}
# data = {'help': ""}

r = requests.post(url=url, headers=headers, json=data)
out = r.text

if r.status_code == 401:
    raise Exception('Invalid token.')

print(out)

```

Manually deploying in OSCAR UI

Do you want to manually deploy your OSCAR services for greater customization? Check how to Manually deploy a serverless inference endpoint.

For example, you can use manual serverless deployments for example a large number of files (eg. historical data) all at once (**OSCAR batch mode**). If you need help setting this usecase, please [contact support](#).

8.3 Deploy a model on dedicated resources using the Dashboard

Requirements

You need a *platform account* with *full access level*.

The Dashboard offers the possibility to deploy an AI module persistently on the platform resources.

Pros	Cons
• Low latency in predictions, • Zero configuration needed as the model is deployed internally.	• You are consuming resources even when not actively making predictions.

8.3.1 1. Choose your module and deploy

The first step is to choose a module from the *Dashboard*. There you will be able to find all the modules ready to be used under the tag *Inference*.

We will first start doing a simple prediction using the popular *YOLO module*.

YoloV8 model

Object detection using YoloV8 model

Try Deploy Codespaces

Ultralytics YOLOv8 represents the forefront of object detection (segmentation/classification) models incorporating advancements from prior YOLO iterations while introducing novel features to enhance performance and versatility.

YOLOv8 prioritizes speed, precision, and user-friendliness, positioning itself as an exceptional solution across diverse tasks such as object detection, oriented bounding boxes detection, tracking, instance segmentation, and image classification. Its refined architecture and innovations make it an ideal choice for cutting-edge applications in the field of computer vision.

NOTE: Among the training arguments, there are options related to augmentation, such as flipping, scaling, etc. The default values are set to automatically activate some of these options during training. If you want to disable augmentation entirely or partially, please review the default values and adjust them accordingly to deactivate the desired augmentations.

References

1. Jocher, G., Chaurasia, A., & Qiu, J. (2023). YOLO by Ultralytics (Version 8.0.0) [Computer software]. <https://github.com/ultralytics/ultralytics>
2. <https://docs.ultralytics.com>
3. Redmon, J., et al., You Only Look Once: Unified, Real-Time Object Detection, 2015, <https://arxiv.org/abs/1506.02640> [cs.CV]

build passing

Tasks

Computer Vision Classification

Libraries

PyTorch

Data Types

Image Video

Tags

deep learning

In the module page, click on the option *Deploy > Inference API (dedicated)*. You will be shown a *configuration page* where the option *DEEPaaS* is selected. You can directly click on *Quick submit* as you don't need to configure anything else.

8.3.2 2. Making a prediction

In the *Deployments* tab, go to the *Modules* table and find your created deployment. This deployment offers the ability to perform predictions either via an API or an UI:

2.1 API prediction

Click the *Quick access* to access the *DEEPaaS* UI.

models		
GET	/v2/models/	Return loaded models and its information
GET	/v2/models/imgclas/	Return model's metadata
POST	/v2/models/imgclas/train/	Retrain model with available data
GET	/v2/models/imgclas/train/	Get a list of trainings (running or completed)
GET	/v2/models/imgclas/train/{uuid}	Get status of a training
DELETE	/v2/models/imgclas/train/{uuid}	Cancel a running training
POST	/v2/models/imgclas/predict/	Make a prediction given the input data

In the POST .../predict method:

- click in the right button Try it out,
- upload a image (eg. a [bear image](#)) to the files parameter,
- click in Execute.

You will be returned a JSON with the results:

```
{
  "status": "OK",
  "predictions": [
    "[\n  {\n    \"name\": \"bear\",\n    \"class\": 21,\n    \"confidence\": 0.93346,\n    \"box\": {\n      \"x1\": 109.39322,\n      \"y1\": 26.39718,\n      \"x2\": 627.42999,\n      \"y2\": 597.74689\n    }\n  }]\n]"
  ]
}
```


Notes

Sometimes the first prediction you make can be a little slow. This is because the AI model has to be download and/or loaded. Subsequent predictions should run much faster.

2.1 UI prediction

Under Info you will also be able to find the UI endpoint. This endpoint offers a fully fledged Gradio UI to perform inference.

imgclas



data

Select the image you want to classify.

urls

Select an URL of the image you want to classify.

timestamp

Model timestamp to use for prediction.

"default_imagenet"

ckpt_name

Checkpoint inside the timestamp to use for prediction.

"final_model.h5"

Clear Submit

message

status

OK

classification scores

brown_bear

brown_bear	97%
koala	1%
American_black_bear	1%
ibex	0%
cougar	0%

Flag

8.4 Deploy a model on your own cloud resources

i Requirements

You need a *platform account* with *basic access level*.

As every module available in the Marketplace is open-source, we offer the possibility to automatically deploy it in your custom cloud resources using the [EGI Infrastructure Manager](#). This feature is available to any registered user ([Access Policies](#)).

Pros	Cons
• You control where you deploy (no need to be a project member).	• More work to configure the deployment, • You are consuming resources even when not actively making predictions (if not deployed as serverless).

For this, *select a module in the Marketplace* and click on Deploy IM.

Train an image classifier

Try ▾

Deploy ▾

Codespaces ▾

Batch

Train your own image classifier with your custom dataset. It comes also pretrained on the 1K ImageNet classes.

The deep learning revolution has brought significant advances in a number of fields [1], primarily linked to image and speech recognition. The standardization of image classification tasks like the [ImageNet Large Scale Visual Recognition Challenge](#) [2] has resulted in a reliable way to compare top performing architectures.

This Docker container contains the tools to train an image classifier on your personal dataset. It is a highly customizable tool that let's you choose between tens of different [top performing architectures](#) and training parameters.

The container also comes with a pretrained general-purpose image classifier trained on ImageNet. The PREDICT method expects an RGB image as input (or the url of an RGB image) and will return a JSON with the top 5 predictions.



References

1. Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. [Deep learning](#). Nature, 521(7553):436-444, May 2015.
2. Olga Russakovsky et al. [ImageNet Large Scale Visual Recognition Challenge](#). International Journal of Computer Vision (IJCV), 115(3):211-252, 2015.

Apache-2.0 build passing

Tasks

Computer Vision

Classification

Libraries

TensorFlow

Keras

Data Types

Image

Tags

deep learning

image classification

vo.imagine-ai.eu

general purpose

Platform Categories

AI4 trainable

AI4 pre trained

AI4 inference

Provenance

View

Download RDF

Additional Resources

GitHub

Report issue

Dataset

Docker

Weights

99 Citation

Dates

2024-04-25

2025-05-22

This will forward you to the Infrastructure Manager, with the module already selected.

Deploy a VM + AI4EOSC DEEPaaS

Description: Deploy a compute node getting the IP and SSH credentials to access via ssh with an extra HD disk. TOSCA template for deploying a DEEPaaS AI model (combine it with Dynamic DNS to set a DNS name)

Infrastructure Name
YOLO model

VM Data
GPU Data
AI4EOSC
Cloud Provider

Docker image of the model to be deployed
ai4oshub/ai4os-yolov8-torch

Enable GPU support (install nvidia drivers and nvidia-docker)
False

Submit
Cancel

If you are not logged in, you will be prompted to log in with your EGI Check-in account.

If you do not have any cloud provider configured, you will be prompted to configure them. You can do so by clicking on the Add button.

VM Data
GPU Data
AI4EOSC
Cloud Provider

Cloud Provider:

Select Cloud Provider:

No Cloud Provider defined. Add it in the Cloud Credentials page:

Add

You can then select any of the cloud providers supported by the IM, and fill in the required information (e.g. Access and Secret Key in case of AWS).

Cloud Credentials

Vault Info
+ New Credential

ID	Type	Info
		amazon EC2
		Google Cloud
		Microsoft Azure
		EGI
		Open Nebula
		openstack.
		Orange
		linode
		kubernetes
		Infrastructure Manager
		OSCAR
		CLOUD & HEAT

Credential

amazon
EC2

Unique ID in your credentials (free-form e.g. SITE_NAME-VO_NAME).

ID

Access Key

Secret Key

Cancel
Add

Finally, you can select the cloud provider you want to use, and the base image you want to use for the deployment.

Deploy a VM + AI4EOSC DEEPaaS

Description: Deploy a compute node getting the IP and SSH credentials to access via ssh with an extra HD disk. TOSCA template for deploying a DEEPaaS AI model (combine it with Dynamic DNS to set a DNS name)

Infrastructure Name
YOLO model

VM Data GPU Data AI4EOSC Cloud Provider

Cloud Provider:
Select Cloud Provider:
AWS
Set image ID:
us-east-1/ami-054d6a336762e438e

Submit Cancel

After clicking on Submit, the deployment will start. Then after a few minutes, the VM will be deployed and the AI4EOSC module configured and you will be able to use it.

My Infrastructures Token Refresh New deployment

10 entries per page Search: yolo

Name	Infrastructure uuid	Cloud Type	Cloud Info	Status	VMs	Actions
Yolo test	2e3a1895-71e4-11f0-a8ec-63fdae15520d		Site: ISAS-FedCloud VO: vo.ai4eos.eu	pending		Log

Showing 1 to 1 of 1 entry (filtered from 10 total entries)

When the status changes to Configured, you can click on Outputs button to see the endpoint of DEEPaaS API UI of the module.

Outputs

Field Name	Value
ai4eos_model_endpoint	https://147.213.76.236/ui
node_creds	Username: cloudadm Download Copy to clipboard
node_ip	147.213.76.236

Close

Click on the link and you will see the API documentation, where you can test the module's functionality, as well as perform other actions.

Swagger [Explore](#)

DEEP as a Service API endpoint ^{2.5.2}

[/swagger.json](#)

Hybrid DataCloud

This is a REST API that is focused on providing access to machine learning models. By using the DEEPaaS API users can easily run a REST API in front of their model, thus accessing its functionality via HTTP calls.

Currently you are browsing the [Swagger UI](#) for this API, a tool that allows you to visualize and interact with the API and the underlying model.

- [Project website](#)
- [Project documentation](#)
- [Model marketplace](#)

[API documentation](#)

versions

[GET /v2/](#) Get V2 API version information

8.5 Deploy a model on the EOSC EU Node

Requirements

- This tutorial requires access to the EOSC EU Node and the usage of available credits in your account.

This tutorial summarizes how to deploy a pre-trained AI model from the AI4EOSC Marketplace in the [EOSC EU Node](#), to be used for inference via the [DEEPaaS API](#).

For this, we will use the **Tools Hub** functionality of the EOSC EU Node, which allows us to deploy customized virtual infrastructures via TOSCA Templates on the available Cloud resources. The Tools Hub currently supports two deployment options:

Option	Pros	Cons
<i>Container</i> (OKD-based)	• Fast deployment times, • Endpoint exposed in a DNS name.	• No access to the execution environment (managed platform). • Does not support deployments with GPUs
<i>Virtual Machine</i> (Openstack-based)	• Full access to the execution environment (via SSH login). • Supports deployments with GPUs	• Longer deployment time, • Endpoint exposed in an IP, without a DNS name.

Based on your requirements, you can select the one who best fits your usecase.

8.5.1 Option 1: Deploy in a Container

Deploy in AI module in the EOSC EU Node (containers)

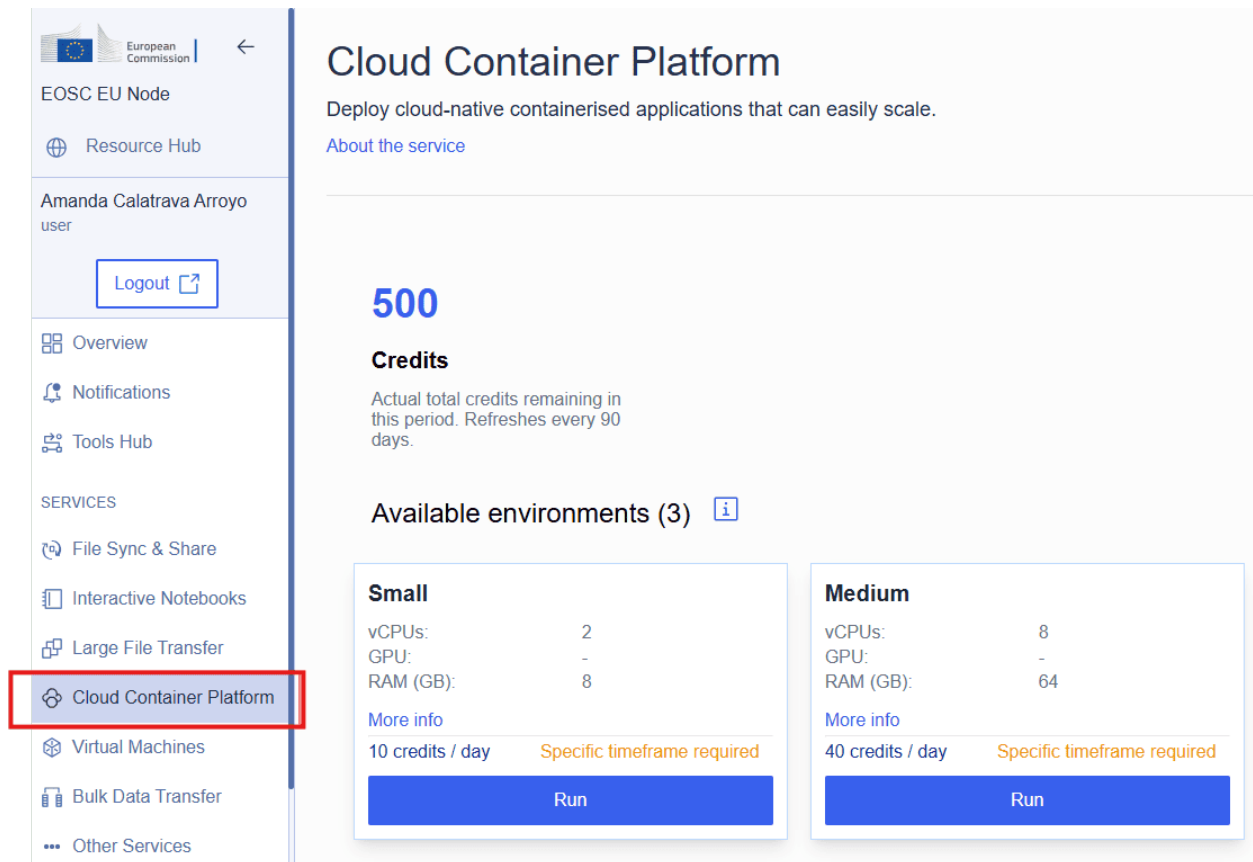
Please, be aware that video demos can become quickly outdated. In case of doubt, always refer to the written documentation.

1.1. Allocate the computing resources

You need to create your “*Default Personal Project*”. For this:

- Login into the [EOSC EU Node](#),
- In the Cloud Container Platform section (left panel), chose the **Small** environment by clicking Run. OKD reserves the corresponding quota for your user,
- Choose the time period and press Submit.

Notice that in this step what you do is to allocate the resources (quota), but you are not actually deploying them.



The screenshot shows the 'Cloud Container Platform' page in the EOSC EU Node. The left sidebar contains the following items:

- EOSC EU Node
- Resource Hub
- Amanda Calatrava Arroyo user
- Logout
- Overview
- Notifications
- Tools Hub
- SERVICES
- File Sync & Share
- Interactive Notebooks
- Large File Transfer
- Cloud Container Platform** (highlighted with a red box)
- Virtual Machines
- Bulk Data Transfer
- Other Services

The main content area displays the following information:

Cloud Container Platform

Deploy cloud-native containerised applications that can easily scale.

[About the service](#)

500 Credits

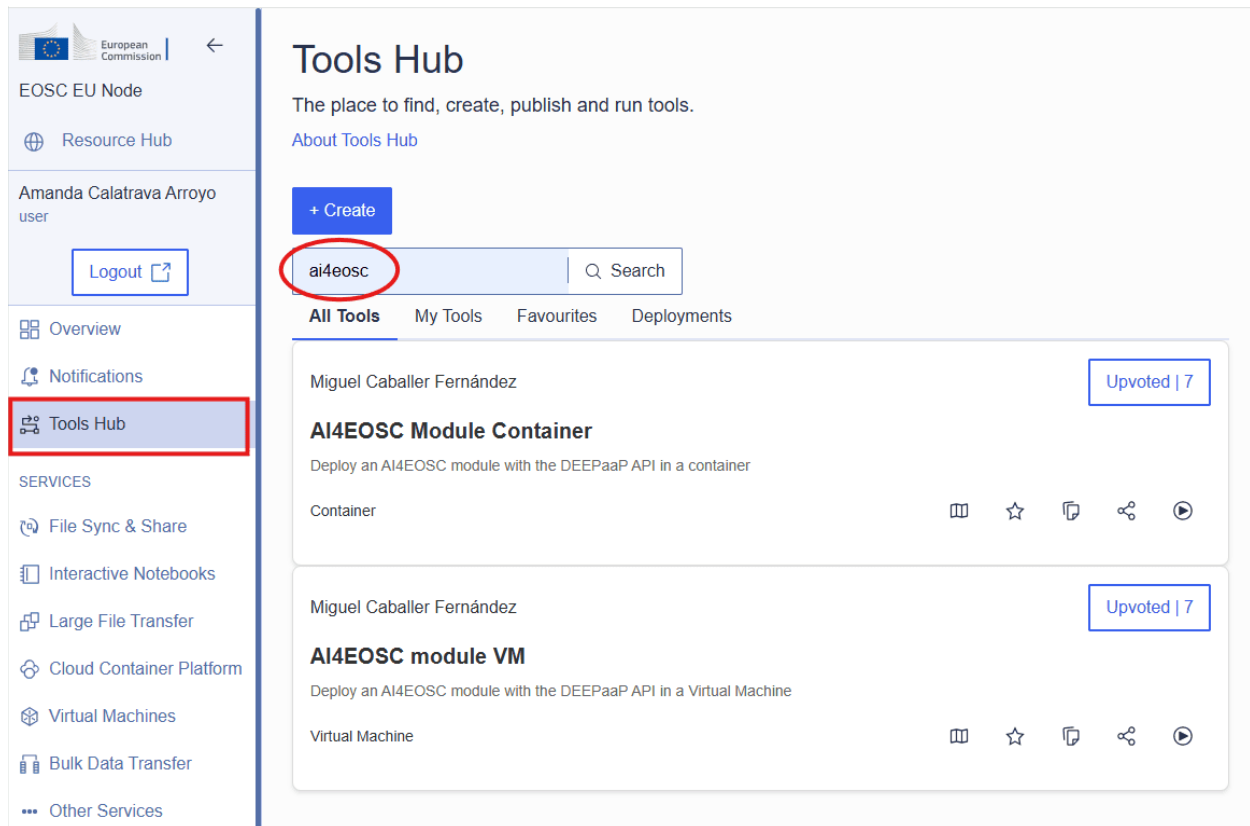
Actual total credits remaining in this period. Refreshes every 90 days.

Available environments (3) [i](#)

Small	Medium
vCPUs: 2	vCPUs: 8
GPU: -	GPU: -
RAM (GB): 8	RAM (GB): 64
More info	More info
10 credits / day Specific timeframe required	40 credits / day Specific timeframe required
Run	Run

1.2. Deploy your tool

In the **Tools Hub** section (left panel), search for **AI4EOSC** and select the **AI4EOSC Module Containers** (uploaded by platform developer *Miguel Caballer*).



The screenshot displays the AI4OS Tools Hub interface. On the left sidebar, the 'Tools Hub' menu item is highlighted with a red box. The main content area features a search bar with 'ai4eosc' entered, which is also circled in red. Below the search bar, there are two tool cards. The first card is for 'AI4EOSC Module Container' by Miguel Caballer Fernández, with an 'Upvoted | 7' button and a 'Show details' button (magnifying glass icon). The second card is for 'AI4EOSC module VM' by Miguel Caballer Fernández, also with an 'Upvoted | 7' button and a 'Show details' button. The sidebar includes the European Commission logo, 'EOSC EU Node', 'Resource Hub', user information for Amanda Calatrava Arroyo, a 'Logout' button, and a list of services including File Sync & Share, Interactive Notebooks, Large File Transfer, Cloud Container Platform, Virtual Machines, Bulk Data Transfer, and Other Services.

Clicking the Show details button, you will be able to see additional information about the tool.

The screenshot shows the 'AI4EOSC Module Container' page in the EOSC EU Node interface. The left sidebar contains navigation links for Overview, Notifications, Tools Hub, SERVICES (File Sync & Share, Interactive Notebooks, Large File Transfer, Cloud Container Platform, Virtual Machines, Bulk Data Transfer, Other Services), GENERAL (Groups, Orders, Credits, Favourites, Settings), and QUICKLINKS (Helpdesk, Resource Hub, Monitoring, Training Platform). The main content area displays the module details:

Information	
Name	AI4EOSC Module Container
Author	Miguel Caballer Fernández
Description	Deploy an AI4EOSC module with the DEEPaaS API in a container
Resource Organisation	AI4EOSC
Keywords	AI4EOSC, Deep Learning
License	Apache License 2.0
Version Date	2024-11-07
Deprecated	No
Scientific Domain	Engineering & Technology
Scientific Subdomain	Electrical, Electronic & Information Engineering
Target Infrastructure	Container

```

1  tosca_definitions_version: tosca_simple_yaml_1_0
2
3  imports:
4    - grycap_custom_types: https://raw.githubusercontent.com/grycap/tosca/main/custom_types.yaml
5
6  description: Deploy an AI4EOSC module with the DEEPaaS API in a container.
7
8  metadata:
9    template_name: AI4EOSC Module
10   template_version: "1.0.0"
11
12  topology_template:

```

Deploy it using the Play button.

You can set custom input values by selecting `customize input values` and `save to new tool` checkbox. This allows to configure the deployment parameters, in particular:

- the docker **model image** (available in the **Docker** value in the *Marketplace module details*)
- the amount of **resources** (CPU and RAM)
- the **endpoint** where the model will be exposed

Deploy Tool

☒ customize input values and save to new tool

new tool name

AI4EOSC Module Container_COPY

cpu

2

gpu

0

memory

4 GB

ai4eosc_model_image

ai4oshub/ai4os-yolov8-torch

endpoint

https://ai4eosc-module

Save and Select Project

Then click on **Save and Select Project**. This will create a new tool in your private **My Tools** tab, that you can use for new deployments. Next, you need to select the “*Default Personal Project*” *created in Step 1*, which is linked to the allocated OKD project, and click on the **Proceed** button.



Deploy Tool

Select a Project to deploy this Tool

Default Personal Project



CPU Cores:

2 Available / 2 Required

Memory:

8 GB Available / 3.7255859375 GB Required

Storage:

200 GB Available / 0 GB Required

Proceed

1.3. Access the tool

Once ready, the new deployment will available in the **Deployments** tab of the **Tools Hub** section.

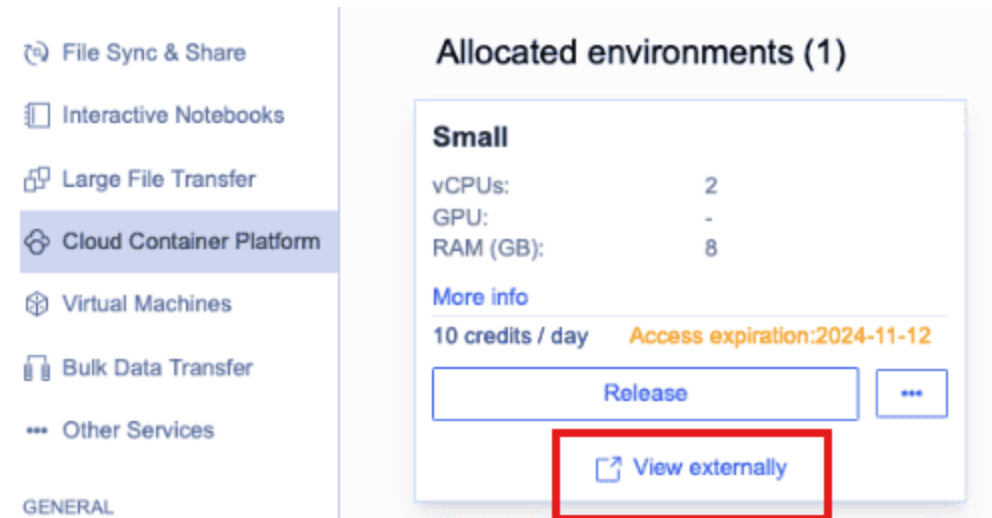
The screenshot shows the 'Tools Hub' interface. On the left is a sidebar with navigation links: 'EOSC EU Node', 'Resource Hub', 'Miguel Caballer Fernández user', 'Logout', 'Overview', 'Notifications', 'Tools Hub', and 'SERVICES' (File Sync & Share, Interactive Notebooks, Large File Transfer, Cloud Container Platform). The main area is titled 'Tools Hub' with the subtitle 'The place to find, create, publish and run tools.' and a '+ Create' button. Below this is a search bar with 'ai4' entered. A tab bar shows 'All Tools', 'My Tools', 'Favourites', and 'Deployments' (selected). A red box highlights a deployment card for 'gp-0193682b-f8c0-4382-bc90-c4d4b42dfac0' with status 'Status: i'. Below this is another card for 'AI4EOSC Module Container' by 'Miguel Caballer Fernández', with description 'Deploy an AI4EOSC module with the DEEPaaS API in a container', ID 'gp-0196c97f-fbc0-4ad0-8d6f-4ef1442784b5', and status 'Status: Deployed i'.

Clicking the Show details button, you will find the deployment endpoint (similar to <https://yolo-ai4eos-9ina.eu-1.open-science-cloud-user-apps.eu/ui>). In the endpoint, you will find the *DEEPaaS API* UI, which you can use to run inference calls on the model.

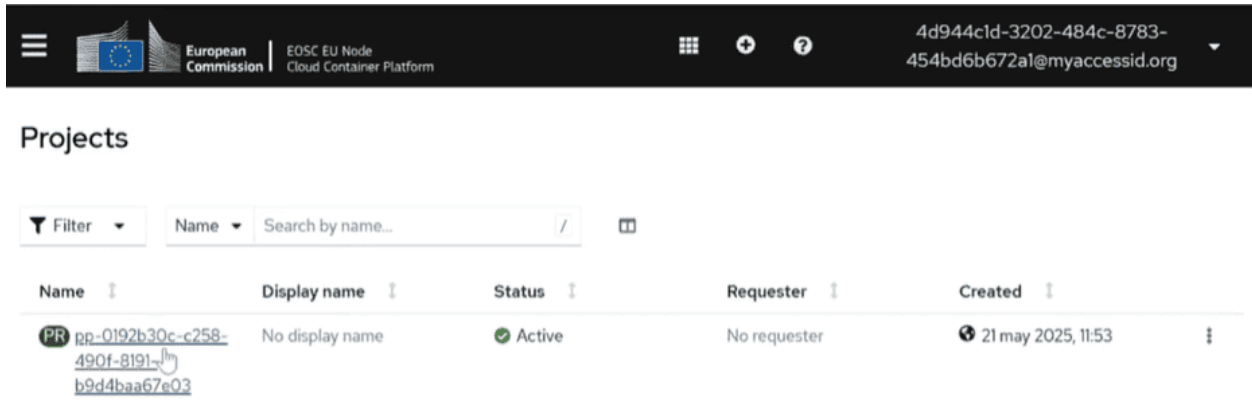
models	
GET	/v2/models/ Return loaded models and its information
GET	/v2/models/imgclas/ Return model's metadata
POST	/v2/models/imgclas/train/ Retrain model with available data
GET	/v2/models/imgclas/train/ Get a list of trainings (running or completed)
GET	/v2/models/imgclas/train/{uuid} Get status of a training
DELETE	/v2/models/imgclas/train/{uuid} Cancel a running training
POST	/v2/models/imgclas/predict/ Make a prediction given the input data

1.4. Managing the resources

Optionally, you can get additional information about your container. For that, go to the Cloud Container Platform section in the EOSC EU Node dashboard, select your allocated environment and click on View externally.



By clicking on this option, you will have access to your allocated environment in the EOSC EU Node Container provider, based in OKD.



Finally, once you have finished using the AI model, you can delete the deployment and release the resources. For that, go to the Deployments section in the Tools Hub section and remove it by clicking on the Trash button.

If you no longer want to deploy additional containers, you should release the allocated project, to avoid using credits. For that, go back again to the Cloud Container Platform section and release your allocated environment by clicking on the Release button. A notification will be sent by the system once the resources have been released.

8.5.2 Option 2: Deploy in a Virtual Machine

2.1. Allocate the computing resources

You need to create your “Default Personal Project”. For this:

- Login into the [EOSC EU Node](#),
- In the **Virtual Machines** section (left panel), chose the **Small** environment. OpenStack creates the corresponding project with the selected quota,
- Click **Run**, choose the time period and press **Submit**.

Notice that in this step what you do is to allocate the resources (quota), but you are not actually deploying them.

European Commission

EOSC EU Node

Resource Hub

Amanda Calatrava Arroyo user

Logout

Overview

Notifications

Tools Hub

SERVICES

File Sync & Share

Interactive Notebooks

Large File Transfer

Cloud Container Platform

Virtual Machines

Bulk Data Transfer

Other Services

Virtual Machines

Design and conduct experiments with flexibility while ensuring reproducibility.

[About the service](#)

500 Credits

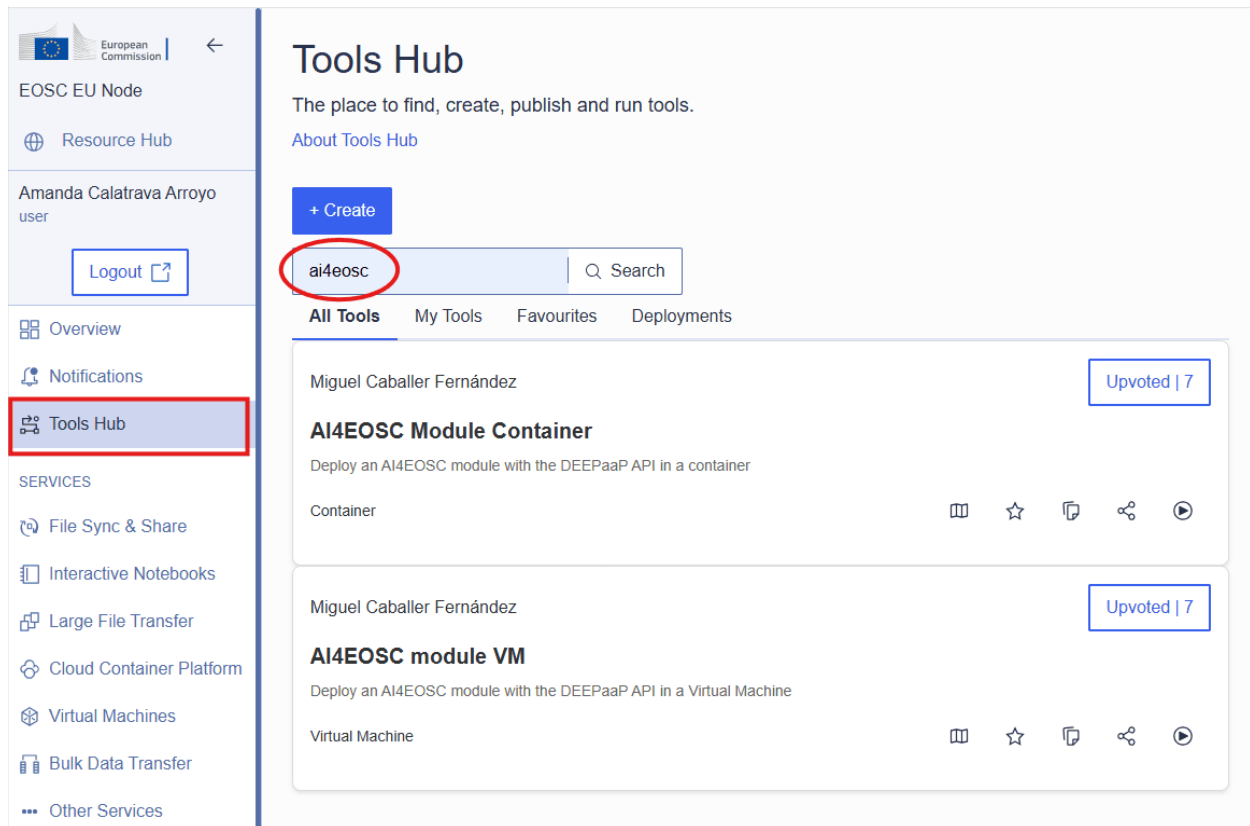
Actual total credits remaining in this period. Refreshes every 90 days.

Available environments (3) [i](#)

Small	Medium
vCPUs: 2	vCPUs: 8
GPU: -	GPU: -
RAM (GB): 8	RAM (GB): 64
More info	More info
10 credits / day Specific timeframe required	40 credits / day Specific timeframe required
Run	Run

2.2. Deploy your tool

In the **Tools Hub** section (left panel), search for **AI4EOSC** and select the **AI4EOSC Module VM** (uploaded by platform developer *Miguel Caballer*).



Deploy it using the Play button.

You can set custom input values by selecting `customize input values` and `save to new tool` checkbox. This allows to configure the deployment parameters, in particular:

- the docker **model image** (available in the **Docker** value in the *Marketplace module details*)
- the amount of **resources** (CPU and RAM)
- the **GPU** support, if needed

Then click on **Save and Select Project**. This will create a new tool in your private **My Tools** list, that you can use for new deployments. Next, you need to select the “*Default Personal Project*” *created in Step 1*, which is linked to the allocated OpenStack project.

Press the **Proceed** blue button and you will get a confirmation message about the deployment.

2.3. Access the tool

The new deployment is available in the **Deployments** tab of the **Tools Hub** section.

Clicking the **Show details** button, you will find the deployment endpoint (similar to `https://62.3.174.94/ui`)

Tool Output



```
{
  "outputs": {
    "ai4eos_model_endpoint": "https://62.3.174.94/ui"
  }
}
```

OK

Warning

Please note that, in the case of Virtual Machines, it will take up to 10 minutes to deploy and configure the VM for the endpoint to be ready. You can periodically try to connect until you'll eventually have access to the Swagger DEEPaaS UI.

In the endpoint, you will find the *DEEPaaS API* UI, which you can use to run inference calls on the model.

models		
GET	/v2/models/	Return loaded models and its information
GET	/v2/models/imgclas/	Return model's metadata
POST	/v2/models/imgclas/train/	Retrain model with available data
GET	/v2/models/imgclas/train/	Get a list of trainings (running or completed)
GET	/v2/models/imgclas/train/{uuid}	Get status of a training
DELETE	/v2/models/imgclas/train/{uuid}	Cancel a running training
POST	/v2/models/imgclas/predict/	Make a prediction given the input data

2.4. Managing the resources

Optionally, you might want to get additional information about your VM. For that, go to the Virtual Machines section in the EOSC EU Node dashboard, select your allocated environment and click on View externally.

Virtual Machines

Design and conduct experiments with flexibility while ensuring reproducibility.

[About the service](#)

440

Credits

Actual total credits remaining in this period. Refreshes every 90 days.

Allocated environments (1)

Small

vCPUs: 2
GPU: -
RAM (GB): 8

[More info](#)

10 credits / day Access expiration: 2024-11-11

[Release](#) [View externally](#)

By clicking on this option, you will have access to your allocated environment in the OpenStack cloud. Go to the Instances section to see your VM up and running:

Instances

Instance ID: Filter [Launch Instance \(Quota exceeded\)](#) [Delete Instances](#) [More Actions](#)

Displaying 1 item

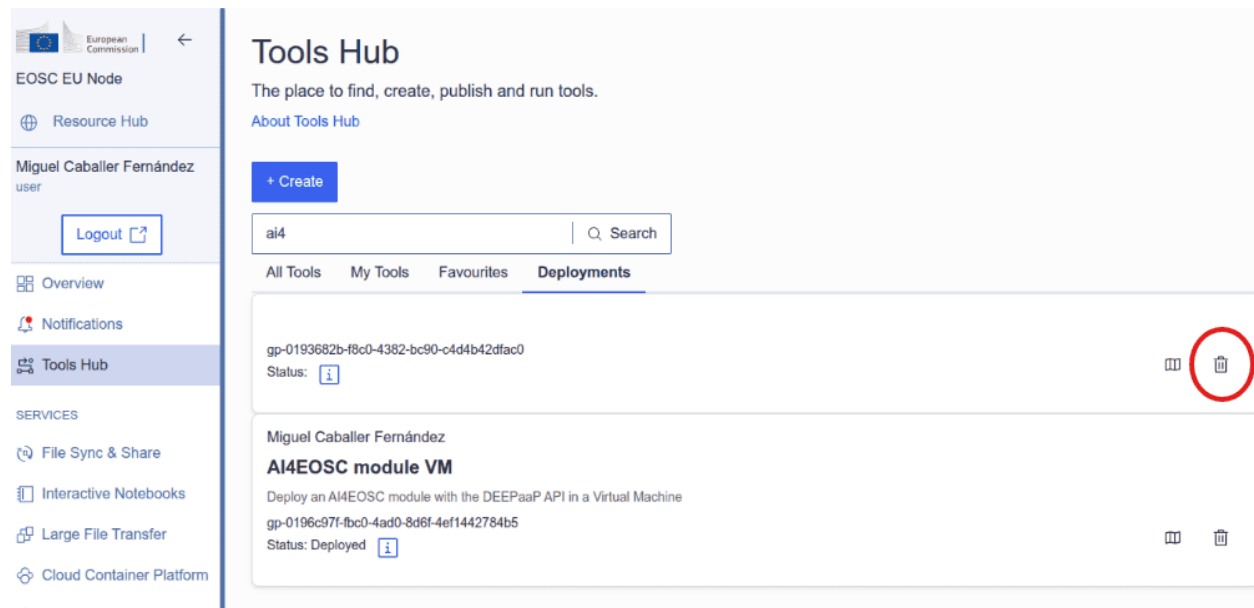
Instance Name	Image Name	IP Address	Flavor	Key Pair	Status	Availability Zone	Task	Power State	Age	Actions
simple-node-34d37ba0-9c53-11ef-8544-e66eef0c6c45	ubuntu-24.04-x86_64-server-cloudimg-20241105	10.0.1.232, 62.3.174.94	M1-NET-2vCPU-8R	-	Active	BSTOK10	None	Running	11 minutes	Create Snapshot

Displaying 1 item

You can obtain further information of your VM by clicking on the Instance Name of your resource.

Finally, once you have finished using the AI model, you can delete the deployment and release the resources. For that,

go to the Deployments tab in the Tools Hub section and remove it by clicking on the Trash button.



If you no longer want to deploy additional VMs, you should release the allocated project, to avoid using credits. For that, go back again to the **Virtual Machines** section and release your allocated environment by clicking on the **Release** button. A notification will be sent by the system once the resources have been released.

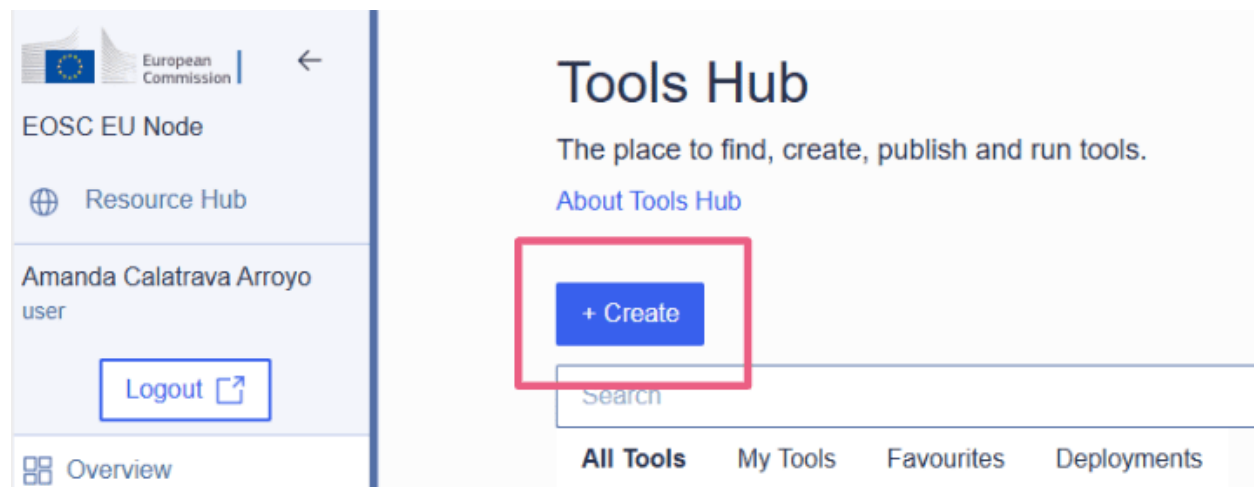
8.5.3 More

Further customize the Tool deployment

If you need to further customize the AI4EOSC model deployments, you can modify the reference TOSCA templates that were used to create the Tools in the Tool Hub:

- TOSCA VM example.
- TOSCA Container example.

To register this new tool, create a new Tool inside the Tools Hub section of the EOSC EU Node portal.



Fill the form and register the tool. Once registered, you can make it public and share it among the community. Notice the TOSCA Template needs to undergo a security assessment before it appears in the public catalog of Tools. This

security assessment is periodically reassessed.

8.6 Deploy an external model

Requirements

You need a *platform account* with *full access level*.

You can deploy models from external marketplaces in the *platform's dedicated resources* to perform inference. Currently we support the following marketplaces:

8.6.1 AI4Life

We currently support all V5 models from the AI4Life *BioImage Model Zoo*. Those model are accessible in the *Dashboard Marketplace panel*.

Marketplaces 

AI4EOSC

External marketplaces:

AI4Life

 Search modules

20 result(s) found

NucleiSegmentationBoundaryModel

Nucleus segmentation for fluorescence microscopy

3D UNet Arabidopsis Apical Stem Cells

3D Unet trained on confocal images of Arabidopsis thaliana apical stem cell:
<https://www.repository.cam.ac.uk/handle/1810/262530>.
Voxel size: [0.25, 0.25, 0.25]

HyLFM-Net-stat

HyLFM-Net trained on static images of arrested medaka hatchling hearts. The network reconstructs a volumetric image from a given light-field.

Feature extraction - MobileSAM Encoder

Pre-trained mobile-sam encoder for feature extraction

GelGenie-Universal-V1

U-Net trained to segment and extract gel bands from gel electrophoresis images. This is the universal version (V1) of the model.

GelGenie-Finetuned-V1

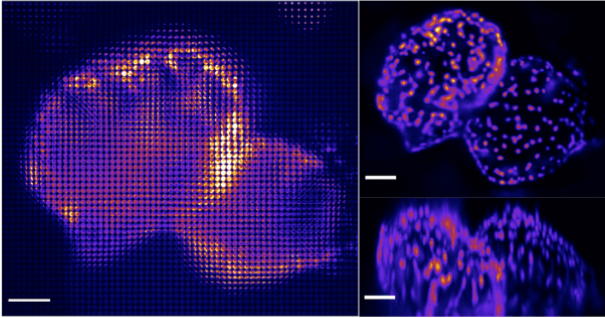
U-Net trained to segment and extract gel bands from gel electrophoresis images. This is the finetuned version (V1) of the model.

When clicking on a given model you will be show the model information along with the metadata of that model.

HyLFM-Net-stat

Deploy

HyLFM-Net trained on static images of arrested medaka hatchling hearts. The network reconstructs a volumetric image from a given light-field.



About

12735

MIT

Tags

biapy_3.5.6-compatible

biapy_3.5.8-compatible

hylfm

bioimageio.core_0.6.9-compatible

image-reconstruction

pytorch

biapy-compatible

Show more

Additional Resources

 Biolmage.IO

Dates

 2024-10-02

Click **Deploy** and *configure the resources*. Once it is deployed you can find it on the Tools deployment table (similar to the *Modules deployments table*).

Deployments 

Modules 

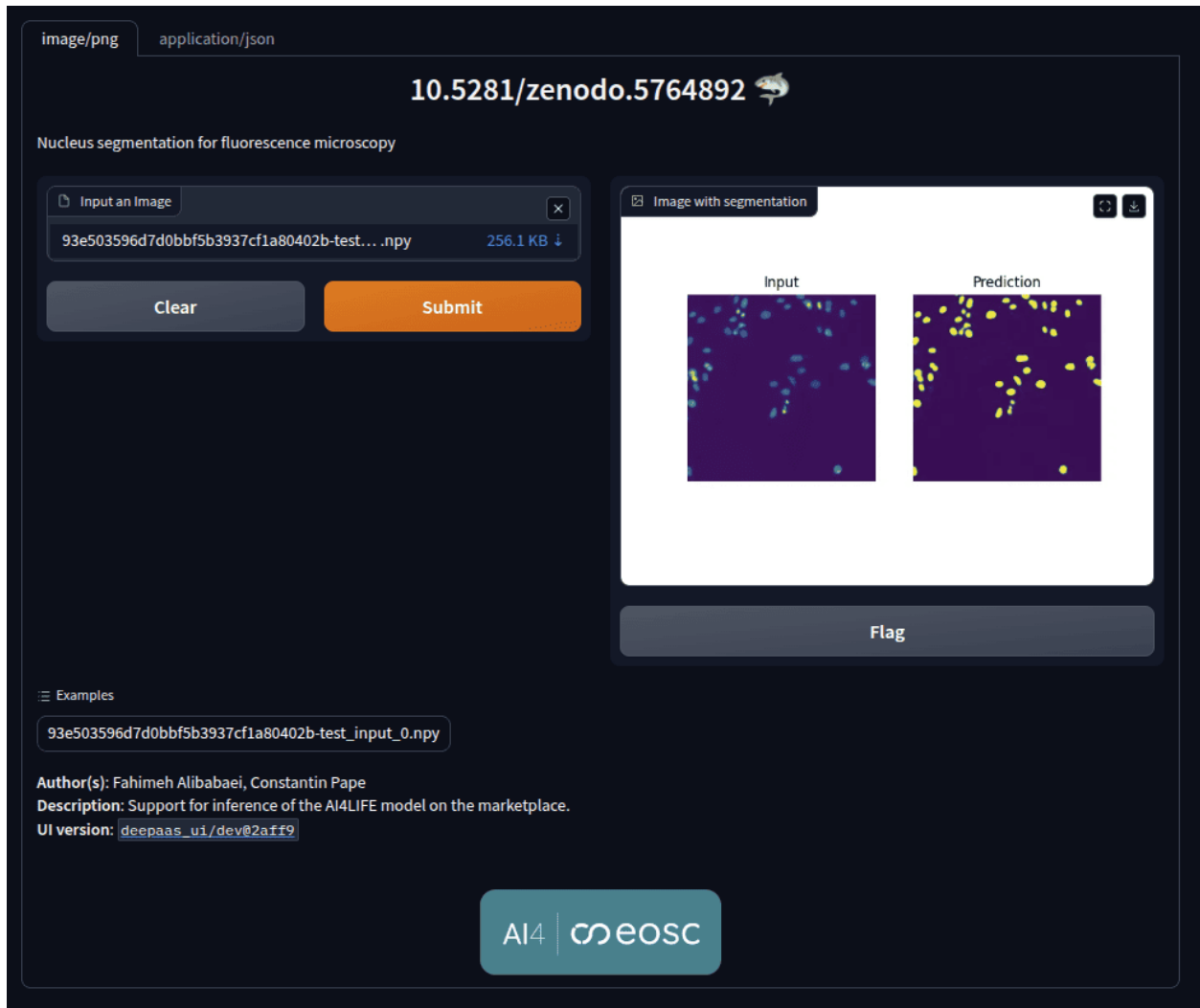
Name	Status	Container name	GPUs	Creation time (UTC)	Actions
gray_guan	running	ai4oshub/ai4os-demo-app:latest	0	2024-11-26 14:10:01	   

New deployment

Tools 

Name	Status	Container name	GPUs	Creation time (UTC)	Actions
CVAT test	running	ai4oshub/ai4os-cvat:v2.7.3-ai4os-ser	0	2024-11-22 14:10:21	  

Clicking on the **Quick access** button you will be redirected to the Gradio UI to make an inference with the model.



8.7 Deploy your own LLM chatbot

i Requirements

You need a *platform account* with *full access level*.

In this tutorial, we will guide you on how to deploy your own LLM instance in the platform.

For the moment, due to resource constraints, this tool allows to deploy only small LLMs that can fit in NVIDIA T4s. If you want to use bigger LLMs on faster resources, please *use the platform LLM*! We have a *detailed comparison* of the two options.

8.7.1 Deploying the LLM

In the *Dashboard Marketplace*, go to the **LLM catalog** and click on the model card you want to deploy. Alternatively, you can go to: **Tools catalog** → **Deploy your LLM** → **Deploy**.

Once in the configuration page, the workflow for deploying the LLM is similar to the one for *deploying a module*. In this particular case, during the LLM configuration phase you will need to pay attention to:

- **type:** choose what to you want to deploy. Options are:
 - **both:** deploy both the backend and the UI,
 - **vllm:** deploy only the backend,
 - **open-webui:** deploy only the UI,
- **LLM model** this is the particular model you want to deploy.

What is the best model for my needs?

For the time being, due to limitations in resources (deployments are made in NVIDIA T4s) we only support small models (eg. [DeepSeek-R1-Distill-Qwen-1.5B](#)) or medium models that have been quantized (eg. [Qwen2.5-7B-Instruct-AWQ](#)).

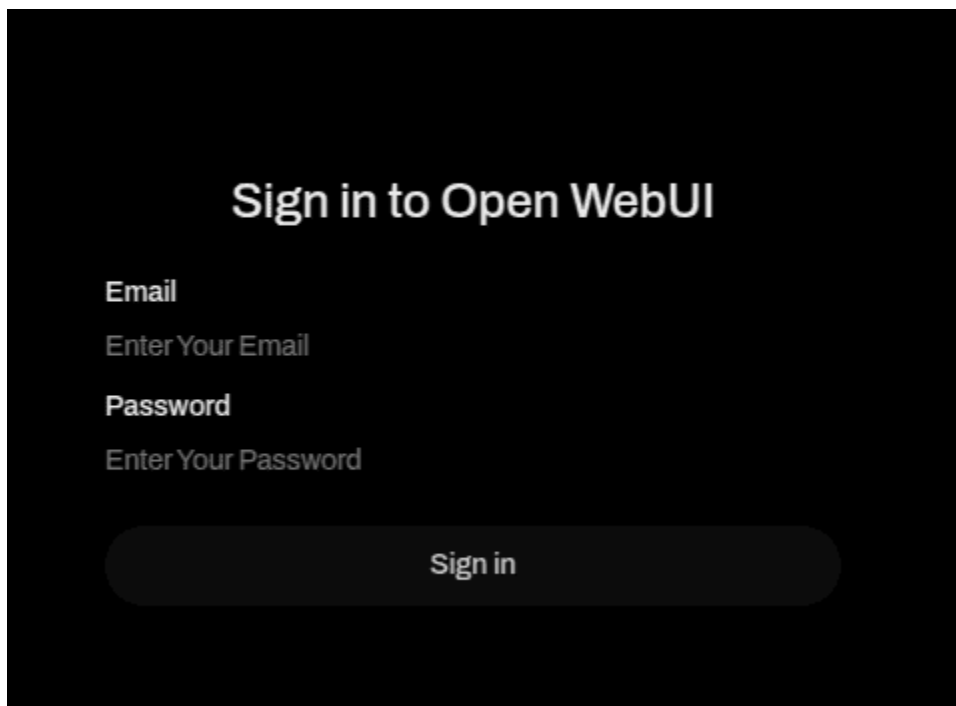
Medium-size quantized models are usually more accurate although they can have slightly slower inference speeds (due to the de-quantization process). We recommend using those.

Among the catalog of models we provide, there are models specialized in solving coding tasks or maths problems, so depending on your usecase you might want to select those.

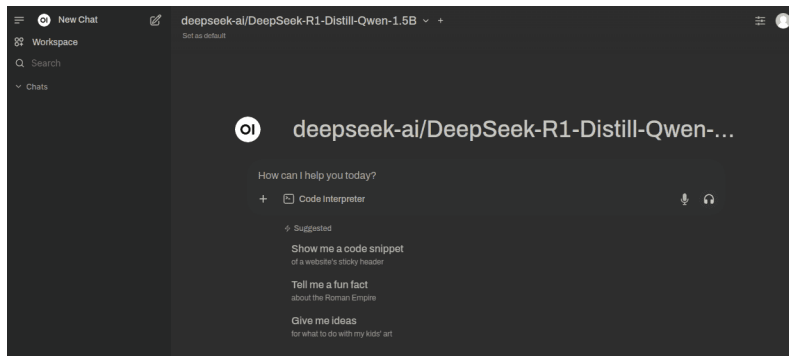
- **UI email, UI password:** these are the credentials to log to the UI. By default, we use your user email from the platform.
- *(Optional)* **HF token:** Using some models might require that you provide a valid HuggingFace token to deploy. For example, to use [LLama models](#) you must login to HuggingFace and accept their terms and conditions. Once this is done, you can automatically [generate a Huggingface token](#) to use Llama models in our platform.
- *(Optional)* **API key/url:** if you choose to deploy only the UI, you must provide the endpoint and key of an OpenAI compatible endpoint, that the UI can connect to.

8.7.2 Using the LLM

In the [deployments list](#) you will be able to see your newly created LLM instance. Clicking the **Quick access** button, you will directly enter the Open-WebUI login screen.



The enter you UI email and UI password and voilà, you're in!



You are the admin of the instance, so you can create new users for other people in your team.

Go to the [Open-WebUI documentation](#) to further configure your instance. For example, you can:

- customize the UI appearance: since current models do not have vision capabilities, so you can disable image upload for users going to Admin Panel → Settings → Models → <model-name> → Capabilities → Vision
- create your own [Knowledge Bases](#) as persistent memory banks,
- use [Functions](#) to create your own agents that use custom prompts, custom Knowledge Bases, and custom input/output filtering,
- *integrate the model with your own services* (use as code assistant in VSCode, use from Python scripts, etc).

CREATE AN AI INFERENCE PIPELINE

In addition to deploying standalone modules, we support the visual concatenation of different AI modules into a multi-step pipeline.

Pipelines with Node-Red Composing AI Inference pipelines with Node-RED.

Composing AI Inference pipelines with Node-RED Pipelines with Elyra Composing AI Inference pipelines based on OSCAR services with Elyra in EGI Notebooks.

Composing AI Inference pipelines based on OSCAR services with Elyra in EGI Notebooks

9.1 Composing AI Inference pipelines with Node-RED

In this document, we will learn about Composing AI Inference pipelines based on OSCAR services with Node-RED, specifically:

- how to create inference services in OSCAR,
- how to create a new “Flows” instance (based on Node-RED) in the OSCAR platform for AI4EOSC,
- how to access the instance and create our first application workflow,
- how to deploy a workflow to call inference services deployed in OSCAR,
- how to delete the Node-RED instance.

For a complete overview of the Node-RED examples, please refer to the [AI4Compose GitHub README](#).

First of all, let’s understand the key technologies involved in this tutorial:

- **Node-RED** is an open-source visual programming tool. Built on Node.js, it allows users to create event-driven systems by connecting nodes representing different functionalities. With a user-friendly web interface and a rich library of pre-built nodes, Node-RED simplifies the visual composition of pipelines.
- **OSCAR** is an open-source serverless platform to support scalable event-driven computations. From **OSCAR v3.6.5**, it provides support for deploying a Node-RED instance. More info regarding this new OSCAR feature can be found [here](#).

In the platform, we use Node-Red to visually compose AI model inference pipelines. Specific custom nodes have been created to perform AI model inference on remote OSCAR clusters.

9.1.1 1. Creating inference services in OSCAR

Let’s start by creating an OSCAR service to use it from Node-RED.

1.1. Deploy YOLOv8 service

Go to [OSCAR Dashboard](#) and, in the Services panel, select Create service → FDL. Use the following configuration:

FDL:

```
functions:
  oscar:
    - oscar-cluster:
        name: yolov8-node-red
        memory: 4Gi
        cpu: '2.0'
        image: ai4oshub/ai4os-yolov8-torch:latest
        script: script.sh
        log_level: CRITICAL
```

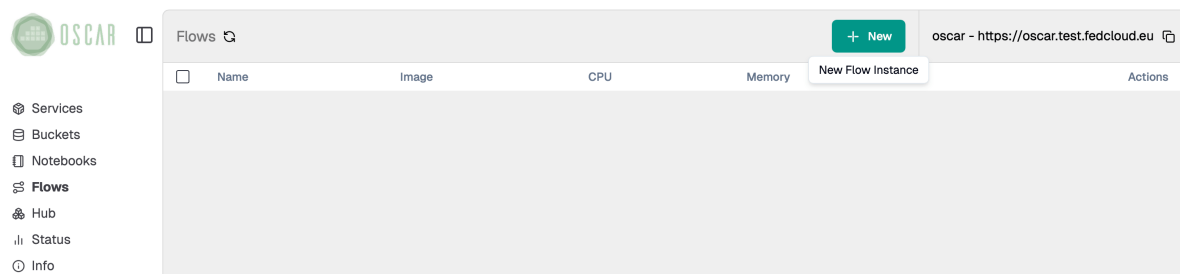
Script:

```
#!/bin/bash
RENAMED_FILE="${INPUT_FILE_PATH}.png"
mv "$INPUT_FILE_PATH" "$RENAMED_FILE"
OUTPUT_FILE="$TMP_OUTPUT_DIR/output.png"
deepaas-cli --deepaas_method_output="$OUTPUT_FILE" predict --files "$RENAMED_FILE" --
→ accept image/png 2>&1
echo "Prediction was saved in: $OUTPUT_FILE"
```

See the our documentation on *how to make an inference in OSCAR* for more information about creating OSCAR services.

9.1.2 2. Creating our Node-RED instance in OSCAR

1. In the [OSCAR Dashboard](#), go to the Flows panel and then click New.



2. Enter the **admin** password that you will be asked to access later on this instance of Node-RED, and select or create a Bucket.

Node-RED Instance Configuration

Service name

flows-dumape-gedzyfeb

CPU Cores

1.0

Memory RAM

2

Gi

VO

vo.imagine-ai.eu

Password Warning

Enter admin password

Credentials secret

Bucket

☐ New Bucket

Enter new bucket name

Deploy

3. After deploying Node-RED, we access its user interface.

Services

Buckets

Notebooks

Flows

Flows

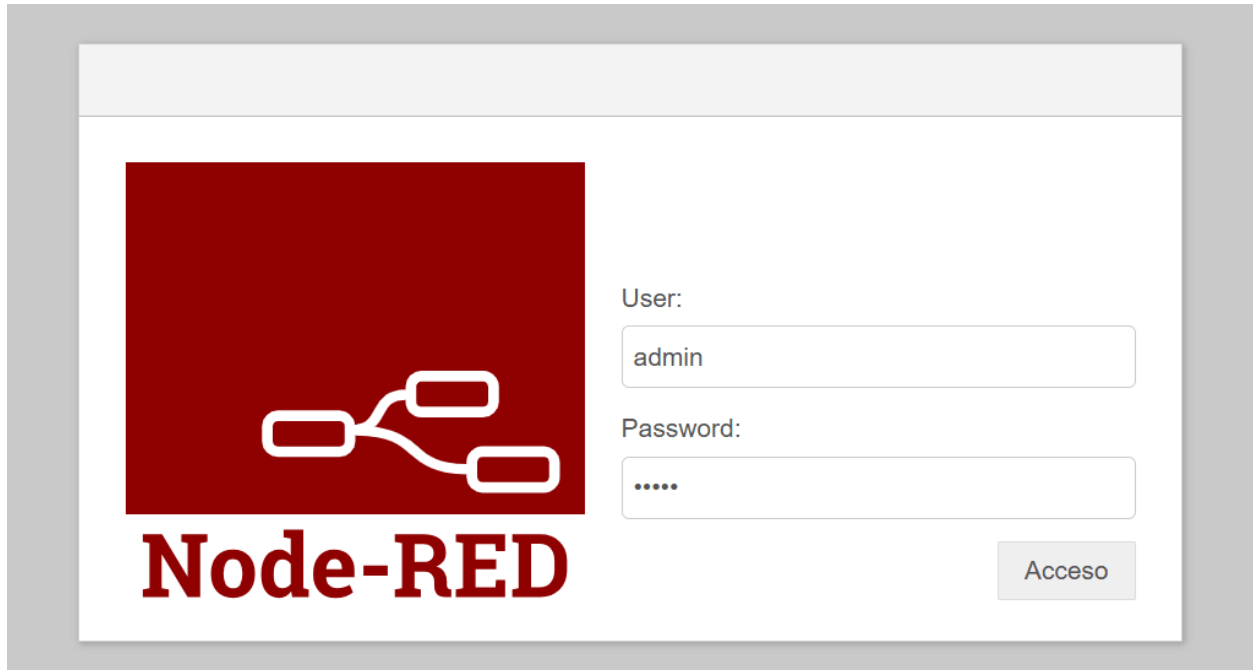
+ New

oscar - https://oscar.test.fedcloud.eu

☐	Name	Image	CPU	Memory	Actions
☐	flows-hipeka-3rmfyecw	ghcr.io/grycap/oscar-flows	1.0	2Gi	

9.1.3 3. Accessing the Node-RED instance

Once the Node-RED instance is up and running, you can log in with your credentials (the user is always `admin`).



9.1.4 4. Creating a workflow in Node-Red

We are going to showcase the usage of Node-RED with a simple example that uses the YOLO model. However, if you want to see some other examples, visit the [AI4Compose](#) repository.

4.1. Designing and creating the workflow

Let's create a workflow that fetches an image from the Internet, makes a request to the YOLO service and visualizes the result.

We need the following list of components from the Node-RED sidebar menu:

- **Common** → inject node
- **Network** → HTTP request node
- **Output** → image node
- **OSCAR** → OSCAR YOLO8 node

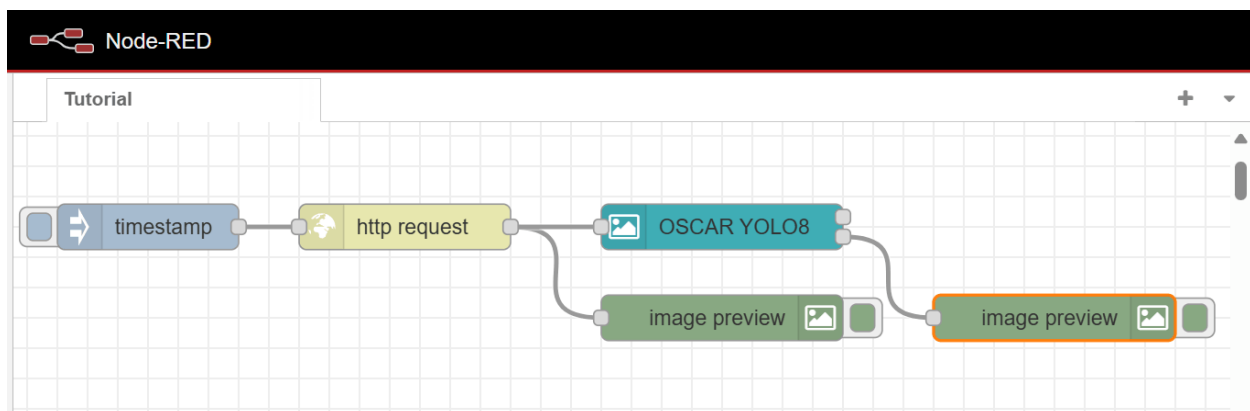
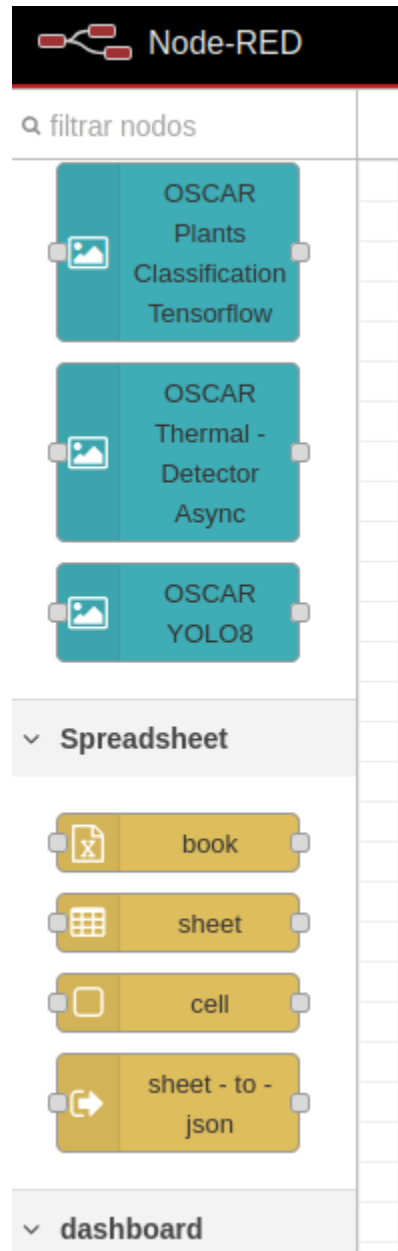
Drag and drop the boxes to the canvas and then connect the components as shown:

Now we need to configure the components. To configure the HTTP request node double-click on it:

- URL: URL of an image you want to analyze with YOLO (for example, you can use this [image](#))
- Payload: Send as request body
- Return: *A binary buffer*

Configure the OSCAR YOLO8 node:

- Server: URL of the OSCAR cluster. You can get it from [OSCAR Dashboard](#) → Info (Sidebar panel) → Endpoint
- Service Name: yolov8-node-red
- Token: Obtain the token from [OSCAR Dashboard](#) → Info (Sidebar panel) → Access token



Edit http request node

Delete

Cancel

Done

⚙️ Properties

⚙️

📄

🔗

☰ Method

GET

▼

🌐 URL

https://upload.wikimedia.org/wikipedia/commons/tl

Payload

Send as request body

▼

☐ Enable secure (SSL/TLS) connection

☐ Use authentication

☐ Enable connection keep-alive

☐ Use proxy

☐ Only send non-2xx responses to Catch node

☐ Disable strict HTTP parsing

← Return

a binary buffer

▼

☰ Headers

📄

☐ Enabled

Edit OSCAR YOLO8 node

⚙️ Properties
⚙️
📄
🖼️

Name

Name

Server

▼ a_z

Service Name

▼ a_z yolov8-node-red

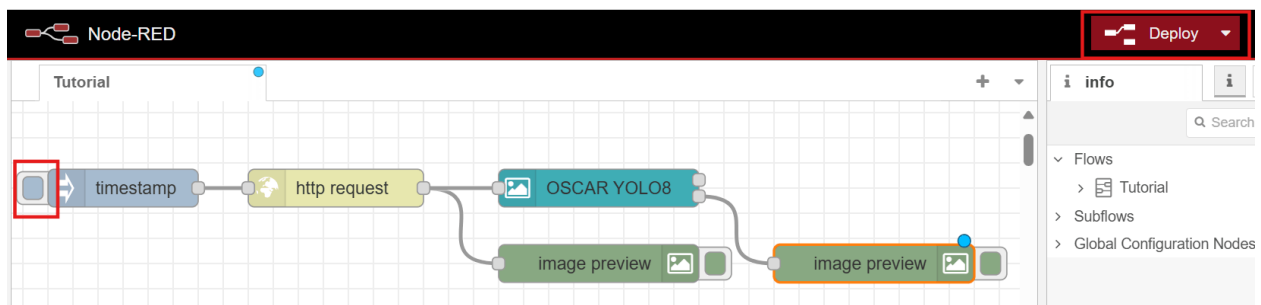
Token

🔒 *****

✎

4.2. Testing the workflow

After configuring your workflow, you can test it in the Node-RED Editor: click Deploy (top right corner) and then click on the inject node:



You should see the result as indicated below.

9.1.5 5. How to delete a Node-Red instance

To delete an instance, you have to click on the Delete button and confirm the operation.

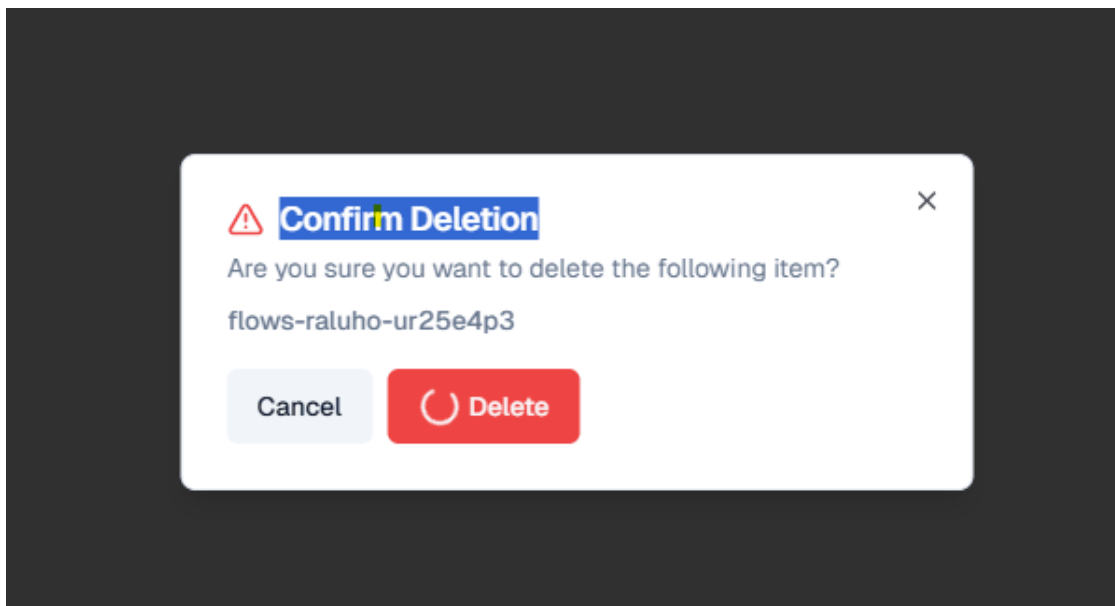
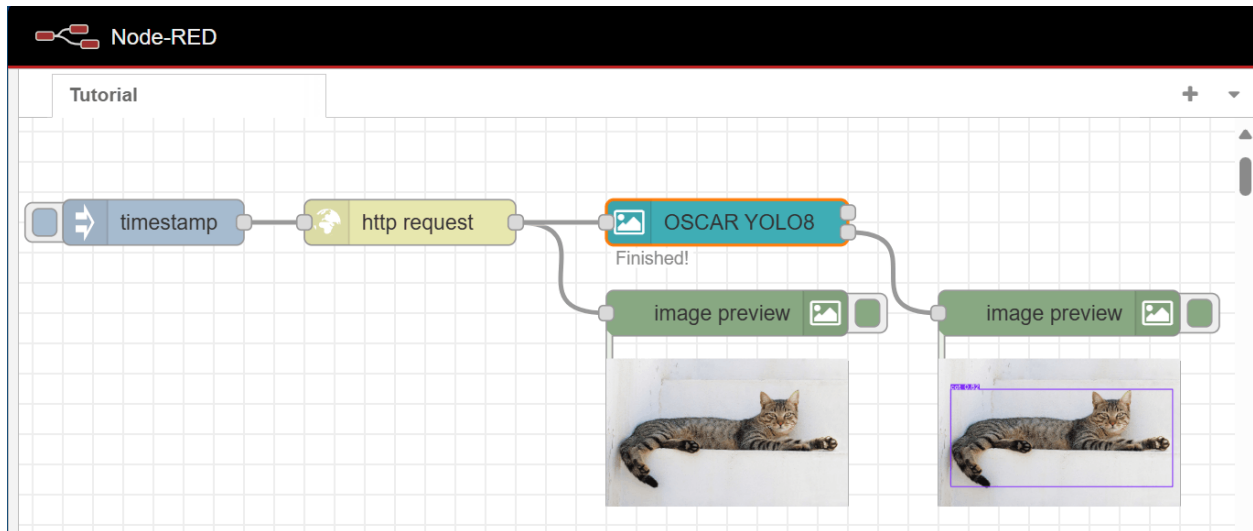
Services

Buckets

Notebooks

Flows

Flows				+ New	oscar - https://oscar.test.fedcloud.eu
☐	Name	Image	CPU	Memory	Actions
☐	flows-hipeka-3rmfyecw	ghcr.io/grycap/oscar-flows	1.0	2Gi	⋮ 📄 🗑️



The MinIO bucket remains available; however, it is a good practice to ensure that you have backed up any important data or configurations before deleting an instance.

9.2 Composing AI Inference pipelines based on OSCAR services with Elyra in EGI Notebooks

Create a pipeline with Elyra

Please, be aware that video demos can become quickly outdated. In case of doubt, always refer to the written documentation.



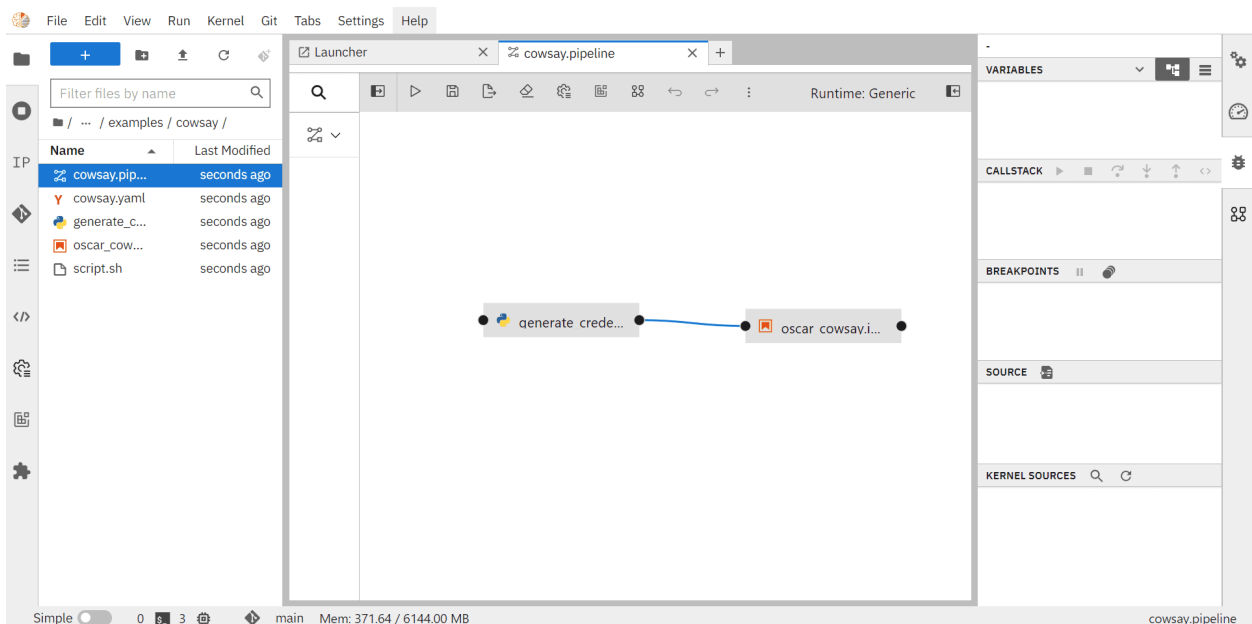
Tutorial update

- **OSCAR v3.6.5** now provides support for deploying a Jupyter Notebook instance with Elyra. Although the tutorial uses the EGI Notebooks service, you can also deploy your own Jupyter Notebook in OSCAR and try the examples of the tutorial. More info regarding this new OSCAR feature can be found [here](#).

In this documentation, we provide a hands-on guide to using [Elyra](#), an AI-centric extension for [Jupyter Notebooks](#), within [EGI Notebooks](#).

This tutorial will focus on:

- how to use Elyra in EGI Notebooks
- how to clone repositories using Elyra
- how to interact with [OSCAR](#), using the EGI Notebook Token.
- how to deploy a pipeline to utilize inference services using OSCAR.



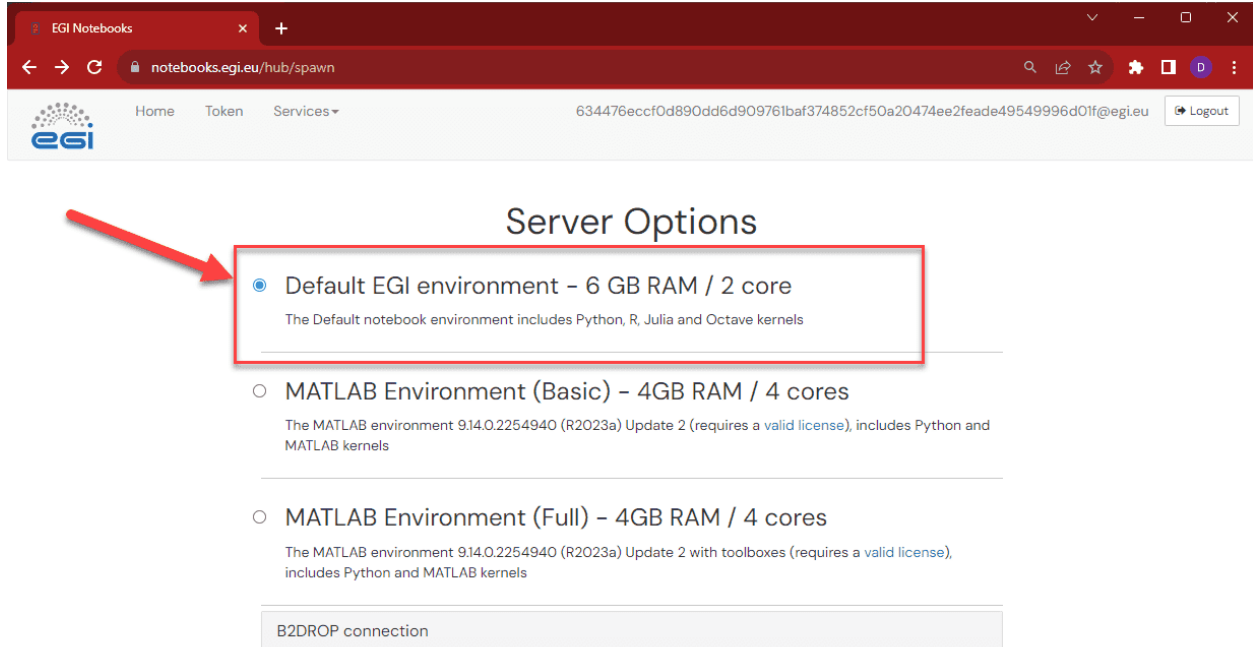
We will walk through the entire process, from setting up the initial environment to running specific OSCAR services like the Cowsay and others.

For a complete view of the repository containing these examples, please refer to the [GitHub README](#).

9.2.1 1. Setting up the environment

1.1 Accessing EGI notebooks

The first step is to access the [EGI Notebooks service](#). Then choose the preferred server option. We will select Default EGI environment:



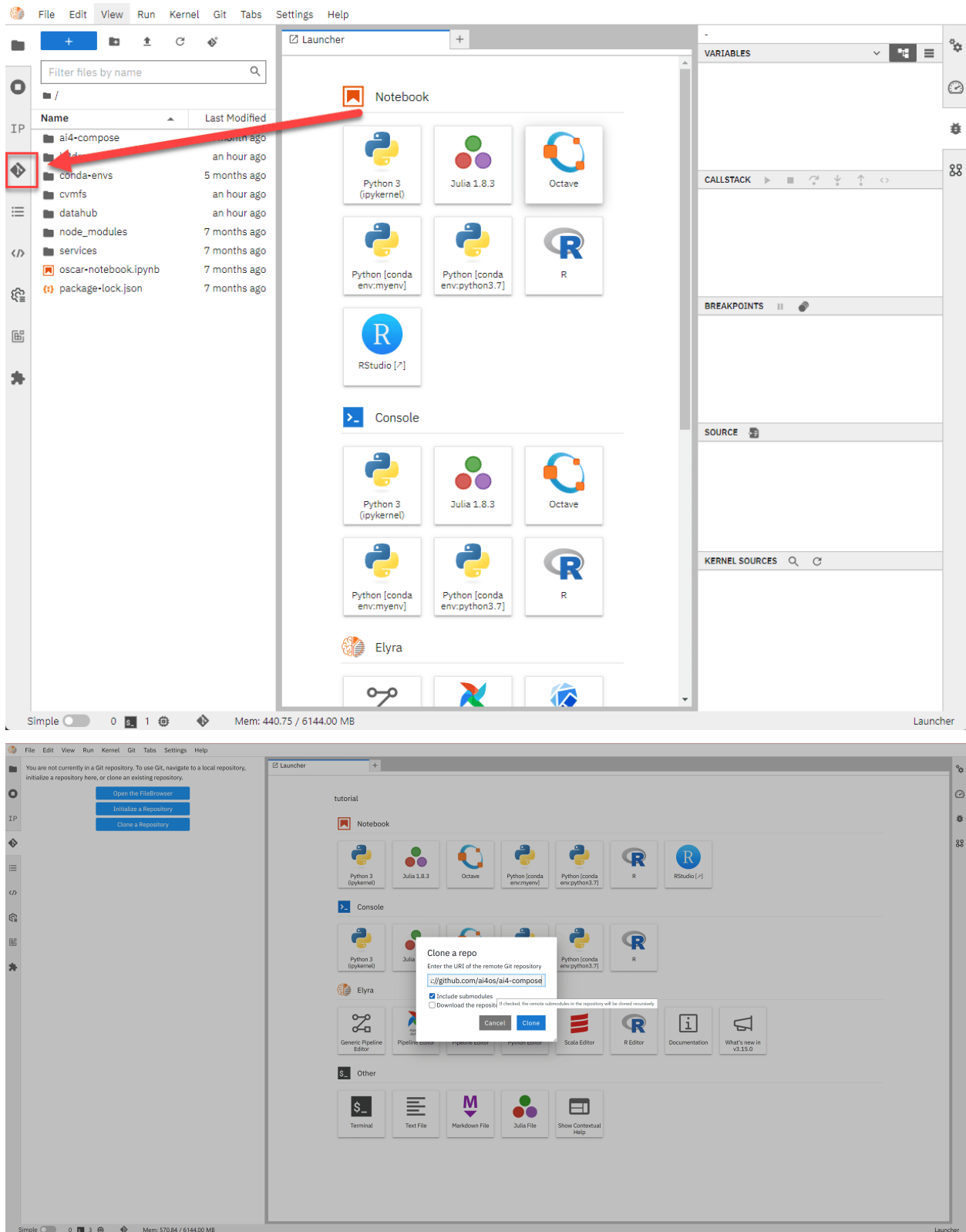
For additional information on using EGI Notebooks, refer to the [official documentation](#).

1.2 Importing OSCAR Elyra from Github

To get started, you will need to clone the repository that contains the example files for Elyra and OSCAR services: <https://github.com/ai4os/ai4-compose>

As a good practice to keep your workspace organized, you may first create a dedicated folder (for example, called **tutorial**) inside your home directory in the EGI Notebooks environment. You can then clone the repository inside this folder. This is optional but recommended to avoid clutter in your main workspace.

You can use the Elyra git tool on the left side panel to clone the repository in the EGI Notebooks environment. This repository contains various examples, including those for Elyra, such as Cowsay, Grayify, and Plants-Theano.



9.2.2 2. Prerequisites for all the examples

2.1 Using OSCAR in Your Workflow

To use nodes that rely on OSCAR, you need to invoke a service through an OSCAR client. First, create your own OSCAR client, configuring it with the necessary credentials and parameters to interact with an OSCAR cluster.

2.2 Setting Up an OSCAR Client

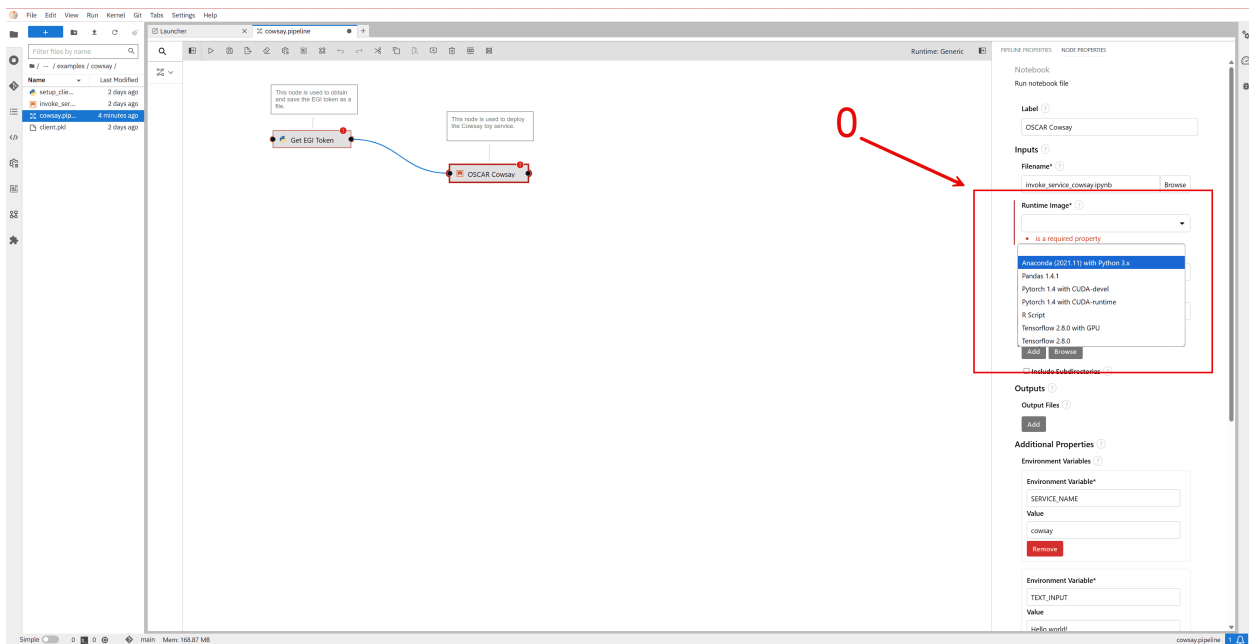
In this example, we will navigate to the `examples` folder, then into the `cowsay` folder, and open the `cowsay.pipeline` file. This pipeline is already set up with the nodes `setup_client.py` and `invoke_service_cowsay.ipynb`.

To configure the `setup_client` node, follow these steps:

0. When creating a new node, or if a node has no configuration, it is necessary to assign a **runtime image**. You can choose from several options, but we recommend selecting:

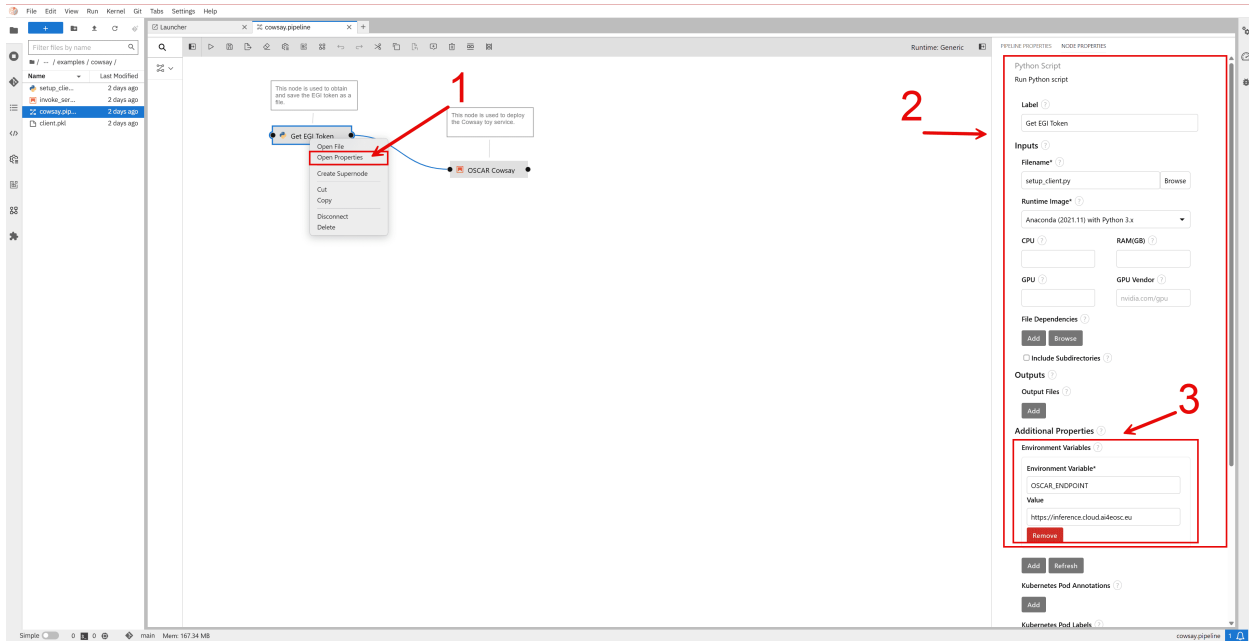
Anaconda (2021.11) with Python 3.x

This will eliminate the "is a required property" error, which causes the node to be marked in red and display an exclamation icon.



1. Right-click on the `setup_client` node and select the **Open properties** option.
2. Go to the side menu to access the configuration fields.
3. Set up the OSCAR client by obtaining the OSCAR endpoint. For AI4EOSC, the endpoint is:

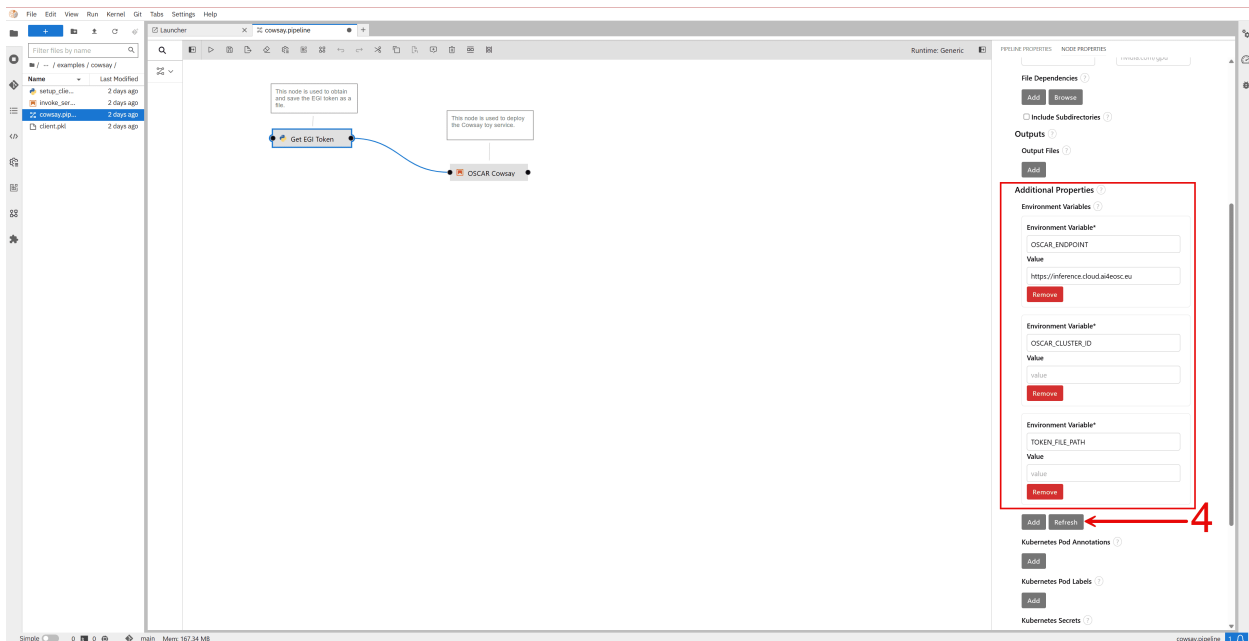
<https://inference.cloud.ai4eosc.eu>



4. In the *Environment Variables* section, click the **Refresh** button. This will populate the variables:

- OSCAR_CLUSTER_ID (optional)
- TOKEN_FILE_PATH

These variables are required to configure the client properly.



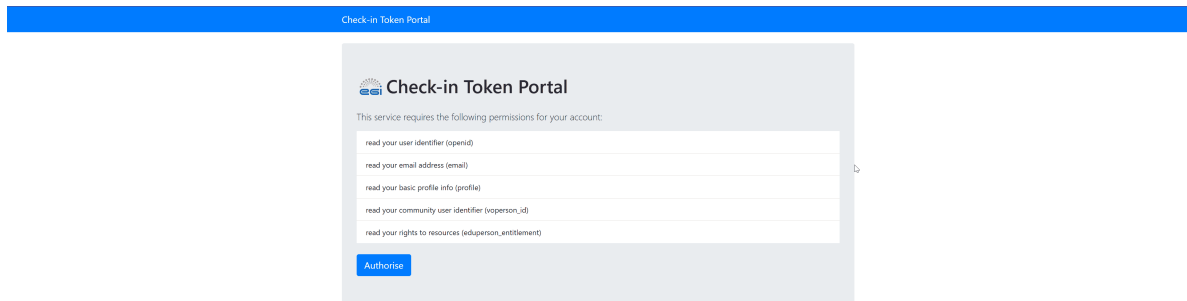
By default, if you are running in the EGI notebook environment, the token is automatically read from the EGI notebook environment, you need to generate a refresh token from the EGI token page:

`/var/run/secrets/egi.eu/access_token`

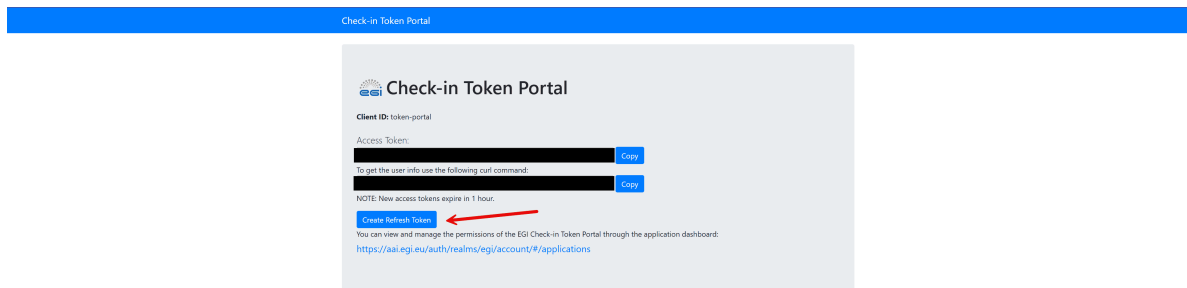
and no further configuration of the token is required. The `TOKEN_FILE_PATH` variable is only relevant if you are running Elyra in your own environment. In that case, you will need to generate a refresh token manually from the [EGI token page](#) and use it to configure the client, which will allow you to interact with OSCAR services.

Follow these steps to generate a refresh token:

1. Authenticate on the EGI portal.



2. Click the button to create a refresh token.



3. Copy the generated refresh token.

Note: providing a token file path is optional, as the setup client node (also called EGI Token node) can extract the token automatically from the Jupyter notebook environment. Use the token file path only if you explicitly generated and want to provide a refresh token.

Once the client is set up, you can seamlessly integrate OSCAR nodes into your workflow.

2.3 Configuring the OSCAR Client

Before running any example, ensure your OSCAR client is properly configured. You may need to set environment variables such as:

- **Endpoint:** The URL of the OSCAR inference service.
- **ID (optional):** The identifier for the OSCAR service.
- **Token file path (optional):** The location of the refresh token, if applicable.

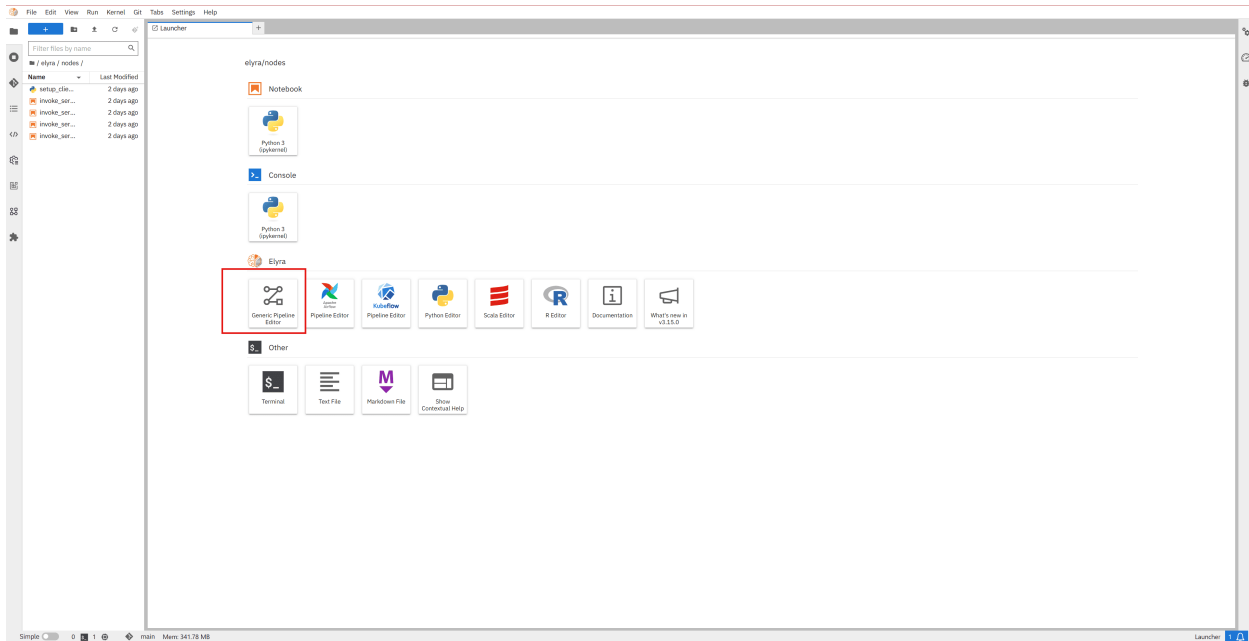
Once configured, you can execute workflows and use OSCAR nodes within your pipeline.

2.4 Creating Your Own Pipeline

Before deploying or configuring a provided pipeline, you may want to create your own from scratch. Elyra makes this process simple through its visual pipeline editor.

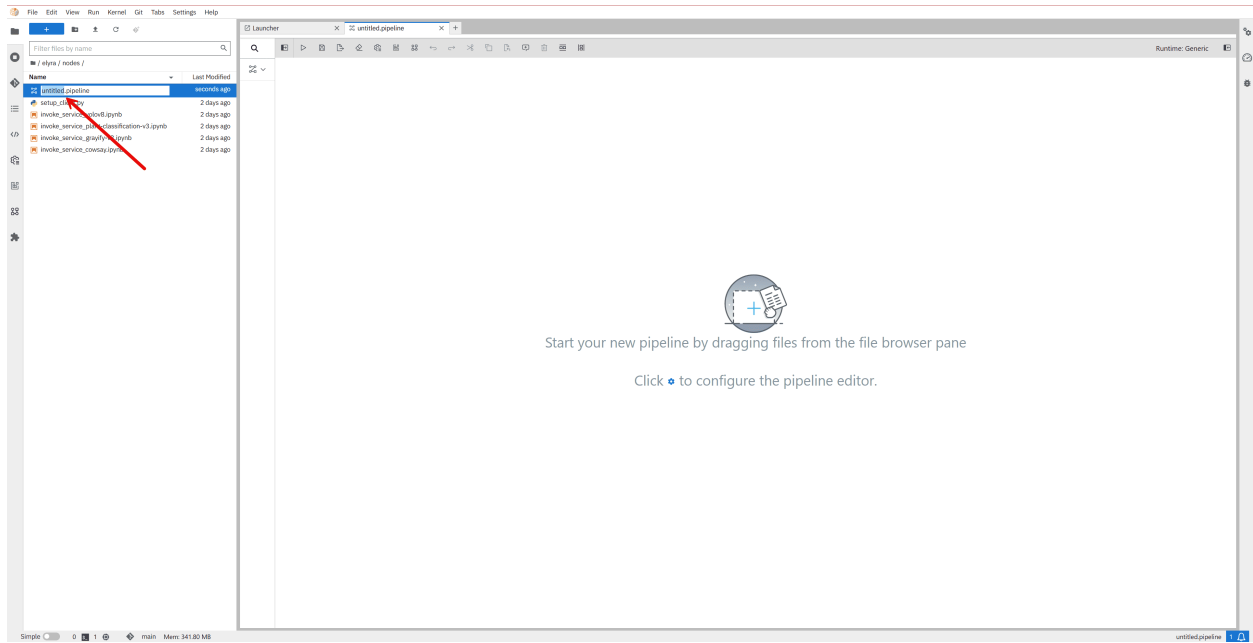
Follow these steps to create a pipeline and add nodes:

1. From the JupyterLab Launcher, click on the **Generic Pipeline Editor** icon under the Elyra section. This will open an empty canvas where you can build your workflow.

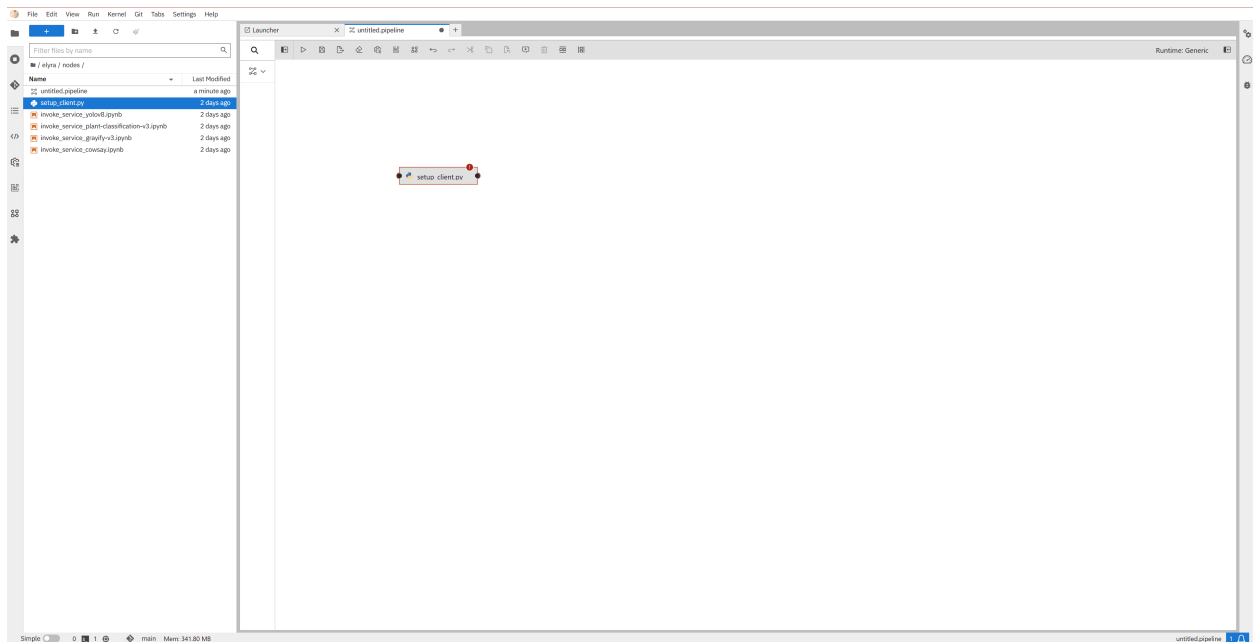


2. In the side panel, navigate to the **File Browser** and locate the notebooks (`.ipynb`) or scripts (`.py`) you want to include as nodes in your pipeline. The provided example nodes can be found in the repository under the folder:

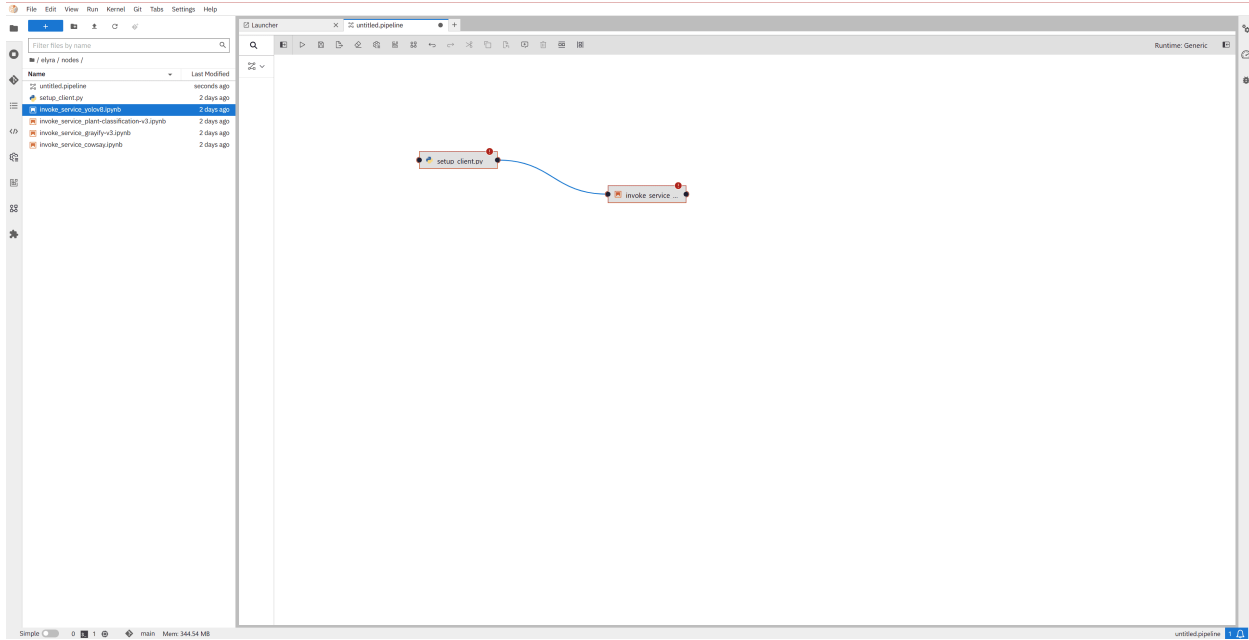
```
/elyra/nodes/
```



3. Drag and drop each file onto the canvas. Each file becomes a node in the pipeline.



4. Connect the nodes by dragging a line from the output of one node to the input of the next, defining the execution order.



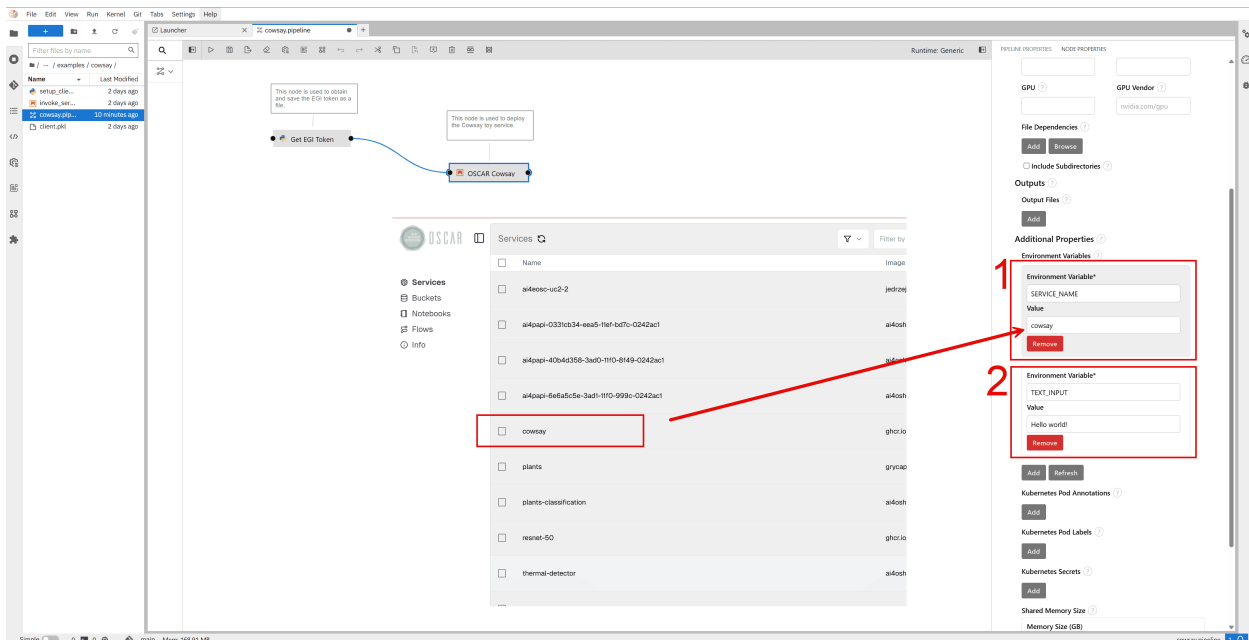
To remove the error shown in the nodes (“*is a required property*”, marked in red with an exclamation icon), simply follow **Step 0** described in section 2.2 and assign a runtime image to each node.

Once your pipeline is built, you can save it (e.g., in the `examples` folder) and proceed to the next section, where we explain in detail how to run and configure the provided `cowsay.pipeline` example.

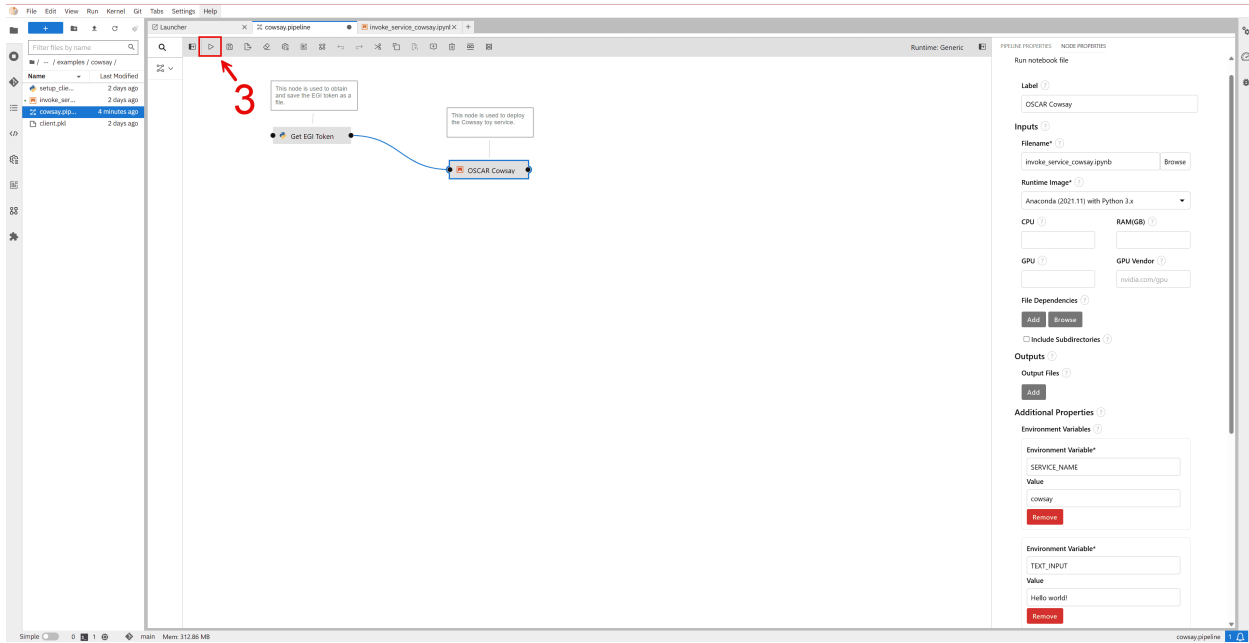
3.1 Running the Cowsay Example

Now let’s get our cow to talk! Follow these steps to set up the workflow in Elyra:

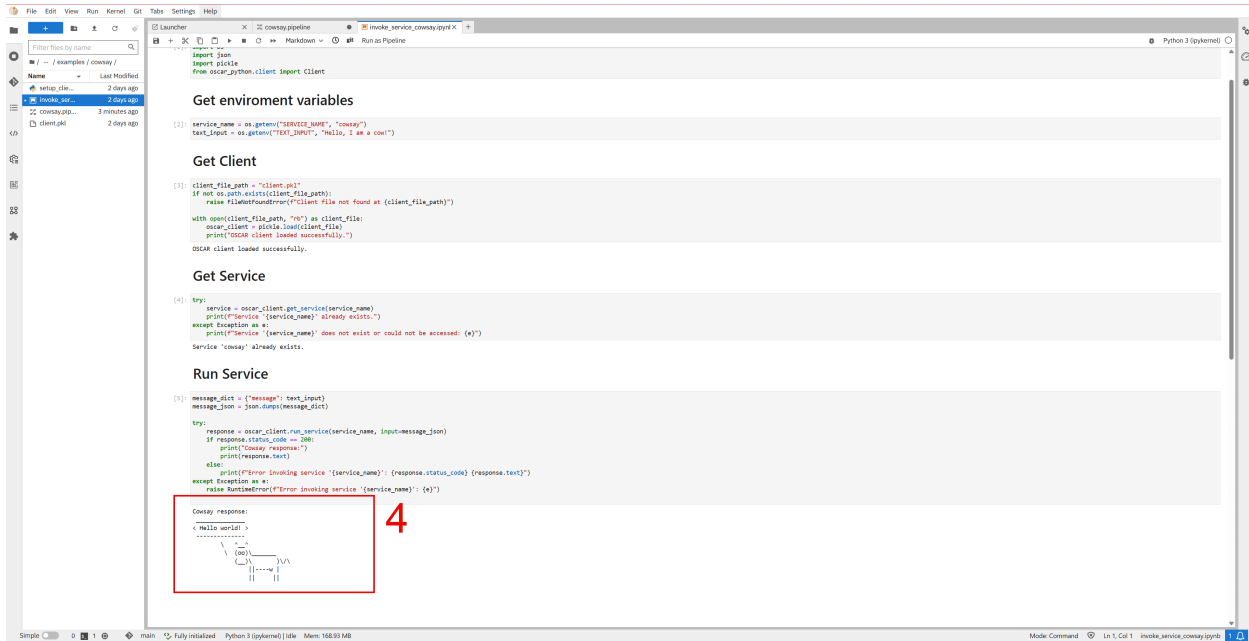
1. In the *Environment Variables* of the `cowsay` service node, set the name of the service assigned to the OSCAR cluster.
2. Specify the text you want the cow to say (e.g., `Hello world!`) as an environment variable.



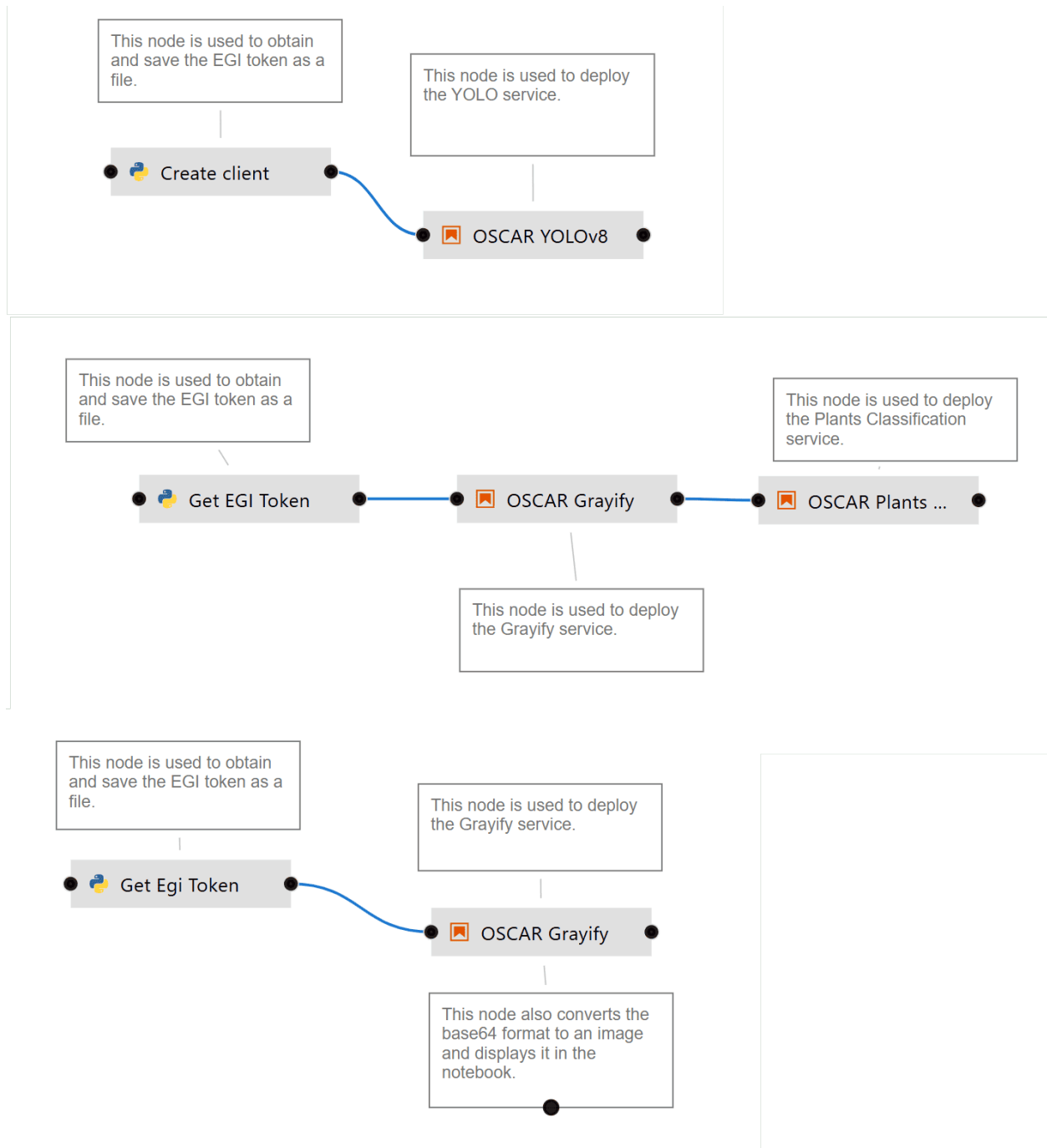
3. Use the start button to execute the pipeline.



4. After setting up the environment variables, proceed to the notebook within this node. Once executed, the notebook should display the cow uttering the text you provided.



3.2 Additional Examples: Grayify and Plants-Theano



There are many more examples and nodes available in the `examples` folder that you can explore. These demonstrate different ways to compose functions for inference with OSCAR, as well as additional nodes that perform tasks such as image encoding/decoding and other preprocessing steps.

DASHBOARD

The Dashboard allows users to access computing resources to deploy, perform inference, and train modules hosted at the Marketplace. The Dashboard simplifies the deployment and hides some of the technical parts that most users do not need to worry about.

Currently, the following platforms have deployed a version of the Dashboard. You should access one of those Dashboards depending on the project you are a member of:

- [AI4EOSC Dashboard](#) (*use this one if you are unsure which one to use*)
- [iMagine Dashboard](#)
- [AI4Life Dashboard](#)
- [KMD4EOSC Dashboard](#)
- [Tutorials Dashboard](#)

The Dashboard has a two views:

- a **public view** that let's you browse through the modules
- a **private view** that additionally allows you to make deployments based on those modules and check the statistics. To access this view you need *authentication*.

In the remaining part of this doc we will assume you have access to this private view.

The Dashboard is divided between modules (AI models) and tools (eg. an image labelling tool, a federated server, etc). In the remaining part of this doc we will focus on how to deploy a module but the workflow is similar for tools.

10.1 Profile

In the top-right corner, you will be able to find:

- : notifications about the platforms, like expected downtimes and issues,
- : your profile details.

AI4 |

Ignacio Heredia

Profile

Personal information

Full name

Ignacio Heredia

Email

iheredia@ifca.unican.es

Virtual organizations

ai4eosc

member

vm_operator

member

imagine-ai

member

member

vm_operator

ai4life

member

vm_operator

Storage providers

AI4OS Nextcloud

share.services.ai4os.eu

Re-link

Custom Nextcloud

Custom domain

Add

Other services

iheredia@ifca.unican.es

.....

Hugging Face

.....

In your profile details you will find:

- **Personal information:** your full name and your email.
- **Virtual organizations:** the VOs you are a member of and which roles you have in them.
- **Linked services:** many of the ecosystem services can be linked to the Dashboard, for a tighter integration that delivers a smoother experience:

You can link for example:

- the *Storage* as well as *external storage providers*,
- the *MLflow Experiment Tracking*,
- your *Huggingface* account,

In order to link a service, you have to click on the **Link** button, log in the popup window and grant access to the Dashboard. If you have previously linked the service, you also have to delete old credentials and generate new ones using **Re-link**.

Link with any NextCloud storage service

Please, be aware that video demos can become quickly outdated. In case of doubt, always refer to the written documentation.

10.2 Navigating the Marketplace

Once you log into the Dashboard, you will be shown all the items the Marketplace has to offer. They are organized around the following catalogs:

The screenshot shows the AI4EOSC Marketplace dashboard. On the left is a sidebar with navigation links: Dashboard, Statistics, Catalog (with 'Modules' selected), Tools, LLMs, Tasks (with sub-links: Try me, Deployments, Batch training, Inference), Platform Services (with sub-links: Storage, Experiment tracking, AI Inference workflows), and a footer note about the dashboard being a service provided by CSIC, co-funded by AI4EOSC. The main area is titled 'Modules' and shows a search bar, filters (Libraries, Tasks, Platform..., Data Ty..., Tags), and a sort dropdown (By name, By most recent). It displays a grid of 9 AI modules, each with a title, description, and a small icon. The modules are: 2D semantic segmentation (Vaihingen dataset), 2D semantic segmentation (multilabels with Unet), ai4os-demo-app, Apple tree blossom image segmentation, Artistic Style Transfer, audio_vessel_classifier, Bird sound classifier, Body pose detection, and Chest x-ray image classifier. An 'Ask AI' button is visible in the bottom right corner of the grid.

10.2.1 Modules catalog

The Modules catalog is a set of AI models designed to perform given AI tasks (eg. image classification). The catalog shows both models developed specifically for the platform, as well as *external model catalogs* that have been integrated to make them deployable in the platform.

You can use filters to quickly find the module you want. We provide filtering by:

- **Libraries:** this is the Deep Learning the module uses (eg. Pytorch, Tensorflow)
- **Tasks:** this is the broad task the module addresses (eg. Computer Vision, Natural Language Processing)
- **Platform Categories:** platform-specific tags. Current options include:
 - AI4 tools
 - AI4 pretrained: modules that already come with a trained AI model
 - AI4 inference: modules that can be used for inference (usually overlaps with *AI4 pretrained*)
 - AI4 trainable: modules that can be trained on a new dataset

For example the [image classifier](#) is both pretrained (because it comes with a model trained on ImageNet), inference-ready (because it allows to use that model to do predictions) and trainable (because it allows to re-train that model on a different dataset, to create for example a [plant classifier](#))

- **Data type:** type of data the module processes (eg. Image, Audio)
- **Tags:** any module-specific tag

10.2.2 Tools catalog

The Tools catalog is a collection of tools that come handy in assisting users during the whole Machine Learning workflow cycle.

We currently offer for example:

- The **Development Environment** to develop new modules ([Develop a model from scratch](#))
- Several **Federated Learning Servers** to make federated privacy-friendly trainings (Flower, NVFLARE)
- The **Computer Vision Annotation Tool** (CVAT) to label images ([Labeling images with CVAT](#))

10.2.3 LLM catalog

The LLM catalog is a collection of LLM models that users can self-deploy. The catalog has multiple open-source models from a wide range of AI-relevant actors (Qwen, DeepSeek, Meta). For more information on how to use them, please refer to the tutorial: [Deploy your own LLM chatbot](#).

10.3 Making a deployment

Let's focus for the time being in the module catalog. Once you choose the module, you will be presented with the module's information:

Train an image classifier

Try ▾

Deploy ▾

Codespaces ▾

Batch

Train your own image classifier with your custom dataset. It comes also pretrained on the 1K ImageNet classes.

The deep learning revolution has brought significant advances in a number of fields [1], primarily linked to image and speech recognition. The standardization of image classification tasks like the [ImageNet Large Scale Visual Recognition Challenge](#) [2] has resulted in a reliable way to compare top performing architectures.

This Docker container contains the tools to train an image classifier on your personal dataset. It is a highly customizable tool that let's you choose between tens of different [top performing architectures](#) and training parameters.

The container also comes with a pretrained general-purpose image classifier trained on ImageNet. The PREDICT method expects an RGB image as input (or the url of an RGB image) and will return a JSON with the top 5 predictions.



References

1. Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. [Deep learning](#). Nature, 521(7553):436-444, May 2015.
2. Olga Russakovsky et al. [ImageNet Large Scale Visual Recognition Challenge](#). International Journal of Computer Vision (IJCV), 115(3):211-252, 2015.

Apache-2.0

build passing

Tasks

Computer Vision

Classification

Libraries

TensorFlow

Keras

Data Types

Image

Tags

deep learning

image classification

vo.imagine-ai.eu

general purpose

Platform Categories

AI4 trainable

AI4 pre trained

AI4 inference

Provenance

View

Download
RDF

Additional Resources

GitHub

Report
issue

Dataset

Docker

Weights

99 Citation

Dates

2024-04-25

2025-05-22

To deploy click in Deploy > Inference API (dedicated) and you will be redirected to a configuration page.

Configure training: **Train an image classifier**

Modules / Train an image classifier / Train

 Show help

1 General configuration 2 Hardware configuration 3 Storage configuration

Deployment options

Deployment title*	Deployment description
Service to run Deepaas	Password* 
Hostname	

Docker options

Docker image deepcdc/deep-oc-image-classification-tf	Docker tag latest
---	----------------------

Next

This page will allow you to configure mainly three aspects:

- **General configuration**, including the service to run and Docker tags.
- **The computing resources** of the new deployment. A user can select multiple CPUs and GPUs, the machine RAM as well as optionally choosing the physical site where the machine must be deployed.
- **The remote storage options**, like tokens for authentication with Nextcloud.

Use the **Show help** toggle to view additional info about the fields to fill.

Once you are happy with the state of your configuration, click **Submit** and you will be redirected to the page listing all the current deployments.

10.3.1 General configuration

The parameters to configure are:

- **Deployment title**: short name/sentence to quickly identify your deployment.
- **Deployment description**: longer description of your deployment.
- **Service** determines which service to launch:
 - For performing simple inference, **DEEPaaS (API)** is the recommended option, as no code changes are required.
 - For retraining a module, **JupyterLab** is the recommended option, as it offers access to Terminal windows which are needed to mount remote data into your machine.
 - For developing a new module, **JupyterLab** is the recommended option, as it offers the possibility to directly interact with the machine to write code. Some modules might offer also **VScode**.

If you select either **JupyterLab** or **VScode** you must set a password at least 9 characters long.

What if I want both DEEPaaS and VSCode ?

We do not provide the option to run both JupyterLab and DEEPaaS at the same time, as code changes performed subsequently via JupyterLab wouldn't be reflected in DEEPaaS (which is launched with the initial codebase), thus potentially leading to confusion.

If you want to have access to both services in the same deployment, launch with JupyterLab. In JupyterLab, open a **Terminal** window ((New launcher) **Others Terminal**). Then run `deep-start --deepaas` to launch DEEPaaS. If you make subsequent code changes, you will have to kill the old DEEPaaS process and launch a new one.

- **Docker** tag selects the appropriate Docker tags of your module (tags may vary across modules). You should choose Docker tag that match with the hardware you selected in the previous step. So if you selected a CPU, look for `latest` or `cpu` tags. If you selected a GPU, look for `gpu` tag.

10.3.2 Hardware configuration

Choose the hardware type to run on:

- For inference and code development, we recommend using CPU as they are low intensity tasks.
- For (re)training, we recommend using GPU as this is a more demanding task. For the time being we limit to 1 GPU per deployment (and 2 GPUs per user) to allow for a fair distribution of resources.

10.3.3 Storage configuration

This is where you can configure how to connect your storage to your deployment. You have two sections:

1. Storage configuration

This is what will allow you to access your storage from inside your deployment.

You will be able to select any storage from the ones you have configured in your [Profile section](#).

For advanced users, it is also possible to fill your RCLONE credentials manually. Please [go here](#) in order to find how to create them.

Configure training: **ai4os-demo-app**

Marketplace / ai4os-demo-app / Train

Show help

General configuration

Hardware configuration

3 Data configuration

Storage configuration

Storage service
share.services.ai4os.eu

Advanced settings

RCLONE user
EGI_Checkin-34d879088c1fa64f1653f8e31df00ad3

RCLONE password
.....

RCLONE configuration
/srv/.rclone/rclone.conf

Storage URL
https://share.services.ai4os.eu/remote.php/webdav/

RCLONE vendor (webdav)
nextcloud

2. Download external datasets

This section provides the option to sync with datasets from multiple external repositories, including [Zenodo](#), [Hugginsface](#), [Figshare](#), [Github](#), [Seano](#), [Data Europa](#), [Dryad](#), [Open Science Framework \(OSF\)](#), [Mendeley Data](#) and [many more!](#)

- For Zenodo, we provide an embedded search functionality to find the datasets attached to any community.
- For all repositories, we provide the ability to directly provide a DOI or a URL.

With any dataset, you can select a `force_pull` option, so that if your dataset already exists in your storage it will overwrite the existing files.

If the module you are deploying has an reference dataset listed by the module's creator, this will appear as a suggested dataset.

Download a dataset from Zenodo

Please, be aware that video demos can become quickly outdated. In case of doubt, always refer to the written documentation.

Dataset configuration

Name	Source	Force pull	Actions
EyeOnWater training dataset for assessing the inclusion of water images	zenodo	☐	
Segmentation masks of ZooScan images focusing on images with several objects separated by a human operator	zenodo	☒	
LifeWatch observatory data: phytoplankton annotated trainingset by FlowCam imaging in the Belgian Part of the North Sea	zenodo	☐	

[Add dataset](#)

[Back](#) [Submit](#)

10.4 Managing the deployments

In the Deployments tab (in the Navigation panel on the left) you have a table view of all the deployments you have made so far, separated both in Modules and Tools tables:

Deployments

Modules 					
Name	Status	Container name	GPUs	Creation time (UTC)	Actions
gray_guan	running	ai4oshub/ai4os-demo-app:latest	0	2024-11-26 14:10:01	   
New deployment					

Tools 					
Name	Status	Container name	GPUs	Creation time (UTC)	Actions
CVAT test	running	ai4oshub/ai4os-cvat:v2.7.3-ai4os-ser	0	2024-11-22 14:10:21	  

Under **Info** you will find details about your deployment such as UUID, resources assigned/requested, error messages, endpoints of all services, etc. For the endpoints of the services you have:

- **API:** *Module's API*, only accessible if you launched with the DEEPaaS command or launched JupyterLab then ran DEEPaaS.
- **IDE:** *Development environment*, only accessible if you launched with the JupyterLab or VScode command.
- **Monitor:** *Training monitoring*, only accessible if the module has been coded to explicitly display monitoring (check the module's README or training arguments) and if a training is currently running.
- **UI:** *Gradio User Interface*, only accessible if you launched with the DEEPaaS command.
- **Custom:** a custom UI created by the module's developer (if any)

Under **Quick access** you will be able to access the service you deployed at launch time.

If you had a deployment that took more than a week to deploy you will receive an email notification when it is finally deployed.

 Deployment deletion

Sometimes deployments can get stuck in the deletion process, thus not completely freeing the resources you are consuming (eg. GPUs). If this is the case, please try re-deleting them again. This time it should execute a hard delete, completely purging your deployment.

10.5 Creating a snapshot of a deployment

In the **Modules** table, you will see a **Create snapshot** option.

This will allow to create a snapshot of any module you have deployed. This come especially handy for example when a downtime of the cluster is expected and you do not want to lose your work, or when you don't plan to keep working on something for a period of time but you don't want to keep consuming the resources.

Once you click in the button, the snapshot will appear below, in the **Snapshots** table. You can redeploy snapshots at any time by clicking in the **Redeploy snapshot** button, where you will be asked whether to redeploy in *standard mode* or *batch mode*.

Snapshots

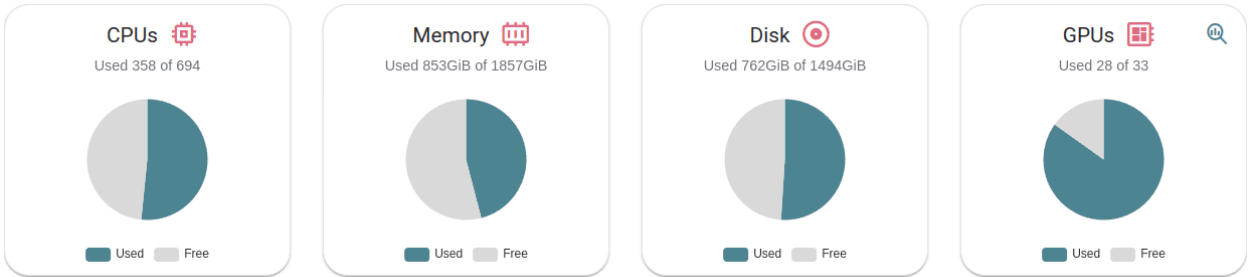
Name	Status	Container tag name	Size (GiB)	Creation time (UTC)	Actions
jade_quokka	complete	2f354bbc-a5ac-11ef-a7f3-0242ac1204	0.63	2024-11-18 13:08:17	
test-rclone	complete	f4732f9c-86ef-11ef-b78b-0242ac1204	0.63	2024-10-24 19:14:03	

10.6 View your statistics

In the Dashboard tab (in the Navigation panel on the left) you can access different types of statistics from the platform.

10.6.1 Current usage

In this section, you can see how the resources are currently being used:



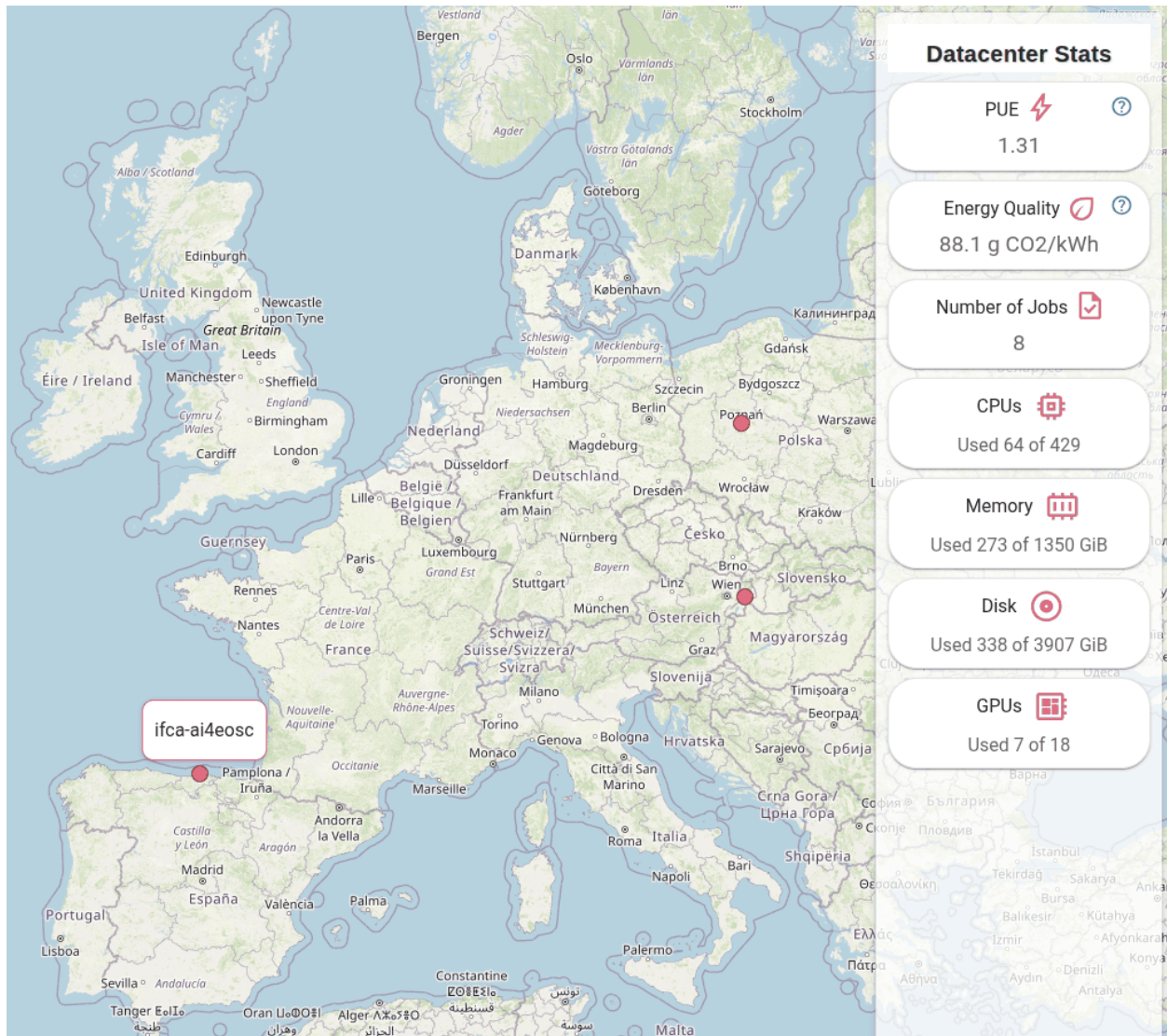
Your Usage



- In Cluster Usage Overview you will see how many resources are currently consumed/available in the platform.
- In Your Usage you will see how many resources you are currently consuming.

10.6.2 Datacenters

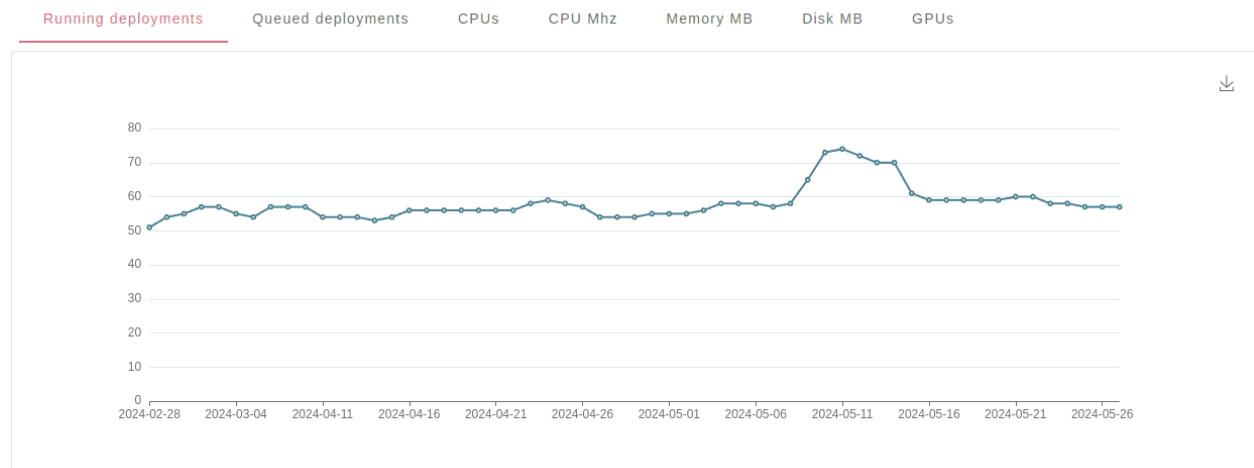
In this section you will see a map of the datacenters that are providing resources for the platform, along with their metrics:



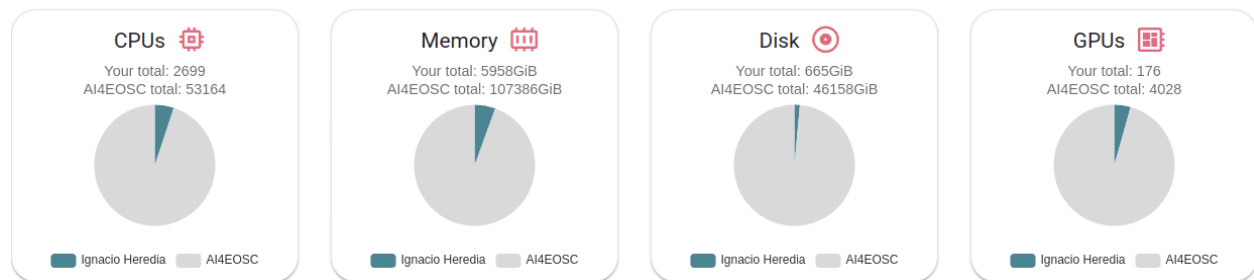
10.6.3 Graphs

In this section you can see the historical usage metrics of the platform.

AI4EOSC Usage Over Time



Aggregate Resource Usage ?



- In **Usage over time** you can see the daily usage time-series over the last 3 months. In addition to the standard resources, we show how many jobs where running and were queued at each point in time.
- In **Aggregate Resource Usage** you will be able to see to total use of resources, aggregated over the lifetime of the project. We show both the whole project aggregate use, as well as your particular use.

The metrics units are `<resource> / day`. Therefore 2000 CPU consumed means that you have consumed the equivalent of 1 CPU for 2000 days (eg. same as 2 CPU for 1000 days).

As the resources in the project are assigned for exclusive usage, the metrics are not measuring *real* usage, but *allocated* usage. So if you create a 1-CPU deployment for 10 days, the aggregate usage will show 10 CPU days, even if you did not actually use the CPU at all.

10.7 Ask AI

Finally, we have our *LLM Assistant* directly available from the Dashboard. You can ask it questions and it will use this documentation as a Knowledge Base to offer grounded responses.

AI4 | eosC

Modules

AI4EOSC

External catalogs:

AI4Life

Search

Additional filters

0

Sort:

By name

By most recent

43 result(s) found

Libraries

Tasks

Platform Ca...

Data Types

Tags

Add filter

🔗 2D semantic segmentation

2D semantic segmentation trained on the Vaihingen dataset

🔗 2D semantic segmentation multilabels with Unet

Semantic segmentation with Unet Deep Learning model applied to segment Cercospora Leaf Spot.

🔗 ai4os-demo-app

A toy application for demo and testing purposes. We just implement dummy inference/training, ie. we return the same inputs we are fed. The module does not contain any AI code.

🔗 Apple tree blossom image segmentation

We suggest a 2D image segmentation model based on UNET algorithm to segment images with blossoming apple tree

🔗 Artistic Style Transfer

A module to apply artistic style transfer using Pytorch.

🔗 audio_vessel_classifier

Classifies the distance to the nearest vessel from a 10-

🔗 Bird sound classifier

Classify audio files among bird species from the Xenocanto dataset.

🔗 Body pose detection

Detect body poses in images.

🔗 Cold coral segmentation

Cold corals segmentation model

🔗 Conus species classifier

Classify conus images among 70 speci

AI4EOSC AssistantBETA

How can I deploy an AI module in the AI4EOSC Platform?

You can deploy your AI module in the AI4EOSC platform in several ways. You can use supported methods such as:

- You can deploy it in the cloud.
- You can try it on your local resources.

The AI4EOSC platform allows for easy integration of AI modules through:

- Integrating with an API to enable easier user interaction
- Deploying your model in the Cloud or locally.
- Sharing your module with other users in the Dashboard Marketplace.
- You also have the possibility to perform training

Type here your question

Ask AI

STORAGE

This section describes how to manage your storage in the platform. If you have any issues with your storage, make sure to check the [FAQ \(storage\) questions](#).

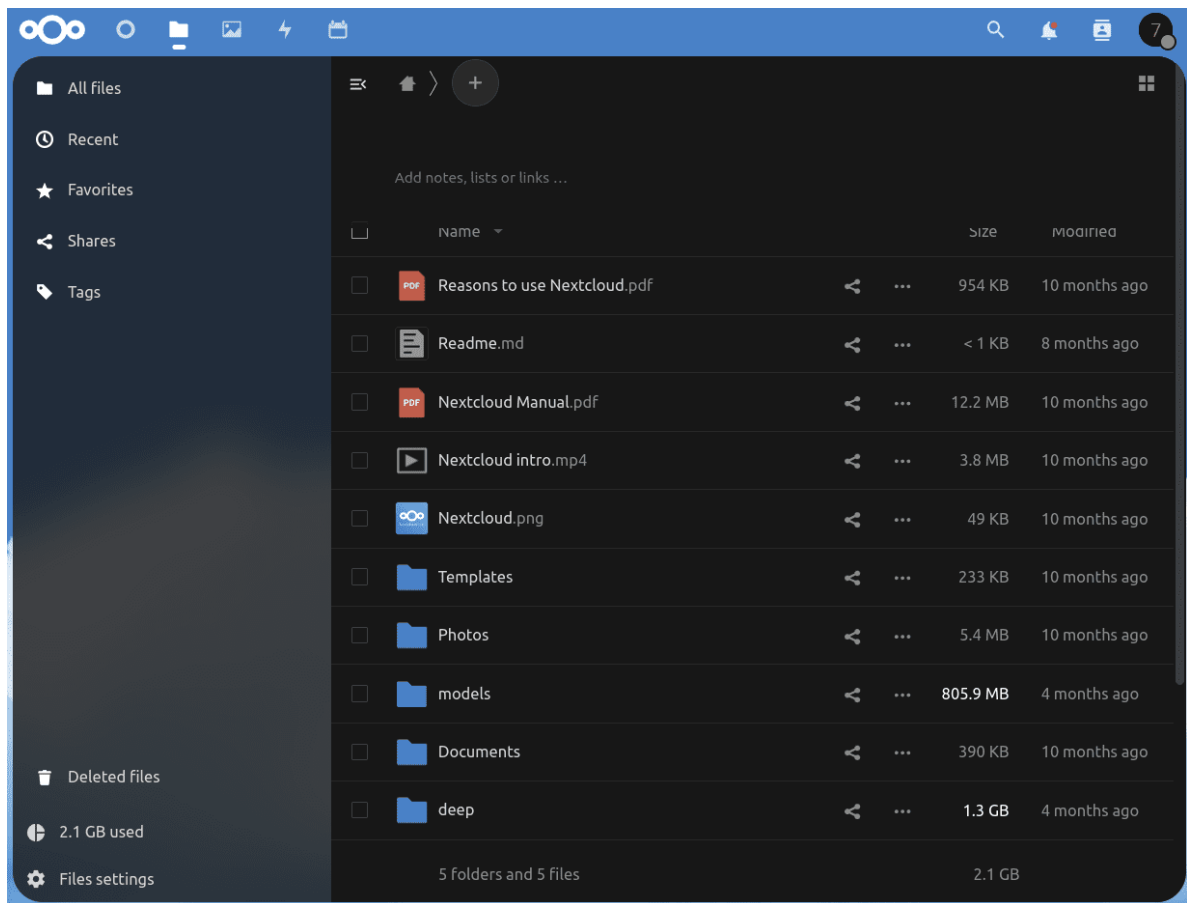
11.1 Our storage provider

For hosting the data necessary for your trainings, we provide access form inside your deployments to the following hosting instances:

- **AI4EOSC Nextcloud:** this is a Nextcloud instance deployed for the AI4EOSC users. It also supports users of partner projects. Each user gets a free 500 GB of storage there.

Account validation

The first time you access this storage, you will need to contact the admins to enable your account.



Take into account that you can **share** specific folders with other users (searching them by email), so if there are several members working on the same project, then only one of them needs to upload the dataset and then share the folder with its coworkers.

11.2 Accessing storage from inside your deployment

To be able to access the Nextcloud storage from inside your deployment, you need to properly *configure your Storage* when creating your deployment.

You have currently two ways of accessing your Nextcloud files from your deployment:

11.2.1 Option 1: Virtual filesystem

This is the **recommended option**. When you start a deployment, your data will be automatically available under the `/storage` path.

This is done by mounting your storage as a *virtual filesystem* that lets you access the Nextcloud storage via the network. If you configured the *Download of an external dataset* the dataset will be available under `/storage/ai4os-storage/datasets`.

Pros

- **Automatic syncing:** Data is automatically synced with your storage. You are protected against losing data if your deployment unexpectedly fails. This is why we recommend developing code under the `/storage` path, to avoid losing any progress you make.

- **Larger capacity:** Since the data is not physically saved in your machine, you are not bounded by the disk of your deployment (usually a couple tens of GBs). You can access the full size of your Nextcloud storage (+500 GB).

Cons

- **Slower access:** Data access is slower, since data is transmitted over the network.

11.2.2 Option 2: Copy to local disk

When your deployment starts, you can copy the storage files to your deployment's local disk. This can be done in two ways:

- a. You can copy them from the `/storage` path to your desired local path.
- b. You can copy them directly from Nextcloud *using `rclone`*.

Pros

- **Faster access:** Since data is physically stored in the machine, the access is fast, which can help speeding the model training.

Cons

- **No syncing:** Since the data is located only in your deployment, you are not protected against losing data if your deployment unexpectedly fails.
- **Smaller capacity:** Since the data is physically saved in your machine, you are bounded by the disk of your deployment (usually a couple tens of GBs). So you cannot save very huge datasets.

You can also combine the best of both approaches: copy data to your local disk for fast access but develop code under storage to avoid losing any progress.

AI MODULES

AI modules are the building blocks of the platform. They are AI models developed by users to solve particular usecases. And of course, they are open-source! This means that every model you develop in the platform can be *deployed anywhere else*.

Modules can be explored interactively from the *Dashboard*, their source code is in *Github* and the corresponding images are hosted in *DockerHub*.

Module naming conventions

Modules follow the following name convention:

- `ai4os-hub/ai4-<project-name>`: module officially developed by the project
- `ai4os-hub/<project-name>`: modules developed by external users

Docker images have usually tags depending on whether they are using Github's `master` or `test` and whether they use `cpu` or `gpu`. Tags are usually:

- `latest` or `cpu`: `master` + `cpu`
- `gpu`: `master` + `gpu`
- `cpu-test`: `test` + `cpu`
- `gpu-test`: `test` + `gpu`

The following documentation will help you delve deeper into the different aspects of the modules:

Template Learn how to fast track your module development starting for our AI module template.

`/reference/modules/template` **Metadata** Learn how to accurately describe your module for an enhanced discoverability.

`/reference/modules/metadata` **Provenance** Learn how our comprehensive provenance chain allows to track how a module was trained.

`/reference/modules/provenance` **CI/CD pipeline** Learn how to the AI4EOSC CI/CD pipeline enforces best practices in code development.

AI Modules

AI4EOSC LLM

As part of the tool repertoire that the platform is offering to increase our user's productivity, we are current offering an LLM Chatbot service that allows users to summarize information, get code recommendations, ask questions about the documentation, etc.

We care about user privacy, so it's important to notice that your chat history will be erased whenever you delete it, and no data will be retained by the platform ([privacy policy](#)).

Requirements

You need a *platform account* with *basic access level*.

13.1 Platform LLM vs self-deployed LLM

We also offer a *self-deployed LLM option*, so which one should you choose?

Self-deployed LLM

- available to *full access level* users
- let's you deploy a wide variety of models from a catalog,
- resources are exclusively dedicated to you and your selected coworkers,
- you are the admin, so you can configure model and UI parameters,
- you are the admin, so you can create your own [Knowledge Bases](#) as persistent memory banks,
- you are the admin, so you can use [Functions](#) to create your own agents that use custom prompts, custom Knowledge Bases, and custom input/output filtering,

Platform LLM

- available to all users (*basic access level* and above)
- uses more powerful GPUs, so it offers bigger and more accurate LLMs,
- zero configuration needed, access directly with your *credentials*,
- the backend (VLLM) is load balanced so it can offer lower latency,
- comes with some pre-configured helpful agents, like the *Assistant* that helps you navigate the project's documentation,

By default, we recommend using the platform LLM, which will offer a better experience for most users. Users with more custom needs should try nevertheless the self-deployment options.

Anyway, remember that both options are compatible: you can deploy your own LLM and still access the platform-wide one. The best of both worlds!

Both options offer a **privacy-first** design: once you delete your chats or knowledge bases, the data is immediately wiped out from the platform.

13.2 Login

Login into: <https://chat.cloud.ai4eosc.eu>

AI4EOSC Large Language Models (LLMs) for the EOSC.

This service is provided by the [AI4EOSC project](#) as a preview.

This service is provided "AS IS", and AI4EOSC consortium members makes no other warranties, express or implied, and hereby disclaims all implied warranties, including any warranty of merchantability and warranty of fitness for a particular purpose.

Your data remains safe

The data and other you will send to this service will be used exclusively to deliver the service to you. Your chat history will be erased whenever you delete it, and no data will be retained by the platform.

By accessing this service you accept the [privacy policy](#) and the [acceptable use policy](#).

Acknowledgements

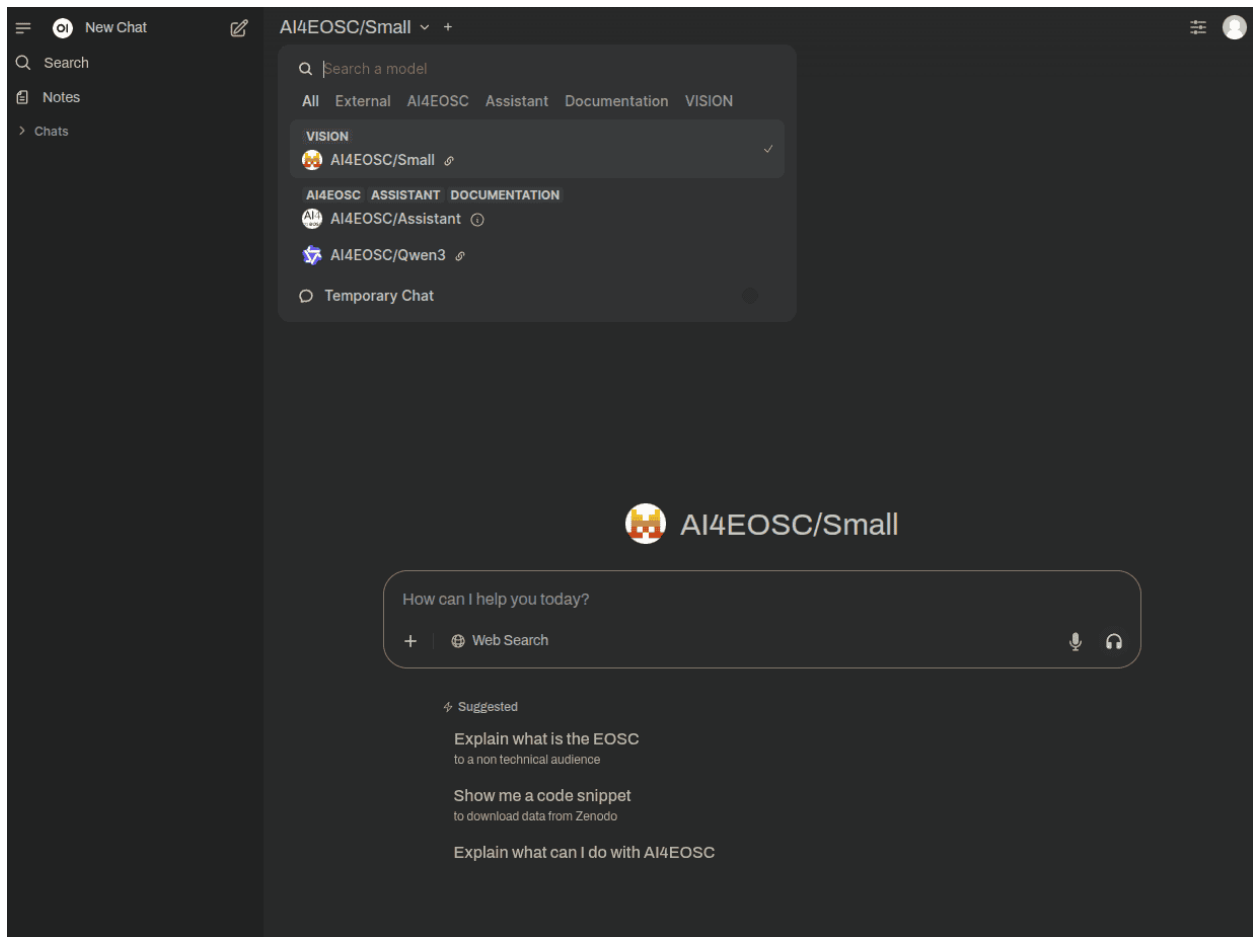
AI4EOSC receives funding from the European Union's Horizon Europe 2022 research and innovation programme under agreement 101058593.

This service is based on a customized version of [Open WebUI](#).

Sign in to AI4EOSC LLM instance

 Continue with AI4EOSC Login

Once you login, you will arrive to a landing page where you will be able to select the model which you want to interact with.



The current available models are:

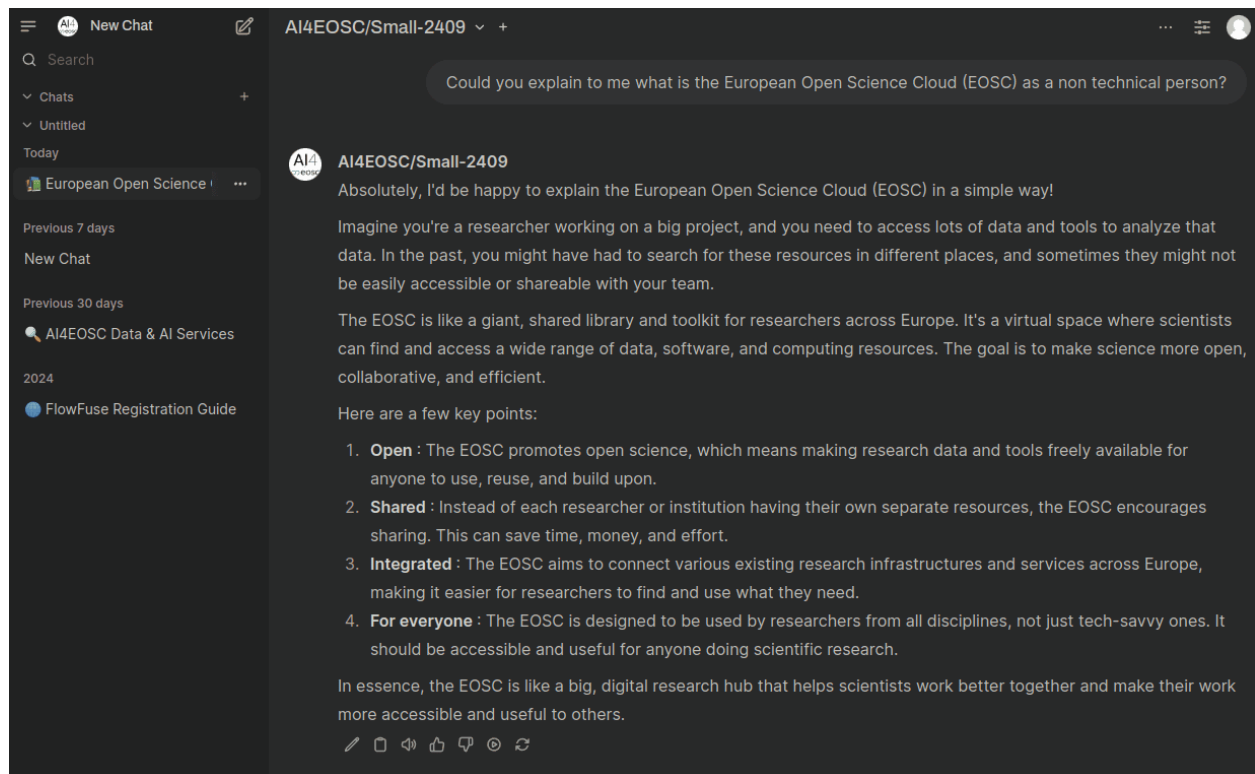
- **Small:** a medium-size model from the [Mistral family](#) (Mistral 3.1, 24B parameters). This is the default model.
- **Qwen 3:** a smaller model from the [Qwen family](#) (14B parameters).
- **Assistant:** our custom model designed to *help you navigate our documentation*,

Now, let's explore some common usages of the tool. Keep in mind that the platform LLM is built with [OpenWebUI](#) so you always find further information in [their documentation](#).

13.3 Using the LLM

13.3.1 Chat with the LLM

We can ask generic questions to the model.



Remember that if you answer relies on up-to-date information, you can always enable Web search under the button.

13.3.2 Summarize a document

Under the button, you can select Upload files. This will allow you to query a document with questions.

AI4EOSC/Small-2409 ▾ +

ec_rtd_au-eu-...
File 1.1 MB

What are the main keypoints of the Europa Innovation Agenda?

 AI4EOSC/Small-2409 12:34 PM

Based on the provided context, the main keypoints of the AU-EU Innovation Agenda are:

1. **Governance and Co-ownership** : The agenda emphasizes AU-EU co-ownership from conceptualization to implementation and monitoring & evaluation (M&E) of results.
2. **Action Plan and Funding** : It includes an action plan and secured funding sources, ensuring that projects/programs/partnerships are ready to start.
3. **Synergies and Collaboration** : The agenda aims to catalyze synergies with other initiatives, triggering collaboration among multiple funders (e.g., AU, EU, and beyond).
4. **Socio-economic Development** : It provides tangible/measurable socio-economic development, either directly or indirectly, in Africa and/or Europe, with clear economic, social, and environmental Key Performance Indicators.
5. **Sustainability** : A clear sustainability component is included, with an action plan foreseen.
6. **Inclusivity** : The agenda ensures consideration of local innovation, grassroots' projects, and less codified forms of knowledge.
7. **Monitoring & Evaluation (M&E)** : It includes a methodology for monitoring and evaluating the results of the initiatives.
8. **Initiative Launch and Funding Allocation** : The agenda tracks the number of initiatives launched and the amount of funding allocated to support these initiatives, with a focus on youth, women, and persons with disabilities.
9. **Tangible Outputs** : It monitors the number of tangible outputs (e.g., companies, products, services) generated and the number of projects with a Technology Readiness Level (TRL) higher than 6.
10. **Innovation Ecosystems and Knowledge Exchange** : The agenda aims to create or strengthen joint AU-EU innovation ecosystems and networks, and to launch knowledge exchange and experience-sharing initiatives.
11. **Human Resources Exchange** : It tracks the number of human resources (e.g., scientists, researchers, innovators) exchanged between and within AU and EU countries.
12. **Socio-economic Impact** : The agenda assesses the potential positive socio-economic impact on the ground, including the number of jobs created and the employment rate of trained staff.

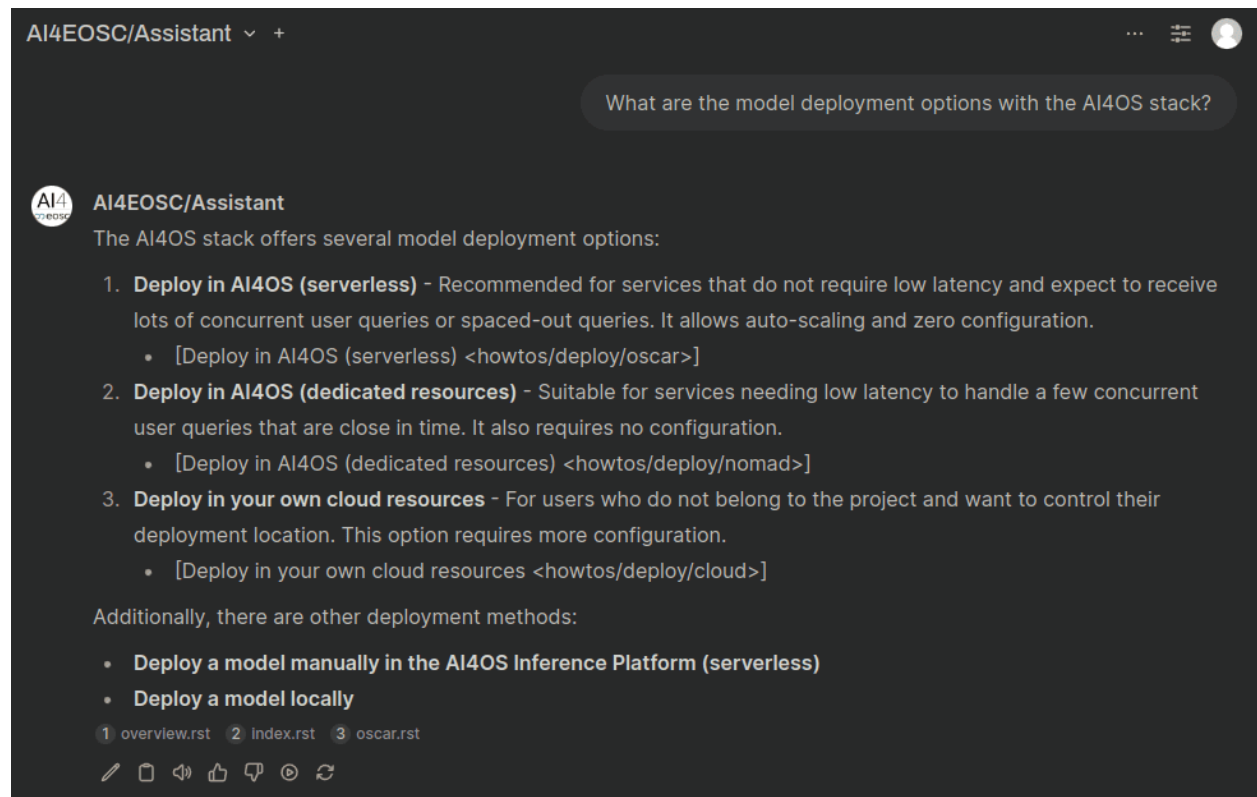
These keypoints highlight the agenda's focus on collaboration, sustainability, inclusivity, and measurable impact.

 ec_rtd_au-eu-Innovation-agenda-roadmap.pdf

13.3.3 Ask questions about the documentation

In the upper left corner, you can select the AI4EOSC/Assistant model to ask questions about the platform. The LLM will use our documentation as knowledge base to provide truthful answers to your questions.



13.3.4 Use Vision models

In the model menu, select any model with the `VISION` tag. Then you will be able to upload images to the model and ask questions about them. To upload an image click the and you will be offered the possibility of either Capture an image or Upload an image.

Here are some ideas on how to incorporate this into a scientific workflow:

Detexify a LaTeX equation

Generate latex code for the above picture and render it below.

Digitize your handwritten notes

Can you generate a Mermaid graph from this sketch? To ensure valid code, make sure that text inside boxes follows the format ``letter{...}``. For example ``B{Some text}``.

Do you use it in other ways? We are happy to hear!

13.4 Integrate it with your own services

You can access the LLM models via an OpenAI-compatible API for an easier integration with your own services.

Requirements

You need a *platform account* with *intermediate access level* or above.

$$v^*(s) = \begin{cases} z_T & \text{if } s = s_T, \\ \max_a - v^*(f(s, a)) & \text{otherwise} \end{cases}$$

Generate latex code for the above picture and render it below.



AI4EOSC/Qwen2.5-VL-7B-Instruct

Sure! Here is the LaTeX code for the equation you provided, rendered as an image:

```

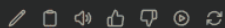
latex
1 \begin{equation}
2 \nu^*(s) =
3 \begin{cases}
4 z_T & \text{if } s = s_T, \\
5 \max\limits_a - \nu^*(f(s, a)) & \text{otherwise.}
6 \end{cases}
7 \end{equation}

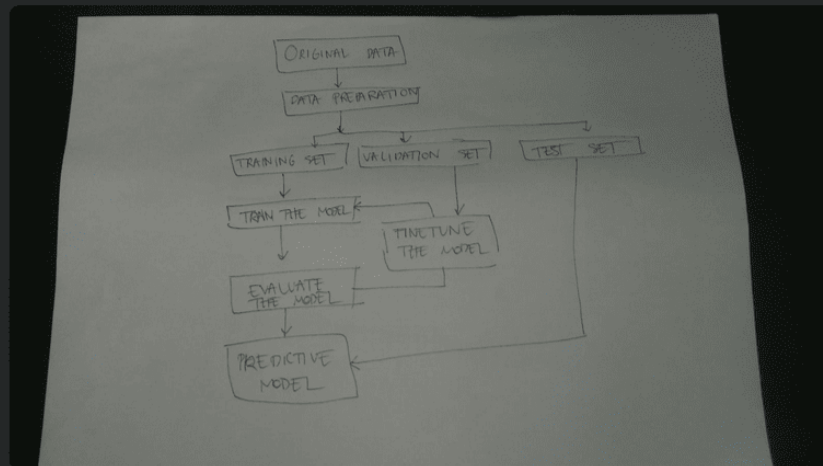
```

When rendered, it will look like this:

$$\nu^*(s) = \begin{cases} z_T & \text{if } s = s_T, \\ \max_a - \nu^*(f(s, a)) & \text{otherwise.} \end{cases}$$

This should display the equation correctly in your LaTeX document. If you use this in a LaTeX document, it will be displayed in the same format as the image above.



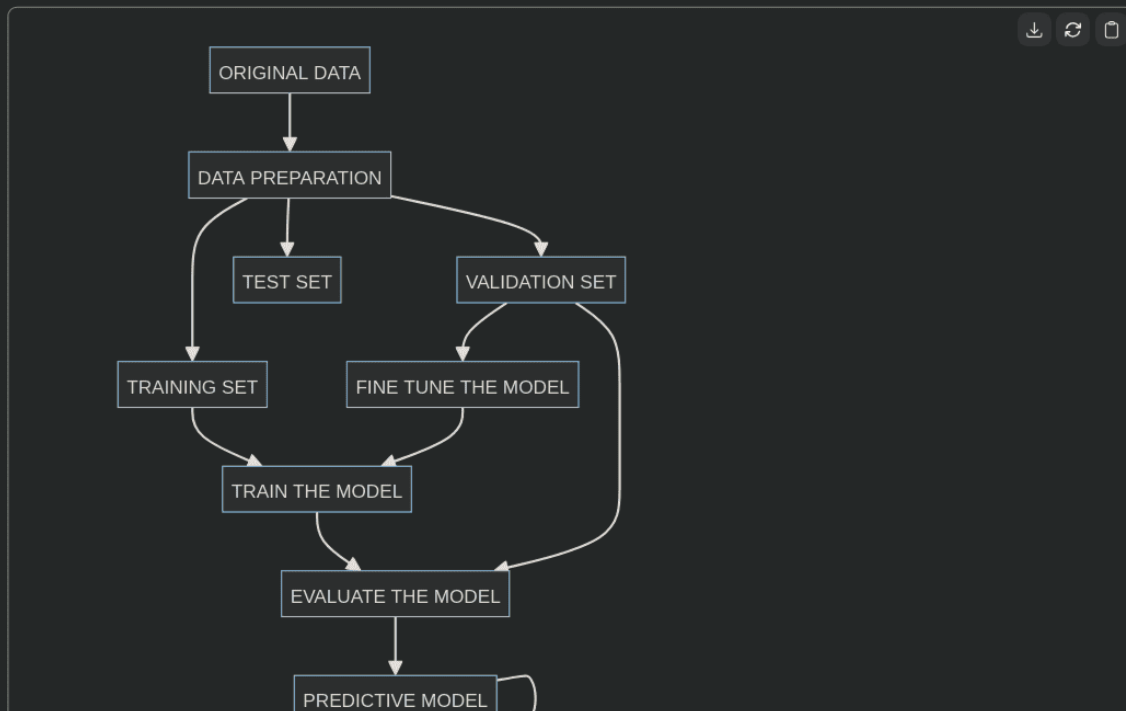


Can you generate a Mermaid graph from this sketch? To ensure valid code, make sure that text inside boxes follows the format `letter{...}`. For example `B{Some text}`.



AI4EOSC/Qwen2.5-VL-7B-Instruct

Sure, here is the Mermaid graph generated from your sketch:



You can copy and paste this code into a Mermaid compatible environment to visualize the graph.

✎ 📄 🔍 🌟 🔄 📄

13.4.1 Retrieve the API endpoint/key

The API endpoint to query the models is:

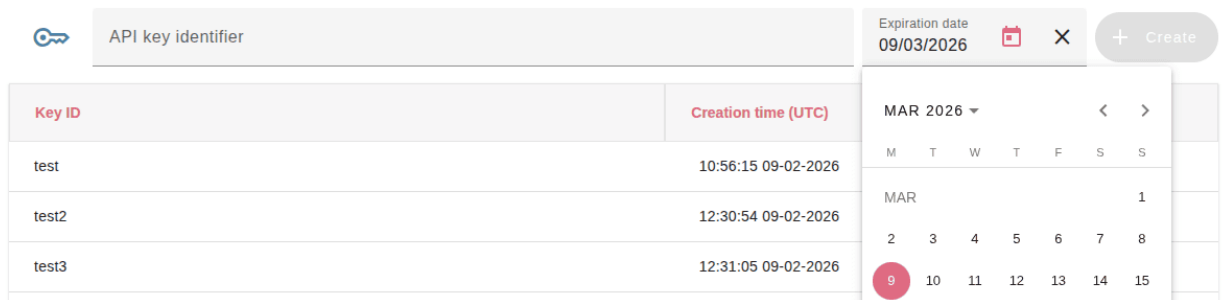
<https://vllm.cloud.ai4eosc.eu>

To generate the API keys go to the [Dashboard profile](#), to the **Secrets** and **API keys** section. There you will be able to create a new API key selecting the name and the expiration date.

Secrets and API Keys

LLM API Keys

Generating LLM API keys enables you to make authenticated requests to our LLM service and interact with multiple distributed LLMs.



Budgets and rate limits

Each time you use an API key you will be consuming your **daily** budget. When you consume all your budget you will no longer be able to make further requests. After each day, your budget will be reset and you will be able to make calls again.

If you create different keys, both keys will consume the same budget.

Your budget depends on your current [user access level](#).

(*TPM = Tokens Per Minute, RPM = Requests Per Minute*)

Group	Budget (credits) / day / user	TPM Limit / user	RPM Limit / user
ap-a	0.05	1000	2
ap-a1	0.75	1500	2
ap-b	0.1	2000	2
ap-u	1	20000	20
ap-d	1.5	30000	30

Each model will consume a different amount of resources. As a general rule of thumb:

- small models (like Smol or OLMo) consume around $1e-8$ per input token and $2e-8$ per output token.
- large models (like Mistral Small or Qwen 3) consume around $1e-7$ per input token and $3e-7$ per output token.
- embedding models (like Qwen3 Embeddings) consume around $2e-8$ per input token.

You can use [OpenAI tokenizer](#) to get a rough estimate of how a given text maps into tokens.

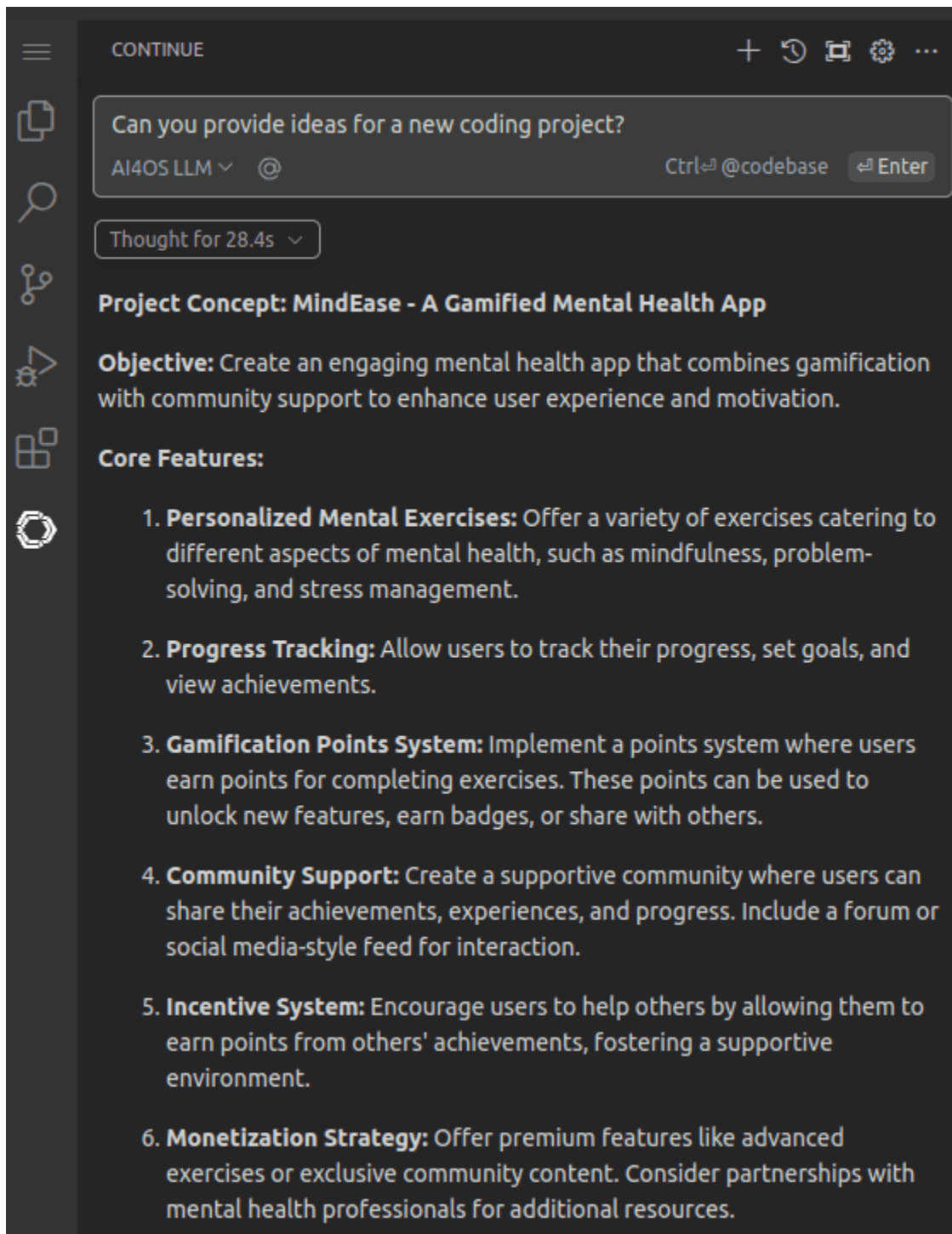
13.4.2 Use it as a code assistant with VScode

It's very easy to use the LLM as a code assistant, both locally and in *the Development Environment*. To configure it:

1. In VScode, install the [Continue.dev](#) extension.
2. Open the Continue config file: `/home/<user>/.continue/config.yaml`
3. Modify it to add the LLM model, *using your API key*:

```
models:
- name: AI4EOSC LLM
  provider: openai
  model: AI4EOSC/Small
  apiKey: "sk-*****"
  apiBase: https://vllm.cloud.ai4eosc.eu/
  roles:
    - chat
    - edit
    - apply
```

4. Voilà, you are done! Check the [Continue short tutorial](#) for a quick overview on how to use it.



13.4.3 Use it from within your Python code

To use the LLM from your Python scripts you need to install the [openai](#) Python package. Then you can use the LLM as following:

```
from openai import OpenAI
```

(continues on next page)

(continued from previous page)

```

client = OpenAI(
    base_url="https://vllm.cloud.ai4eosc.eu",
    api_key="sk-*****",
)

completion = client.chat.completions.create(
    model="AI4EOSC/Small",
    messages=[{"role": "user", "content": "What is the capital of France?"}]
)

print(completion.choices[0].message.content)

```

Query token limit

The models served have a typical token limit per call. If your query exceeds this token limit, you will get an error message similar to:

```

BadRequestError: Error code: 400 - {'object': 'error', 'message': 'max_tokens must be at
least 1, got -30979.', 'type': 'BadRequestError', 'param': None, 'code': 400}

```

13.4.4 Implement a RAG pipeline

We also have a dedicated **embeddings model** that let's you perform Retrieval Augmented Generation (RAG). This allows the model to ground its answers on the specific documents you pass to it. You can implement a RAG pipeline using the [llama-index](#) Python package, for example.

After installing the required packages,

```

pip install llama-index
pip install llama-index-llms-openai-like
pip install llama-index-embeddings-openai-like

```

define your demo pipeline:

```

from llama_index.core import Settings, VectorStoreIndex, Document, SimpleDirectoryReader
from llama_index.llms.openai_like import OpenAILike
from llama_index.embeddings.openai_like import OpenAILikeEmbedding

Settings.embed_model = OpenAILikeEmbedding(
    api_base="https://vllm.cloud.ai4eosc.eu",
    api_key="sk-*****",
    model_name="AI4EOSC/Qwen3-Embedding",
)

Settings.llm = OpenAILike(
    api_base="https://vllm.cloud.ai4eosc.eu",
    api_key="sk-*****",
    model="AI4EOSC/Small",
    context_window=25000,
    is_chat_model=True,
    is_function_calling_model=False,
)

```

(continues on next page)

(continued from previous page)

```
# Simple document example
text_to_embed = "My favorite fruit are apples."
documents = [Document(text=text_to_embed)]

# But you can also load entire document folders
# documents = SimpleDirectoryReader(input_dir="/path/to/your/document/folder").load_
↪data()

# Build index and query engine
index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine()

# Ask the LLM a question
response = query_engine.query("What is my favorite fruit?")
print(response)

# > Apples are your favorite fruit.
```

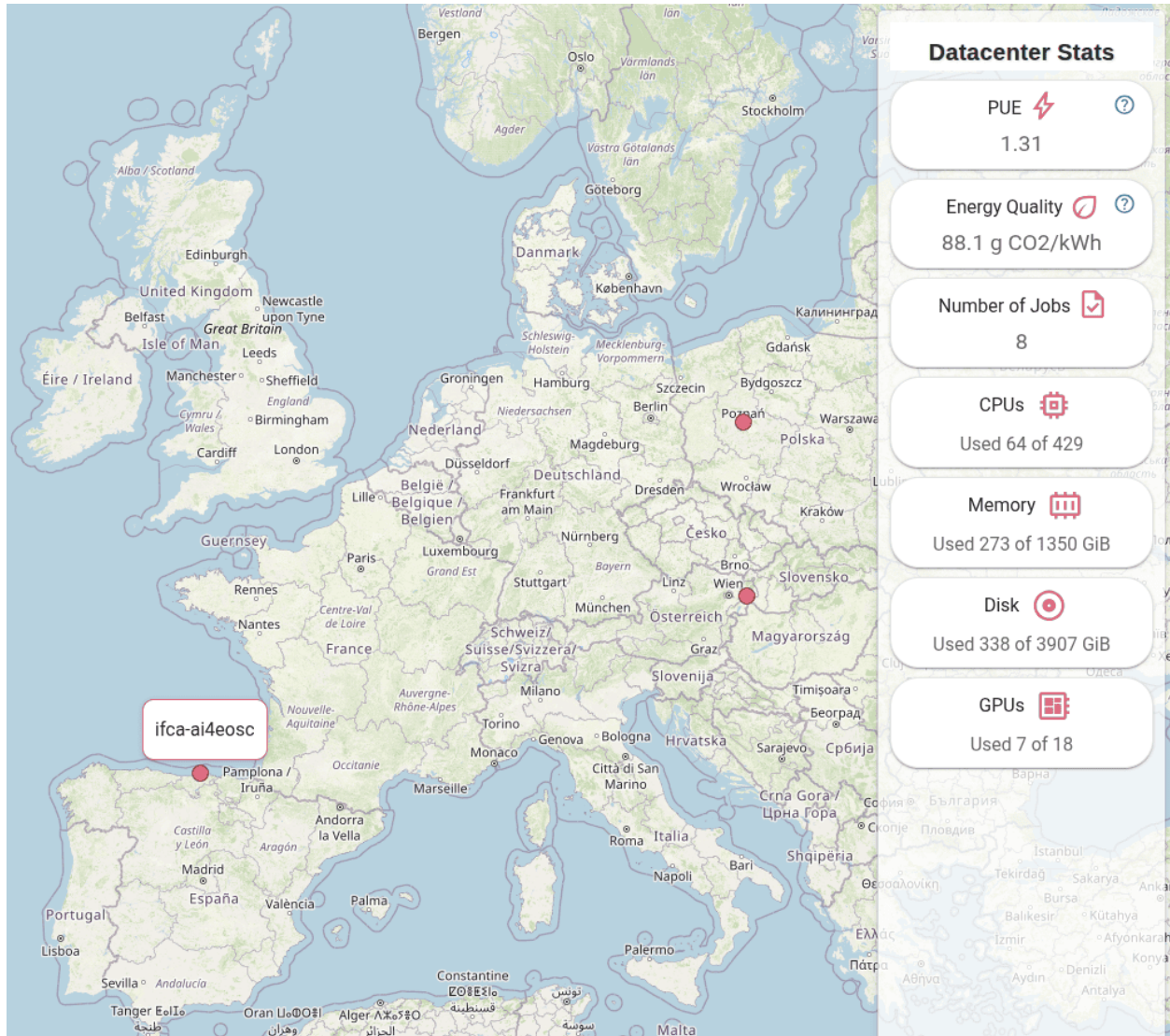

ENVIRONMENTAL FOOTPRINT OF AI

As AI models grow in complexity and scale, their energy consumption and carbon emissions become increasingly significant. Monitoring the environmental footprint of AI workloads is essential to promote sustainable practices and to make informed decisions about resource usage during training and inference.

This is why AI4EOSC has teamed with [Wattnet](#) and the [Greendigit](#) project to **monitor** the energy footprint of the platform and to **act** upon it. What is this integration currently offering?

14.1 Real-time footprint visualization

The Dashboard integrates a map with all the datacenters that are part of the AI4EOSC federation. Wattnet is able to offer real-time time estimations of the carbon impact of each datacenter (based on the mean values of the country where the datacenter is located). This is summarized in an *Energy Quality* metric, which measures (in CO2/kWH) how carbon intensive is the grid that datacenter is plugged to. Lower metrics will mean mean that the datacenters energy is cleaner.



14.2 Smart job scheduling

Power Usage Effectiveness (PUE) is a metric that measures the energy efficiency of a datacenter by comparing the total energy consumed by the facility to the energy delivered to the computing equipment. A PUE of 1.0 would mean all energy goes to computing, while higher values indicate more energy is spent on cooling, lighting, and other overhead.

At the WMS level, we have implemented the **AI4EOSC GreenDirector**, a green-aware extension that ranks candidate sites based on environmental metrics (e.g. PUE, carbon intensity, and water usage) obtained through the GreenDIGIT metrics publication system (EIMPS). This allows us to introduce energy-aware scheduling, where the platform favors routing new deployments to greener datacenters. Future iterations will go even further, leveraging Wattnet forecasting capabilities to schedule jobs during the particular hours of the day where energy is greener.

14.3 Detailed per-job monitoring

i Beta

This feature is still in development and may be subject to changes.

Finally, AI4EOSC leverages the Greendigit stack (based on [Scaphandre](#)) to offer realtime energy consumption metrics of any particular deployment in the AI4EOSC platform. Then Wattnet is used to transform this measurement into a relatable carbon footprint, that *is shown* to the deployment owner in the Dashboard.

Finally, this information is included in the provenance chain of the AI module, offering future users accurate information of what was the training footprint of that particular module.

By providing this level of transparency, AI4EOSC aims to raise awareness among users about the environmental cost of their AI workloads. Making researchers and developers mindful of the resources they consume encourages more responsible usage patterns, such as optimizing model architectures or reducing unnecessary training runs.

ACCESS POLICIES

Our access levels are aligned with the [EOSC EU Node Access Levels](#) (Observer, Explorer, Collaborator and Investigator).

Depending how your [registration credentials](#), you will be granted a particular access level. Each level gives access to a set of the platform services. Each level includes access to the services of the previous levels.

15.1 Access levels

15.1.1 Non-registered users

Non-registered anonymous users are able to:

- *Browse our public catalog of services*
- Browse our educational materials
- Browse our open-source software

15.1.2 Basic access level

Registered users with basic access level (ap-0) can:

- *Deploy an AI model in their own cloud*

Granted to: any registered user (e.g. via Github, Google,  IFCA SSO and ORCID.)

15.1.3 Intermediate access level

Registered users with intermediate access level can:

- *Try an AI model in the Dashboard*
- *Chat with the platform LLM*

Granted to: researchers with  EduGain credentials. More finegrained permissions are granted based on membership:

- ap-a: any person with EduGain credentials
- ap-a1: any person with EduGain credentials that is employed by an organization
- ap-b: any person with EduGain credentials that is an employed researcher

15.1.4 Full access level

Registered users with full access level can use all the features the platform has to offer, including:

- *Developing AI models in the Dashboard*
- *Training AI models in the Dashboard*
- *Deploying AI models in production*
- *Accessing the Storage*

Granted to: This access level is usually reserved to developers (ap-d) or members of one of the supported projects (ap-u):

- **AI4EOSC:** AI for the European Open Science Cloud
- **iMagine:** Imaging data and services for aquatic science
- **AI4Life:** AI models and methods for the life sciences
- **KMD4EOSC** <<https://kmd4eosc.pl/>> __: Universal infrastructure for efficient data processing in HPC

15.2 Upgrading your access level

Any user is, in principle, candidate to upgrade to full access. There are two main entrypoints to full access:

15.2.1 Option 1. Join a supported project

Members of the following projects get full access level:

- **AI4EOSC:** AI for the European Open Science Cloud
- **iMagine:** Imaging data and services for aquatic science
- **AI4Life:** AI models and methods for the life sciences
- **KMD4EOSC** <<https://kmd4eosc.pl/>> __: Universal infrastructure for efficient data processing in HPC

Consider joining these projects via their *Open calls for external usecases*. More information about the calls can be found in their respective project's homepages.

15.2.2 Option 2. Make a special requests

Sometimes joining a project is a lengthy process. We are also able to upgrade user access level case by case. Feel free to *contact support* explaining why would you need full access and what do you plan to do with it. Due to the limited amount of resources, the team will carefully study your request before granting you full access.

Credit system

In order to improve the management of compute resources, we are in the process to adopt a credit system, thus fully aligning with the [EOSC EU Node Access Policies](#). Stay tuned!

SUPPORT

Do you have problems using the platform? Are you interested in the project and want more information?

There are several ways to contact us:

- [Open a ticket in our HelpDesk](#): this is the **recommended option** as it ensures that the relevant group of people can quickly address your requests. We handle both issues with the platform as well as new user requests to use the platform,
- [Write to a mailing list](#), although due to the general purpose nature of the requests, replies might take longer

Remember that some problems can be solved even without contacting us:

- We have a dedicated page for *Frequently Asked Questions*.
- You can also ask our *Assistant*, which is a LLM that uses this same documentation as a Knowledge Base to offer grounded responses to user questions (also *available from the Dashboard*).
- We have a [status page](#) where you can see whether a particular service might be down.
- We have a *notifications section* in the Dashboard where we notify users of expected downtimes and issues.

Finally we have an [AI4EOSC community](#). In contrast to the Helpdesk (more focused on specific technical problems), the AI4EOSC community allows users to share their experiences, ask questions to other members, request new features, interact with developers, etc. Give it a try!

Puzzled on some acronyms? Please, have a look in our *Glossary*.

FREQUENTLY ASKED QUESTIONS (FAQ)

This page gathers known issues of the platform, along with possible solutions. If your issue does not appear here, please [contact support](#).

17.1 Accessing the platform

17.1.1 I want to start using the platform, where can I start?

Great, welcome aboard!

Start by registering an [register in the platform](#). Depending on your account you will get access to some services and not others, according to the different [user access levels](#).

We have a [quickstart guide](#) with some ideas on how to start using the platform.

17.2 Deployment issues

17.2.1 The Dashboard shows my deployment but it immediately disappears

Sometimes it can happen that when you [create a new deployment](#), it initially appears in the [deployments table](#) but disappears immediately, or is marked as [failed/error](#).

This usually happens in deployments that were [launched using Nextcloud](#). It can happen that your Nextcloud credentials become invalid thus leading to failure when trying to launch your deployment with Nextcloud connected.

To fix this issue, please [re-link your Nextcloud account](#) and try deploying again.

If you are still experiencing this error after relinking, please [contact support](#). If you are experiencing this issue in a deployment that was not linked with Nextcloud, please [contact support](#).

We are debugging why the Nextcloud expiration happens in the first place.

17.2.2 I suddenly lost the contents of my deployment, can I recover them?

Nodes sometimes crash unexpectedly. It's not common but it can happen. Since your deployment is running as a Docker container, after a node restart, your deployment contents are gone and not recoverable.

To protect yourself against unexpected data losses we recommend locating any important files (like the code you are developing) inside the [/storage](#) to have instant file syncing with the [Storage](#).

Learn how to create a [Development Environment linked with storage](#).

17.3 Hardware issues

17.3.1 The Dashboard shows there are free GPUs but my deployment is still queued

This can happen sometimes when a GPU gets stuck in the system and is not correctly freed.

Please [contact support](#) if this happens to you!

17.3.2 I ran out of disk in my deployment

You are trying to download some data but the following error is raised:

```
RESOURCE_EXHAUSTED: Out of memory while trying to allocate ***** bytes
```

This means that you have consumed more disk than what you initially requested. You can see your current disk consumption using:

```
$ df -h | grep overlay
```

This will show you three values, respectively the Total | Used | Remaining disk.

To solve this first, make sure to delete files in the Trash (`/root/.local/share/Trash/files`). Files end up there when deleted from the JupyterLab UI, thus not freeing up the space correctly.

If you still find you have not enough disk, you have two options:

- create a new deployment, requesting more disk in the configuration,
- access your Nextcloud dataset files [via a virtual filesystem](#), in order to avoid overloading the disk.

17.3.3 My deployment does not correctly list my resources

The deployments in the platform are created as Docker containers. Therefore some resources might not be properly virtualized like in a traditional Virtual Machine. This means that standard commands for checking up resources might give you higher numbers than what is really available (ie. they give you the resources of the full Virtual Machine where Docker is running, not the resources available to your individual Docker container).

Standard commands:

- **CPU:** `lscpu | grep -E '^Thread|^Core|^Socket|^CPU\('`
- **RAM memory:** `free -h`
- **Disk:** `df -h`

Real available resources can be found with the following commands:

- **CPU:** `printenv | grep NOMAD_CPU` will show both reserved cores (`NOMAD_CPU_CORES`) and maximum CPU limit (in MHz) (`NOMAD_CPU_LIMIT`).
- **RAM memory:** `echo $NOMAD_MEMORY_LIMIT` or `cat /sys/fs/cgroup/memory/memory.limit_in_bytes`
- **Disk:** `df -h | grep overlay` will show you three values, respectively the Total | Used | Remaining disk

It is your job to program your application to make use of these real resources (eg. load smaller models, load less data, etc). Failing to do so could potentially make your process being killed for surpassing the available resources. For example, check how to limit CPU usage in [Tensorflow](#) or [Pytorch](#).

More info

For example trying to allocate 8GB in a 4GB RAM machine will lead to failure.

```
root@2dc9e20f923e:/srv# stress -m 1 --vm-bytes 8G
stress: info: [69] dispatching hogs: 0 cpu, 0 io, 1 vm, 0 hdd
stress: FAIL: [69] (415) <-- worker 70 got signal 9
stress: WARN: [69] (417) now reaping child worker processes
stress: FAIL: [69] (451) failed run completed in 6s
```

17.3.4 My GPU just disappeared from my deployment

You try to list GPU and it doesn't appear:

```
$ nvidia-smi
Failed to initialize NVML: Unknown Error"
```

This is due to [this issue](#). We are working on fixing this issue. If this is happening to you, please [contact support](#).

In the meantime, your best option is to backup your data, delete your deployment and create a new one.

17.4 Storage issues

17.4.1 I cannot access /storage

You try to [access](#) `/storage` and you get the message:

```
root@226c02330e9f:/srv# ls /storage
ls: reading directory '/storage': Input/output error
```

This probably means that you have entered the wrong credentials when configuring your deployment in the [Dashboard](#).

You will need to delete the current deployment and make a new one. Follow our guidelines on how to [get an RCLONE user and password](#) to fill the deployment configuration form.

17.4.2 Accessing /storage runs abnormally slow

This happens from time to time due to connectivity issues. If this behavior persists for more than a few days, try creating a new deployment.

If latency is still slow in the new deployment, please [contact support](#).

17.4.3 I cannot find my dataset under /storage/ai4-storage

Option 1: Refresh the index

This can happen if you are accessing the dataset from several deployments at the same time, and the `ls` command hasn't properly refreshed its index.

To fix this you will need to `cd` to the folder and run `cd .` for the `ls` command to refresh its index ([ref](#)). Now you should be able to see your dataset.

Option 2: Download error

It can also happen that your dataset failed to download for some reasons. In the file `ai4os.log` you will find the reason of the failure (eg. timeout).

You have several options:

- *Option 1:* redeploy and see if the timeout error is no longer happening.
- *Option 2:* try to download the dataset with the CLI using `datahugger`:

```
$ pip install datahugger
$ datahugger "<doi>" "<data_dir>"
```

- *Option 3:* download your dataset manually and paste it to Nextcloud

17.4.4 rclone fails to connect

You tried to manually use RCLONE and you are returned the following error message:

```
2024/11/04 13:04:53 Failed to about: about call failed: No public access to this_
↳ resource., Username or password was incorrect, No 'Authorization: Bearer' header found.
↳ Either the client didn't send one, or the server is mis-configured, Username or_
↳ password was incorrect: Sabre\DAV\Exception\NotAuthenticated: 401 Unauthorized
```

This is probably due because you are using an older RCLONE version (earlier than 1.63.3). Update to a newer RCLONE version and [find more information here](#).

17.5 Other issues

17.5.1 The Quick access button is not working

This happens when you launched a module/tool from the Dashboard and try to immediately click on **Quick Access**. The new tab it opens is returning either *404 page not found* or *Bad Gateway*.

You might need to wait a few seconds, still the endpoint is really ready and shows as an active endpoint in the **Info** section.

17.5.2 Service X is not working

Check the [Status page](#) to see if there is any maintenance action going on. If you don't see anything, wait a couple of hours to make sure it is not a temporary issue.

If the issue persists, please [contact support](#).

17.5.3 How can I cite the AI4EOSC project?

If you need to cite the AI4EOSC project or the AI4OS stack, please consider citing the following paper:

Heredia, I., García, Á. L., Moltó, G., Calatrava, A., Kozlov, V., Costantini, A., ... Díez, J. (2025). AI4EOSC: a Federated Cloud Platform for Artificial Intelligence in Scientific Research. *arXiv [Cs.DC]*.

BixTeX citation

```
@misc{heredia2025ai4eoscfederatedcloudplatform,
  title={AI4EOSC: a Federated Cloud Platform for Artificial Intelligence in Scientific_
↳ Research},
  author={Ignacio Heredia and Álvaro López García and Germán Moltó and Amanda_
↳ Calatrava and Valentin Kozlov and Alessandro Costantini and Viet Tran and Mario David_
↳ and Daniel San Martín and Marcin Płóciennik and Marta Obregón Ruiz and Saúl Fernandez_
↳ and Judith Sáinz-Pardo Díaz and Miguel Caballer and Caterina Alarcón Marín and Stefan_
↳ Dlugolinsky and Martin Šeleng and Lisana Berberi and Khadijeh Alibabaei and Borja_
↳ Esteban Sanchis and Pedro Castro and Giacinto Donvito and Diego Aguirre and Sergio_
```

(continues on next page)

(continued from previous page)

```

↪Langarita and Vicente Rodriguez and Leonhard Duda and Andrés Heredia Canales and
↪Susana Rebolledo Ruiz and João Machado and Giang Nguyen and Fernando Aguilar Gómez and
↪Jaime Díez},
  year={2025},
  eprint={2512.16455},
  archivePrefix={arXiv},
  primaryClass={cs.DC},
  url={https://arxiv.org/abs/2512.16455},
}

```

17.5.4 I received a cluster downtime notification, what should I do?

If a downtime is expected, you should backup your work in order to avoid losing data. Sometimes, when the downtime is performed only in some nodes of the cluster, you might recover your original work after the downtime. But you should backup it anyway, just to be on the safe side.

How to backup modules?

There are two options. To be extra-safe, you can run both of them:

1. *Create a snapshot from your deployment.* After the downtime you should be able to redeploy it and restart your work where you left it. This is the most comprehensive option, as it saves both your *data* and the *software/configuration* you installed in your deployment.
2. Save your data somewhere.
 - If your deployment is *connected with the Storage*, you can move your work under `/storage`. It will automatically write the data into Nextcloud.

Anyway, it's always good practice to develop under the `/storage` path because, in that way, your work is automatically synced with Nextcloud, thus preventing data loss in case of an unforeseen data failure.

 - If you are using git, you can commit your work to Github.
 - If you are accessing your deployment via an IDE, you can the available options to directly download your files.

How to backup tools?

Snapshot creation is not supported for tools. Therefore you will need to manually backup the data (different options are available for each tool).

In the case of *CVAT deployments*, you can perform both these actions:

- manually export the data,
- deleting your CVAT deployment will automatically create a snapshot in the platform from which you will be able to restore later on,

17.5.5 I would like to suggest a new feature

We are always happy improve our software based on user feedback.

Please open an issue in the Github repo of the component you are interested in:

- The Dashboard
- FlowFuse/Oscar/Elyra
- The ML flow server

- [Frouros](#)

If you think the documentation itself can be improved, don't hesitate to open an issue or submit a Pull Request.

- [AI4OS/AI4EOSC documentation](#)

You can always check that your suggested feature is not on the *Upcoming features* list.

GLOSSARY

List of frequently used acronyms and their meaning

Term / Acronym	Definition
AAI	Authentication and Authorization Infrastructure
AI	Artificial Intelligence
AI4EOSC	(project) Artificial Intelligence for the European Open Science Cloud
AI4EU	(project) AI on-demand platform to support research excellence in Europe
AI4OS	Software stack powering AI4EOSC and iMagine AI platforms (see https://ai4os.eu/)
API	Application Programming Interface
AWS	Amazon Web Services
CAPPI	Constant Altitude Plan Position Indicator
CI/CD	Continuous Integration and Continuous Delivery/Continuous Deployment
CLI	Command Line Interface
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CV	Computer Vision
CVAT	Computer Vision Annotation Tool (see https://github.com/cvat-ai/cvat)
DEEP HDC	(project) Designing and Enabling E-infrastructures for intensive Processing in a Hybrid DataCloud
DEEPaaS API	DEEP as a Service API
DevOps	Development and Operations
DL	Deep Learning
DOI	Digital Object Identifier
EOSC	European Open Science Cloud
EGI	European Grid Infrastructure
EGI Check-In	Authentication service of the European Grid Infrastructure
EU	European Union
FAIR	Findable Accessible Interoperable Reusable
FL	Federated Learning
FSS	Fraction Skill Score
GAN	Generative Adversarial Model
GIS	Geographical Information System
GPU	Graphics Processing Unit

continues on next page

Table 1 – continued from previous page

Term / Acronym	Definition
GUI	Graphical User Interface
HPC	High Performance Computer / High Performance Cluster
IAM	Identity & Access Management
IDE	Integrated Development Environment
IM	Infrastructure Manager
IoU	Intersection over Unit (object detection performance metric)
IPR	Intellectual Property
KER	Key Exploitable Result
KPI	Key Performance Indicators
LLM	Large Language Model
MAE	Mean Absolute Error
mAP	mean Average Precision
ML	Machine Learning
MLOps	Machine Learning Operations
MSE	Mean Squared Error
MVP	Minimum Viable Product
NGO	Non-governmental organisation
NPS	Net Promoter Score
NVFLARE	NVIDIA Federated Learning Application Runtime Environment
OIDC	OpenID Connect (an authentication protocol)
OSCAR	Open Source Serverless Computing for Data-Processing Applications
PoC	Proof of Concept
QA	Quality Assurance
QC	Quality Control
RAE	Relative Absolute Error
RAM	Random-Access Memory (a computer memory)
RDF	Resource Description Framework
RI	Research Infrastructures
RMSE	Root Mean Squared Error
RNN	Recurrent Neural Network
RSE	Relative Squared Error
R&D	Research and Development
SAM	Segment Anything Model
SOTA	State Of The Art
SQA	Software/Service Quality Assurance
SRE	Signal-to-Reconstruction Error Ratio
TB	Terabyte
TF	Tensorflow
UC	Use Case
US	User Story
UAV	Unmanned Aerial Vehicle
URL	Uniform Resource Locator
VGG	a series of neural networks developed by the Visual Geometry Group (Oxford)
VIA	VGG Image Annotator
VM	Virtual Machine

continues on next page

Table 1 – continued from previous page

Term / Acronym	Definition
VSCoDe	Visual Studio Code (IDE for coding)
WP	Work Package
XAI/xAI	eXplainable Artificial Intelligence
YOLO	You Only Look at Once, algorithm for object detection

EXTERNAL LINKS AND RESOURCES

This is a compendium of all the platform production services.

Homepage

A high level overview of the project.

Documentation

The main source of knowledge on how to use the project. Refer always to here in case of doubt.

Dashboard (AI4EOSC, iImagine, AI4Life, Tutorials)

View the catalog of AI modules and make deployments.

NextCloud

The service that allows to store your data remotely and access them from inside your deployment.

Github (software)

The code of the software powering the platform.

Github (modules)

The code of all the modules available in the platform.

DockerHub

Where the Docker images of the modules are stored.

Harbor

Custom Docker image registry we deployed to overcome DockerHub limitations.

CI/CD pipeline

Continuous Integration and Continuous Development Jenkins instance to keep everything up-to-date with latest code changes.

Status of services

Check if a specific platform service might be down for some reason.

Module template

Create new modules based on our project's template.

MLflow server (AI4EOSC, iImagine)

Log your trainings parameters and models with our MLflow server.

DriftWatch

Monitor you inference models to detect drift.

Inference platform (OSCAR) (AI4EOSC, iImagine)

Scalable serverless inference of AI models.

Inference pipelines platform (Flowfuse)

Compose custom AI inference pipelines.

LLM Chatbot

Chat with our LLM bot and interact live with our documentation.

Youtube channel

Find video-tutorials and more.

USEFUL MACHINE LEARNING RESOURCES

This is a piece of documentation trying to offer some advice on tools to use to answer common problems (non ML expert) users might face.

20.1 AI4EOSC webinars

The AI4EOSC project has organized a series of webinars on the use of the platform (based on the AI4OS software stack), AI, machine learning, deep learning, image processing, image segmentation and other relevant topics. These can be accessed on YouTube at the following links:

- [Introduction to the platform](#)
- [Image processing with AI4EOSC](#)

Warning

Please, be aware that video demos can become quickly outdated. In case of doubt, always refer to the written documentation.

20.2 Tutorials

Here are some basic resources to get you quickly started in the Deep Learning / Machine Learning world.

20.2.1 Books

- *Deep Learning with Python*, F. Chollet
- *The FastAI book*
- *Deep Learning Book*, Ian Goodfellow

20.2.2 Courses

- [Google Machine Learning Crash Course](#)
- [Machine Learning Mastery](#)
- [DAIR ML YouTube Courses](#)
- [Stanford CS231N \(2017\): Introduction to Convolutional Neural Networks for Visual Recognition](#)
- [Stanford CS230 \(2018\): Deep Learning](#)

20.3 Datasets

20.3.1 Dataset labeling

Some tools to help you getting started creating your dataset.

- [CVAT](#) - Image annotation tool (with integration with the Segment Anything Model)
- [LabelStudio](#) - General annotation (text, images, etc)
- [LabelImg](#) - Image annotation
- [refinery](#) - Labeling for NLP
- [superintendent](#) - ipywidget-based interactive labelling tool for your data.
- [VGG Image Annotator \(VIA\)](#) - Image annotation
- [Biigle](#) - Web based annotation and exploration of images and videos
- [Roboflow](#) - only free if your dataset is public
- [Labelbox](#) - paid tool (free with educational license)

20.3.2 Find a dataset

If you don't have any data, try find an open dataset that suits you.

- [Google Dataset search](#)
- [Graviti Open Datasets](#)
- [DataHub](#)
- [Kaggle](#)
- [Paperwithcode Datasets](#)
- [OpenML datasets](#)
- [Dataworld datasets](#)
- [Eden library](#) - agriculture datasets

20.3.3 Explore your dataset

Less make sure the dataset does not contain errors.

- [Google's Know your data](#) - only valid for common Tensorflow Datasets
- [Sweetviz](#) - explore and compare tabular data
- [cleanlab](#) - dataset cleaning
- [FastDup](#) - dataset cleaning. Find anomalies, duplicate and near duplicate images, clusters of similarity, broken images, image statistics, wrong labels.
- [deepchecks](#) - checks related to various types of issues, such as model performance, data integrity, distribution mismatches, and more.
- [kangas](#) - exploring, analyzing, and visualizing large-scale multimedia data
- [Impute](#) - missing data

20.3.4 Feature selection

Some times less is more. Learn how to select the appropriate features of your dataset.

- [sklearn - feature selection](#)
- [mlxtend](#)

20.3.5 Imbalanced learning

Do you have too much data from one class and too few from others. Let's balance things out!

- [Sklearn imbalanced](#)

20.3.6 Data augmentation

Do you have few data? Make the most out of it!

- [Augly](#) - General augmentation (text, images, etc.)
- [imgaug](#) - Image augmentation

20.3.7 Dataset shift

Is your dataset likely to degrade over time (eg. cam gets dirty). Keep on eye on it!

- [Frouros](#)
- [Alibi-detect](#)
- [Avalanche](#) - Continual Learning library based on Pytorch
- [River](#) - Online learning
- [Cinnamon](#)
- [Eurybia](#)

20.4 Model development

If you want to develop a model from scratch don't try to be a hero! [Papers with Code](#) gathers top performing models for multiple tasks with their corresponding code. Reuse them for your usecases! Try not to look for the top model but for the one with the cleanest code.

If you want nevertheless develop your model from scratch here are some recommendations.

20.4.1 Tensorflow related

- [Sonnet](#)
- [TensorLayer](#)

Extensions for JAX:

- [Flax](#) - NN library for JAX
- [Haiku](#) - Sonnet for JAX
- [Trax](#) - like Keras for advanced deep learning
- [Difffrax](#) - Numerical differential equation solvers in JAX
- Others: [Equinox](#)

Tensorflow extensions:

- [Tensorflow Quantum](#)
- [TensorFlow Probability](#) - probabilistic reasoning and statistical analysis.
- [Tensorflow Graph Neural Networks](#)
- [Tensorflow Model Optimization Pruning API](#) - build sparse models
- [Tensorflow Similarity](#) - Entrenar modelos similares. Sirve también para Self Supervised Learning ([ref](#)).
- [TF-GAN](#) - reproducible GANs
- [Tensorflow Ranking](#) - recommender systems
- [TFX](#) - production applications

Tensorflow tutorials:

- <https://github.com/vahidk/EffectiveTensorflow>

20.4.2 Pytorch related

- [Pytorch Lightning](#)
- [Pytorch image models](#) - the largest collection of PyTorch image encoders / backbones. Including train, eval, inference, export scripts, and pretrained weights. Integrates the `timm` library
- [Composer](#) - library with raining methods and best practices for efficient training
- [Torchdim](#)
- [PyTorch Tabular](#) - for tabular data
- [Pyro](#) - Probabilistic programming
- [VISSL](#) - Self Supervised Learning
- [TorchSSL](#) - Semi Supervised Learning
- [Kornia](#) - acelerar image processing con GPUs. Lo puedo usar para depth estimation
- [Torch Geo](#) - Geospatial ML
- [TorchMetrics](#)
- [torchtyping](#) - enforce torch type checks
- [lovelytensors](#) - human friendly debugging of torch tensors
- [skorch](#) - A scikit-learn compatible neural network library that wraps PyTorch.
- [Additional optimizers](#)

Pytorch tutorials

- <https://pythonrepo.com/repo/ritchieng-the-incredible-pytorch>
- <https://github.com/yunjey/pytorch-tutorial>

20.5 Other

20.5.1 Computing

Some useful non-AI packages to run computations:

- [numba](#) - see `@jit` decorator

- [cython](#)
- [numpy](#) - *important*: Install OPENBLAS with Numpy to accelerate computation
- [pandas](#)
- [xarray](#) - work better with multidimensional array by labelling dimensions
- [numexpr](#) - accelerate Numpy computations
- [intelext](#) - Intel extension to accelerate sklearn
- [dask](#) - parallel computation
- [fugue](#) - execute Python, pandas, and SQL code on Spark, Dask and Ray without rewrites
- [FAISS](#) - efficient similarity search and clustering of dense vectors

20.5.2 GPU acceleration

Some packages to accelerate non-AI operations with GPUs.

- [pycuda](#)
- [triton](#) - simple high performance GPU programming (openai)

You can use GPU based alternatives of common libraries for faster performance:

- [cudf](#) - alternative to Pandas
- [cuml](#) - alternative to sklearn
- [cusignal](#) - alternative to scipy signal
- [cugraph](#) - for graph algorithms
- [cupatial](#) - for geospatial operations
- [cuxfilter](#) - accelerate visualization (Bokeh, DataShader, Panel, Falcon, Jupyter)

20.5.3 Training monitoring

Let's keep an eye on the training status.

- [Tensorboard](#) - only works with Tensorflow
- [TensorboardX](#) - framework agnostic
- [LabML](#)

20.5.4 Training debugging

Is your training failing for some reason?

- [Netron](#) - visualize DL models
- [Cockpit](#) - debug training

20.5.5 Model optimization

Do you need your model to go faster?

- [VoltaML](#) - accelerate ML models with a single line of code
- [sparse-ml](#)
- [deep-sparse](#)

- [Pytorch quantization](#)
- [AItemplate](#) - transforms deep neural networks into CUDA (NVIDIA GPU) / HIP (AMD GPU) C++ code for lightning-fast inference serving
- [Hummingbird](#) - transform traditional ML models (eg. Random Forest) to neural networks, and benefit from hardware acceleration

VIDEO DEMOS

Please refer to the [AI4EOSC Youtube channel](#) for a comprehensive list of video demos.

 Warning

Please, be aware that video demos can become quickly outdated. In case of doubt, always refer to the written documentation.

DEEPAAS API

The **DEEPaaS API** enables a user friendly interaction with the underlying Deep Learning modules and can be used both for training models and doing inference with services. It provides some common methods to query all the modules in the Marketplace.

For a detailed up-to-date documentation please refer to the [official DEEPaaS documentation](#).

22.1 Integrate your model with the API

The best approach to integrate your code with DEEPaaS is to create an empty template using the AI Modules Template. This will take care of creating a Python package of your model with all the appropriate structure for your model (entrypoints, files, etc).

Once the template is created, install the package in editable mode and move your existing code (if any) it.

```
$ pip install -e .
```

To define the API methods that will interface with your existing code, we have to modify the `api.py` file. The **API methods** you can define are:

- **Enable prediction:** implement `get_predict_args()` and `predict()`.
- **Enable training:** implement `get_train_args()` and `train()`.
- **Enable model weights preloading:** implement `warm()`.

You don't need to define all the methods, just the ones you need. Every other method will return a `NotImplementError` when queried from the API. The `get_metadata()` should be already defined for you.

If you don't feel like reading the DEEPaaS docs (which you should), we recommend taking a look at the [AI4OS demo app](#). There, you will be able to see examples on:

- how to define your predict function, with multiple types of inputs and outputs.
- how to choose to return different formats, be it a JSON or a file (eg. image, zip). If you choose to return a JSON, please define a `schema` to validate the output predictions.
- how to define a training function that logs metrics into Tensorboard monitoring.

Decoupling code and API

Try to keep your module's code as decoupled as possible from DEEPaaS code, so that any future changes in the API are easy to integrate. This means that the `predict()` in `api.py` should mostly be an interface to your true predict function. In pseudocode:

```
#api.py
import utils # eg. this is where your true predict function is

def predict(**kwargs):
    args = preprocess(kwargs) # transform deepaas input to your standard input
    out = utils.predict(args) # make prediction
    resp = postprocess(out)    # transform your standard output to deepaas output
    return resp
```

22.1.1 Additional considerations

The values you use in your `get_predict_args()` will be used to generate the Gradio UI in *Try-me deployments*.

In particular, this affects to how files (`webargs.fields.Field`) will be rendered in the UI:

- if `image` is found in description, it will be rendered as a Gradio image
- if `audio` is found in description, it will be rendered as a Gradio audio
- if `video` is found in description, it will be rendered as a Gradio video
- if `#noparse` is found in description, it will be rendered as a Gradio generic file
- if more than one is found in description (eg. `image` and `video`), it will be rendered as a Gradio generic file
- if no keyword is found, it will be rendered as a Gradio generic file

```
def get_predict_args():
    arg_dict = {
        "demo_image": fields.Field(
            required=True,
            type="file",
            location="form",
            description="test upload",
            # "image" not in description, thus rendered as a generic file in the Gradio_
↪ UI
        ),
        "demo_image_1": fields.Field(
            required=True,
            type="file",
            location="form",
            description="test image upload",
            # "image" is indeed in description, thus rendered as an image in the Gradio_
↪ UI
        ),
    }
    return arg_dict
```

In addition, if you have not defined a schema to validate your JSON response, the output in the UI will be the plain JSON response, instead of a feature-rich UI. The same point about rendering files also applies here.

```
schema = {
    "demo_image_2": fields.Str(
        description="some file"
    ), # "image" not in description, thus rendered as a generic file in the Gradio UI
    "demo_image_3": fields.Str(
```

(continues on next page)

(continued from previous page)

```

        description="some image file"
    ), # "image" is indeed in description, thus rendered as an image in the Gradio UI
    "accept": fields.Str(),
}

```

22.2 Running the API

To start the API run:

```
$ deepaas-run --listen-ip 0.0.0.0
```

and go to <http://0.0.0.0:5000/ui>. You will see the Swagger UI with all the methods:

models		
GET	/v2/models/	Return loaded models and its information
GET	/v2/models/imgclas/	Return model's metadata
POST	/v2/models/imgclas/train/	Retrain model with available data
GET	/v2/models/imgclas/train/	Get a list of trainings (running or completed)
GET	/v2/models/imgclas/train/{uuid}	Get status of a training
DELETE	/v2/models/imgclas/train/{uuid}	Cancel a running training
POST	/v2/models/imgclas/predict/	Make a prediction given the input data

If running the API from inside a module's Docker container, you can use the command:

```
$ deep-start --deepaas
```

Troubleshooting: I only see a module called deepaas-test

It can happen that when you launch deepaas, you cannot find your module in the UI and see something like the following instead:

models

GET	/v2/models/	Return loaded models and its information
GET	/v2/models/deepaas-test/	Return model's metadata
POST	/v2/models/deepaas-test/train/	Retrain model with available data
GET	/v2/models/deepaas-test/train/	Get a list of trainings (running or completed)
GET	/v2/models/deepaas-test/train/{uuid}	Get status of a training
DELETE	/v2/models/deepaas-test/train/{uuid}	Cancel a running training
POST	/v2/models/deepaas-test/predict/	Make a prediction given the input data

This mean that deepaas had problems to load your module and loaded instead the dummy module (deepaas-test). You can usually debug by running your module's API script and looking at the error it raised:

```
$ python ai4os_demo_app/api.py
File "ai4os_demo_app/api.py", line 48
    var =
        ^
SyntaxError: invalid syntax
```

Once fixed, launch deepaas again and your module should appear.

HOW TO USE NEXTCLOUD WITH RCLONE

rclone is a tool that enables you to synchronize contents between your machine and a remote storage. It is kind of an **rsync** but for remote storages. Although we will demonstrate here how to use it with Nextcloud, it can be used with many different remote storages (Dropbox, Google Drive, Amazon S3, etc)

23.1 1. Installing rclone

All applications in the *Dashboard* are packed in a Docker image and have **rclone** installed by default. If you want to create a Docker containing your own application, you should install rclone in the container to be able to access the data stored remotely. When developing an application with the AI Modules Template, the Dockerfile already includes installation of rclone.

To install rclone on a Docker container you should add the following code to your Dockerfile:

```
RUN curl https://rclone.org/install.sh | bash
```

To install it locally on your machine:

```
$ curl https://rclone.org/install.sh | sudo bash
```

23.2 2. Configuring rclone

23.2.1 ... in a Dashboard deployment

Before creating the deployment you must do the following:

1. *Sync Nextcloud from the profile section*. This will create your rclone credentials.
2. *Select Nextcloud when configuring the deployment*. This will set up rclone configuration for you via setting environment variables in your deployment.

When the deployment is created, you should have your env variables available to work with the latest RCLONE:

```
$ printenv | grep RCLONE_CONFIG_RSHARE_  
RCLONE_CONFIG_RSHARE_VENDOR=nextcloud  
RCLONE_CONFIG_RSHARE_PASS=<YOUR-PASSWORD>  
RCLONE_CONFIG_RSHARE_URL=https://share.cloud.ai4eosc.eu/remote.php/dav/files/<YOUR-USER>  
RCLONE_CONFIG_RSHARE_TYPE=webdav  
RCLONE_CONFIG_RSHARE_USER=<YOUR-USER>
```

A note on older RCLONE versions

In RCLONE versions $\leq 1.62.2$, the RCLONE environment variables had the following shape ([issue](#)).

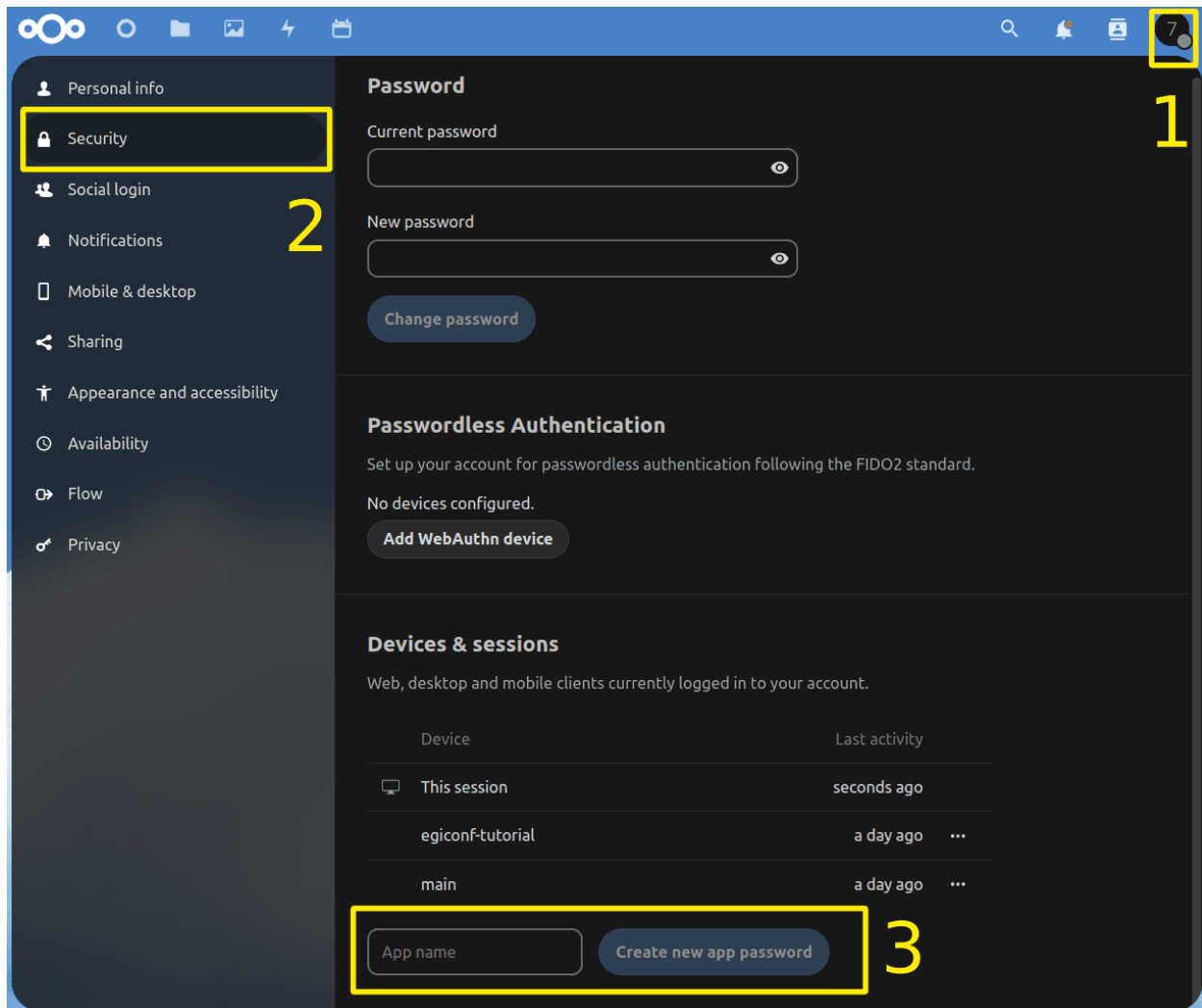
```
RCLONE_CONFIG_RSHARE_VENDOR=nextcloud
RCLONE_CONFIG_RSHARE_PASS=<YOUR-PASSWORD>
RCLONE_CONFIG_RSHARE_URL=https://share.cloud.ai4eosc.eu/remote.php/webdav/
RCLONE_CONFIG_RSHARE_TYPE=webdav
RCLONE_CONFIG_RSHARE_USER=<YOUR-USER>
```

So in your module uses an old version of RCLONE, you can either:

1. Update to a newer RCLONE version,
2. Still use an old version but make sure to update your `.bashrc` with these variables.

23.2.2 ... in your local machine

First, you need to generate your RCLONE credentials. For this, log into the [AI4OS Nextcloud](#), go to (1) **Settings** (top right corner) (2) **Security** (3) **Devices & sessions**. Set a name for your application (typically in the docs we will use `rshare`) and click on **Create new app password**. This will generate your `<user>` and `<password>` credentials. Your username should start with `EGI_Checkin-....`



Then run `rclone config` command, these are the answers you should provide:

```
$ rclone config
# choose "n" for "New remote"
# choose name for AI4OS Nextcloud --> rshare
# choose "Type of Storage" --> Webdav
# provide AI4OS Nextcloud URL for webdav access --> https://share.cloud.ai4eosc.eu/remote.
  ↳ php/dav/files/<YOUR-USER>
# choose Vendor --> Nextcloud
# specify "user" --> (see <user>` in "Configuring rclone" above).
# password --> y (Yes type in my own password)
# specify "password" --> (see <password>` in "Configuring rclone" above).
# bearer token --> ""
# Edit advanced config? --> n (No)
# Remote config --> y (Yes this is OK)
# Current remotes --> q (Quit config)
```

This will create an configuration file in `$HOME/.config/rclone/rclone.conf`:

```
[rshare]
type = webdav
url = https://share.cloud.ai4eosc.eu/remote.php/dav/files/<YOUR-USER>
vendor = nextcloud
user = <YOUR-USER>
pass = <YOUR-PASSWORD> --> this is equivalent to `rclone obscure <password>`
```

Security warning

For security reasons, the `rclone.conf` should never be saved as part of the Docker image. If you are running `rclone` from inside a Docker container, you should mount `rclone.conf` at runtime directly as a volume.

```
$ docker run -ti -v $HOSTDIR_WITH_RCLONE_CONF/rclone.conf:$HOME/.config/rclone/
  ↳ rclone.conf <your-docker-image>
```

One can also mount the `rclone.conf` file at a custom location and tell `rclone` where to find it:

```
$ docker run -ti -v $HOSTDIR_WITH_RCLONE_CONF/rclone.conf:/custom/path/to/rclone.conf
  ↳ <your-docker-image>
$ rclone --config /custom/path/to/rclone.conf
```

23.3 3. Using rclone

You can check that everything works fine with:

```
$ rclone listremotes    # check you don't have two remote storages with same name
$ rclone about rshare: # should output your used space in Nextcloud.
```

Troubleshooting duplicate remotes

If `listremotes` is listing two remotes with the same name you probably configured the `rclone` twice. Most likely you ran `rclone config` on a machine deployed with the Dashboard, so you have both the `env` and `rclone.conf` configurations.

To fix this, either remove the `env` variables (echo `unset` command into the `.bashrc`) or delete the `rclone.conf` file.

You can start copying files from your remote to your local:

```
$ rclone copy rshare:/some/remote/path /some/local/path
```

Uploading to Nextcloud can be particularly slow if your dataset is composed of lots of small files. Considering zipping your folder before uploading.

```
$ zip -r <foldername>.zip <foldername>  
$ unzip <foldername>.zip
```

RCLONE CODE EXAMPLES

24.1 Example code on usage of rclone from python

24.1.1 Simple example

A simple call of rclone from python is via `subprocess.Popen()`

```
import subprocess

# from "rshare" remote storage into the container
command = (['rclone', 'copy', 'rshare:/Datasets/dogs_breed/data', '/srv/dogs_breed_det/
↳data'])

result = subprocess.Popen(command, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
output, error = result.communicate()
```

Important

When deploying a module on the DEEP Pilot testbed, you pass rclone parameters e.g. `rclone_user` and `rclone_password` during the deployment. If you use our [general template](#), the name of the remote storage has to be `rshare` as in the example above (`rshare:/Datasets/dogs_breed/data`). If you create your own TOSCA template, you need to pay attention on matching these names in your code and in the template (for example, see environment parameters in the [general template](#) like `RCLONE_CONFIG_RSHARE_USER` etc).

24.1.2 Advanced examples

More advanced usage includes calling rclone with various options (`ls`, `copy`, `check`) in order to check file existence at Source, check if after copying two versions match exactly.

- `rclone_call`

```
def rclone_call(src_path, dest_dir, cmd = 'copy', get_output=False):
    """ Function
        rclone calls
    """
    if cmd == 'copy':
        command = (['rclone', 'copy', '--progress', src_path, dest_dir])
    elif cmd == 'ls':
        command = (['rclone', 'ls', src_path])
    elif cmd == 'check':
```

(continues on next page)

(continued from previous page)

```

        command = (['rclone', 'check', src_path, dest_dir])

        if get_output:
            result = subprocess.Popen(command, stdout=subprocess.PIPE, stderr=subprocess.
↪PIPE)
        else:
            result = subprocess.Popen(command, stderr=subprocess.PIPE)
            output, error = result.communicate()
        return output, error

```

- rclone_copy

```

def rclone_copy(src_path, dest_dir, src_type='file', verbose=False):
    """ Function for rclone call to copy data (sync?)
    :param src_path: full path to source (file or directory)
    :param dest_dir: full path to destination directory (not file!)
    :param src_type: if source is file (default) or directory
    :return: if destination was downloaded, and possible error
    """

    error_out = None

    if src_type == 'file':
        src_dir = os.path.dirname(src_path)
        dest_file = src_path.split('/')[-1]
        dest_path = os.path.join(dest_dir, dest_file)
    else:
        src_dir = src_path
        dest_path = dest_dir

    # check first if we find src_path
    output, error = rclone_call(src_path, dest_dir, cmd='ls')
    if error:
        print(['ERROR, rclone_copy()] %s (src):\n%s' % (src_path, error))
        error_out = error
        dest_exist = False
    else:
        # if src_path exists, copy it
        output, error = rclone_call(src_path, dest_dir, cmd='copy')
        if not error:
            output, error = rclone_call(dest_path, dest_dir,
                                       cmd='ls', get_output=True)
            file_size = [ elem for elem in output.split(' ') if elem.isdigit() ][0]
            print(['INFO Copied to %s %s bytes' % (dest_path, file_size)])
            dest_exist = True
            if verbose:
                # compare two directories, if copied file appears in output
                # as not found or not matching -> Error
                print(['INFO] File %s copied. Check if (src) and (dest) really match..'
↪% (dest_file)])
            output, error = rclone_call(src_dir, dest_dir, cmd='check')
            if 'ERROR : ' + dest_file in error:

```

(continues on next page)

(continued from previous page)

```

        print('[ERROR, rclone_copy()] %s (src) and %s (dest) do not match!'
              % (src_path, dest_path))
        error_out = 'Copy failed: ' + src_path + ' (src) and ' + \
                    dest_path + ' (dest) do not match'
        dest_exist = False
    else:
        print('[ERROR, rclone_copy()] %s (src):\n%s' % (dest_path, error))
        error_out = error
        dest_exist = False

    return dest_exist, error_out

```

```

import subprocess

def sync_nextcloud(frompath, topath):
    """
    Mount a NextCloud folder in your local machine or viceversa.
    """
    command = (['rclone', 'copy', frompath, topath])
    result = subprocess.Popen(command, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
    output, error = result.communicate()
    if error:
        warnings.warn("Error while mounting NextCloud: {}".format(error))
    return output, error

sync_nextcloud('rshare:/your/dataset/folder', '/your/data/path/inside/the/container') #_
↪ sync local with nextcloud
sync_nextcloud('/your/data/path/inside/the/container', 'rshare:/your/dataset/folder') #_
↪ sync nextcloud with local

```


SUPPORT A NEW PROJECT

These are the steps required to onboard a new project:

- **support the new project authentication:**
 - Option 1: create a new VO in EGI Check-In (eg. `vo.ai4life.eu`),
 - Option 2: federate their authentication in Keycloak,
- **support the new Dashboard domain:**
 - create the domain in the DNS (eg. `https://ai4life.cloud.ai4eosc.eu`),
 - add the domain in the Cloud proxy, generate LetsEncrypt certs,
 - add the domain in the Keycloak Dashboard client,
- **adapt Nomad:**
 - create new Nomad namespace (eg. `ai4life`),
 - add new Nomad CPU/GPU nodes supporting that namespace,
- adapt PAPI to support new VO,
- deploy a dedicated Dashboard in the target domain,
- add the new dashboard to the docs,

Optional: * adapt OSCAR to start supporting new VO.

HOW TO ORGANIZE A TUTORIAL

- Your users should be in `training.egi.eu` VO.
- Then send an email to `giuseppe.larocca@egi.eu` with the complete list of participants so that he can approve them.
- Once your users are approved they will be able to access the *Tutorials Dashboard*. Bear in mind that the Tutorial Dashboard won't let them use GPUs (scarce resource!) and will have other small limitations in terms of RAM, CPUs and disk.

STORAGE PROVIDERS

This section explains additional information about the storage providers compatible with the platform.

27.1 AI4EOSC Storage

The [Storage](#) is a custom Nextcloud instance deployed for the project users. Each user gets a free 500 GB of storage there.

As expected, this storage is compatible by default with the platform.

Account validation

The first time you access this storage, you will need to contact the admins to validate your account.

27.2 Nextcloud

In order to link an account from a **custom Nextcloud** to the [Dashboard](#), you have to make sure that your Nextcloud allows requests from the different dashboard domains. This can be done in the Apache configuration file of your Nextcloud.

Here is an example which shows such configuration including the regular expression that encloses all the different domains that must be able to make requests.

```
<Directory "/var/www/nextcloud/">
  Options +FollowSymlinks
  AllowOverride All
  SetEnvIf Origin "^http(s)?://(.\+\.)?(dev.ai4eosc\.eu|cloud.ai4eosc\.eu|dev.imagine-ai\.  
→eu|cloud.imagine-ai\.eu)$" origin_is=$0
  <IfModule mod_headers.c>
    Header always set Strict-Transport-Security "max-age=15552000; includeSubdomains"
    Header always set Access-Control-Allow-Origin %{origin_is}e env=origin_is
    Header always set Access-Control-Allow-Credentials "true"
    Header always set Access-Control-Allow-Methods "PROPFIND"
    Header always set Access-Control-Allow-Headers "authorization,content-type"
  </IfModule>

  # Specific handling for OPTIONS requests
  <If "%{REQUEST_METHOD} == 'OPTIONS'">
    Header merge Access-Control-Allow-Origin %{origin_is}e env=PREFLIGHT_REQUEST_FROM_  
→WEBSITE
```

(continues on next page)

(continued from previous page)

```
Header merge Access-Control-Allow-Methods "PROPFIND" env=PREFLIGHT_REQUEST_FROM_
↪WEBSITE
Header merge Access-Control-Allow-Headers "authorization,content-type" env=PREFLIGHT_
↪REQUEST_FROM_WEBSITE
Header merge Access-Control-Allow-Credentials "true" env=PREFLIGHT_REQUEST_FROM_
↪WEBSITE
# Respond with 204 No Content for OPTIONS requests
# Note: This directive might not be respected depending on the server configuration.↪
↪and Apache version
RewriteEngine On
RewriteCond %{REQUEST_METHOD} OPTIONS
RewriteRule .* - [R=204,L]
</If>

<IfModule mod_dav.c>
  Dav off
</IfModule>

SetEnv HOME /var/www/nextcloud
SetEnv HTTP_HOME /var/www/nextcloud
</Directory>
```